

SCUOLA DI SCIENZE
Corso di Laurea in Informatica

**FEDERATED LEARNING VS.
KNOWLEDGE DISTILLATION:
CONFRONTO DI TECNICHE DI
APPRENDIMENTO DISTRIBUITO
E VALIDAZIONE
PER L'ANALISI DI ROUTINE
FISIOTERAPICHE**

Relatore:
Chiar.mo Prof.
MARCO DI FELICE

Presentata da:
MILENA MAZZA

Correlatore:
Dott.
ALFONSO ESPOSITO

Sessione II
Anno Accademico 2024-2025

Abstract

Negli ultimi anni, la crescente disponibilità di sensori e dispositivi intelligenti ha aperto nuove prospettive per la riabilitazione motoria assistita, permettendo di monitorare e valutare in modo automatico le prestazioni dei pazienti. Tuttavia, l'adozione di modelli di intelligenza artificiale in ambito clinico è spesso ostacolata dalla natura sensibile e distribuita dei dati sanitari, che ne limita la condivisione e complica l'addestramento centralizzato dei modelli.

Questa tesi affronta tali problematiche attraverso lo studio di metodologie di *Federated Learning* e *Knowledge Distillation*, con l'obiettivo di sviluppare sistemi di apprendimento cooperativo capaci di preservare la privacy dei dati e mantenere alte prestazioni predittive. Il lavoro si colloca all'interno del progetto I-TROPHYTS, dedicato alla realizzazione di un ecosistema intelligente per la riabilitazione motoria, basato su tecnologie *IoT*, robotica umanoide e analisi automatica dei segnali fisiologici.

Nel corso della ricerca sono state analizzate e confrontate diverse strategie di federated learning — *FedAvg*, *FedProx*, *FedAdam* e *FedAdagrad* — insieme a paradigmi distillativi sia centralizzati che federati. Gli esperimenti, condotti sui dataset *FEMNIST* e *IRDS*, hanno permesso di valutare le tecniche in condizioni realistiche di distribuzione non-IID, tipiche dei contesti clinici.

I risultati ottenuti mostrano che le strategie federate classiche, in particolare *FedAvg* e *FedProx*, garantiscono prestazioni più elevate e stabili rispetto alle varianti basate su ottimizzatori adattivi e agli approcci distillativi, con accuratezze superiori al 90% anche in presenza di forte eterogeneità statistica. Le tecniche di distillazione, pur offrendo vantaggi in termini di modularità e protezione della privacy, si sono dimostrate più sensibili alla qualità e alla rappresentatività del dataset di consenso, evidenziando limiti nella generalizzazione.

Nel complesso, la tesi conferma la solidità e l'affidabilità delle strategie federate classiche nei contesti clinici distribuiti, evidenziando come la distillazione della conoscenza, pur promettente, richieda ulteriori adattamenti metodologici per raggiungere la stessa efficacia in scenari reali e complessi.

Introduzione

Garantire un accesso equo e continuo alla fisioterapia rappresenta oggi una delle sfide più importanti per i sistemi sanitari moderni, in particolare in una società che vede un costante aumento dell'età media e delle patologie croniche legate alla sedentarietà. Negli ultimi decenni, l'invecchiamento della popolazione europea e la diffusione di stili di vita poco salutari hanno determinato un incremento del numero di persone affette da disturbi motori o da condizioni metaboliche come l'obesità, che necessitano di interventi riabilitativi mirati per recuperare funzionalità e autonomia. La fisioterapia si configura quindi come un elemento cardine per il mantenimento della salute, poiché contribuisce al recupero delle capacità motorie, alla prevenzione di complicanze e al miglioramento complessivo della qualità della vita, in particolare negli anziani e nei soggetti fragili.

L'emergenza sanitaria legata al COVID-19 ha messo in ulteriore evidenza la necessità di garantire la continuità delle cure anche a distanza e di ottimizzare il carico di lavoro di medici e fisioterapisti. In questo scenario, la telemedicina ha dimostrato di poter offrire un valido supporto, consentendo ai pazienti di proseguire i programmi riabilitativi da casa, sotto la supervisione remota del personale sanitario. La possibilità di accedere a trattamenti personalizzati direttamente nel proprio domicilio ha permesso di superare barriere logistiche e di mobilità, riducendo lo stress legato agli spostamenti e migliorando l'aderenza terapeutica. Nonostante i numerosi vantaggi, è importante sottolineare che la telemedicina deve essere considerata come un'integrazione e non come una sostituzione dell'intervento in presenza: la relazione diretta tra paziente e fisioterapista resta infatti un aspetto insostituibile per la valutazione clinica e per l'adattamento dinamico delle terapie. Le attuali linee guida nazionali stabiliscono, ad esempio, che la prima visita debba avvenire in presenza, e che le consultazioni da remoto non siano indicate in situazioni di emergenza o urgenza, in cui l'intervento diretto resta imprescindibile.

Sulla scia di queste evoluzioni tecnologiche si colloca il progetto **I-TROPHYTS** [23] (*IoT and humanoid RObotics for autonomous Physio-Therapeutic monitoring, coaching and supervising in smart Spaces*), finanziato dal *Ministero dell'Università e della Ricerca* (MUR). Il progetto nasce con l'intento di sviluppare un ecosistema tecnologico integrato per la riabilitazione motoria, basato sulla sinergia tra dispositivi IoT, robotica umanoide e tecniche avanzate di intelligenza artificiale. L'obiettivo è creare un ambiente riabilitativo intelligente, capace di monitorare in tempo reale lo stato fisico del paziente, guidare le sessioni di allenamento e supportare il fisioterapista nel controllo e nella personalizzazione del percorso terapeutico. Il sistema prevede l'interazione costante tra un robot umanoide e un spazio sensorizzato, dotato di una rete di sensori ambientali e dispositivi indossabili che raccolgono dati fisiologici, cinematici e psicofisici del soggetto durante l'attività motoria. Queste informazioni vengono elaborate per adattare le routine di esercizio, garantendo al contempo sicurezza, distanza fisica e possibilità di

intervento tempestivo in caso di condizioni anomale o situazioni di rischio.

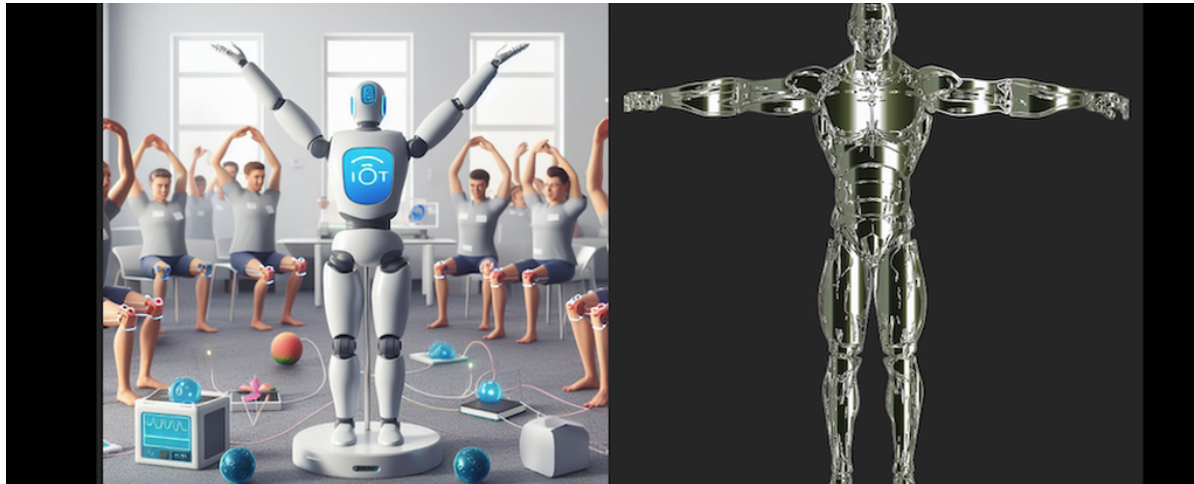


Fig. 1: Rappresentazione concettuale del progetto **I-TROPHYTS**, che integra robotica umanoide e tecnologie *IoT* per il supporto alla riabilitazione motoria.

All'interno di questo ampio quadro applicativo, il presente lavoro di tesi si concentra sulla componente di intelligenza artificiale per l'analisi distribuita dei dati clinici, con l'obiettivo di sviluppare e valutare modelli capaci di apprendere efficacemente da dati sensoriali eterogenei e distribuiti in modo non centralizzato. Nei contesti sanitari, infatti, la raccolta e l'elaborazione dei dati sono fortemente limitate da vincoli etici, legali e di privacy, che rendono complessa la condivisione diretta di informazioni sensibili tra strutture o pazienti. A ciò si aggiunge la marcata eterogeneità dei dati — dovuta a differenze tra dispositivi, condizioni cliniche e protocolli di acquisizione — che compromette le prestazioni dei modelli addestrati in modo convenzionale. L'apprendimento federato rappresenta quindi una soluzione naturale a tali problematiche, poiché consente di addestrare modelli globali mantenendo i dati localmente sui dispositivi o presso le strutture sanitarie, riducendo il rischio di violazioni della privacy e migliorando la generalizzazione su domini non omogenei.

Il punto di partenza metodologico di questa tesi è rappresentato dal framework *TL-VAE*, proposto da Esposito et al., che integra un estrattore di feature basato su MobileNetV2 congelata con un Class-Informed Variational Autoencoder addestrato in modo federato tramite *FedAvg*. Questa architettura ha dimostrato buone capacità nella costruzione di rappresentazioni latenti robuste in scenari non-IID, ma lascia aperte alcune questioni riguardanti l'impatto delle diverse strategie di aggregazione lato server e il confronto con approcci alternativi, come la distillazione della conoscenza. Il presente lavoro si propone dunque di indagare fino a che punto le strategie federate classiche (*FedAvg*, *FedProx*), le varianti basate su ottimizzatori (*FedAdam*, *FedAdagrad*) e i paradigmi distillativi possano affrontare le sfide poste dai dati clinici distribuiti ed eterogenei.

Per la validazione sperimentale sono stati utilizzati due dataset complementari: FEM-NIST, benchmark di riferimento per la ricerca nel campo del federated learning, e IRDS, corpus clinico composto da dati raccolti su pazienti reali durante esercizi riabilitativi, che riflette in modo realistico la complessità dei dati sanitari non-IID. La valutazione è stata condotta in termini di accuratezza, stabilità e capacità di generalizzazione, al fine di fornire una visione completa del comportamento delle diverse strategie nei contesti

clinici distribuiti.

La tesi si articola lungo un percorso che parte da una rassegna dello stato dell'arte, per poi passare alla progettazione, all'implementazione e infine alla valutazione sperimentale dei metodi proposti.

Il *Capitolo 1* è dedicato allo stato dell'arte e presenta i principali algoritmi di aggregazione federata e i più recenti approcci di distillazione della conoscenza.

Il *Capitolo 2* descrive la fase di progettazione sperimentale, introducendo i dataset, le architetture utilizzate e i paradigmi di riferimento.

Il *Capitolo 3* illustra i dettagli implementativi, dalle librerie adottate alle procedure di preprocessing, fino alla realizzazione dei sistemi federati e distillativi.

Il *Capitolo 4* raccoglie e discute i risultati sperimentali, analizzando le performance e le implicazioni pratiche delle diverse strategie.

Infine, il *Capitolo 5* conclude il lavoro sintetizzando i contributi principali e delineando possibili direzioni future di ricerca.

Indice

Abstract	1
Introduzione	2
1 Stato dell'arte	8
1.1 Apprendimento federato	9
1.2 Algoritmi federati classici	11
1.3 Algoritmi di distillazione	14
1.4 Algoritmi distillazione federata	16
1.5 Sintesi dello stato dell'arte	18
2 Progettazione	20
2.1 Obiettivo del lavoro	21
2.1.1 Dataset utilizzati	21
2.1.2 MobileNetV2 come estrattore di feature	25
2.2 Paradigma 1: Federated Learning classico	27
2.2.1 Fondamenti teorici e motivazioni	27
2.2.2 Architettura del sistema federato	28
2.2.3 CI-VAE: encoder, decoder e classificatore	29
2.2.4 Algoritmi di aggregazione lato server	31
2.2.5 Progettazione sperimentale	33
2.3 Paradigma 2: Knowledge Distillation	35
2.3.1 Motivazioni e principi concettuali	35
2.3.2 Distillazione della conoscenza: formulazione matematica	35
2.3.3 Architettura del paradigma di distillazione centralizzata	36
2.3.4 Progettazione sperimentale	38
2.4 Paradigma 3: Knowledge Distillation Federata	42
2.4.1 Architettura e flusso operativo (FedMD)	42
2.4.2 Progettazione sperimentale	44
2.4.3 Architettura e flusso operativo (FedKD)	47
2.4.4 Progettazione sperimentale	48
3 Implementazione	51
3.1 Tecnologie e librerie utilizzate	51
3.1.1 Python	51
3.1.2 PyTorch	52
3.1.3 Framework Flower (flwr)	52

3.1.4	Pandas	53
3.1.5	Libreria NumPy	53
3.1.6	Libreria scikit-learn	54
3.1.7	Modulo itertools	54
3.1.8	Libreria matplotlib	54
3.1.9	Libreria seaborn	55
3.1.10	Altre librerie ausiliarie	55
3.2	Preprocessing, partizionamento e caricamento dei dati	56
3.2.1	Estrazione delle feature con MobileNetV2	56
3.2.2	Partizionamento del dataset FEMNIST	56
3.2.3	Partizionamento del dataset IRDS	57
3.2.4	Funzione di caricamento unificata	58
3.3	Paradigma 1: Federated Learning classico	59
3.3.1	Architettura generale del sistema federato	59
3.3.2	Server e Strategie di aggregazione	59
3.3.3	Costruzione e comportamento del client federato	61
3.3.4	Costruzione dei modelli locali	64
3.3.5	Training e ottimizzazione locale	66
3.3.6	Funzione di loss composita	67
3.3.7	Validazione e metriche federate	68
3.3.8	Early stopping nel server federato	70
3.3.9	Logging avanzato dei risultati sperimentali	71
3.4	Paradigma 2: distillazione centralizzata	72
3.4.1	Struttura generale dell'esperimento	72
3.4.2	Architetture dei modelli	74
3.4.3	Meccanismo di distillazione	77
3.4.4	Adattamento della funzione di caricamento dati	78
3.5	Paradigma 3: Federated Model Distillation	80
3.5.1	Ciclo operativo del protocollo FedMD	80
3.5.2	Costruzione dei modelli locali	82
3.5.3	Generazione del dataset sintetico	84
3.5.4	Funzione di perdita nella distillazione federata	85
3.5.5	Salvataggio dei risultati e monitoraggio	86
3.6	Paradigma 3: FedKD	88
3.6.1	Struttura generale del codice	88
3.6.2	Struttura delle classi FedKDClient e FedKDServer	89
3.6.3	Modello condiviso: StudentModel	92
3.6.4	Distillazione della conoscenza	93
3.6.5	Logging, metriche e salvataggio dei risultati	94
4	Risultati	95
4.1	Metriche di valutazione	95
4.2	Paradigma 1: Federated Learning Classico	98
4.2.1	Risultati su FEMNIST	98
4.2.2	Risultati su IRDS	105
4.3	Paradigma 2: Distillazione Centralizzata	111
4.3.1	Risultati su FEMNIST	111

4.3.2	Risultati su IRDS con MLP	115
4.3.3	Risultati su IRDS con CI-VAE	118
4.4	Paradigma 3: FedMD	121
4.4.1	Risultati su FEMNIST	121
4.4.2	Risultati su IRDS	124
4.5	Paradigma 3: FedKD	127
4.5.1	Risultati su FEMNIST	127
4.5.2	Risultati su IRDS con FedKD	128
4.5.3	Confronto tra FedMD e FedKD	130
4.6	Confronto tra paradigmi	132
5	Conclusioni	135
5.1	Risultati principali	135
5.2	Implicazioni e Sviluppi Futuri	137

Capitolo 1

Stato dell'arte

L'apprendimento federato è un paradigma relativamente recente, nato per rispondere alla crescente esigenza di addestrare modelli di machine learning in contesti decentralizzati preservando al tempo stesso la privacy dei dati. In diversi scenari clinici o industriali, infatti, i dati sensibili non possono essere centralizzati per ragioni normative o pratiche, e devono quindi rimanere sui dispositivi che li generano. Questo vincolo ha stimolato lo sviluppo di una vasta gamma di algoritmi federati, volti a conciliare tre sfide principali: (i) l'eterogeneità statistica tra le distribuzioni locali; (ii) la limitata capacità computazionale dei dispositivi partecipanti, spesso eterogenei per risorse e architetture; (iii) la robustezza della cooperazione, ovvero la capacità di garantire convergenza e prestazioni elevate anche in condizioni di forte squilibrio dei dati.

Nel corso degli ultimi anni la letteratura ha proposto due linee principali di sviluppo. La prima, rappresentata dagli *algoritmi federati classici*, è centrata sulla definizione di strategie di aggregazione dei pesi che cercano di mitigare il fenomeno del *client drift* e accelerare la convergenza globale. La seconda, più recente, esplora il paradigma della *knowledge distillation federata*, in cui i client non condividono i parametri dei modelli ma soltanto informazioni derivate dalle predizioni, spesso su dataset pubblici o sintetici. Questi approcci permettono di superare i vincoli architetturali tipici del modello parametric sharing, garantendo maggiore flessibilità e, in alcuni casi, migliori proprietà di privacy.

Nel seguito di questo capitolo vengono presentati i principali algoritmi e contributi che costituiscono lo *stato dell'arte*, organizzati in quattro sezioni principali: (i) una sezione introduttiva dedicata all'*apprendimento federato*, che ne illustra i principi fondamentali e le principali prospettive teoriche e sistemiche; (ii) gli algoritmi di *federated learning* classici basati su aggregazione parametrica, che rappresentano le prime implementazioni operative del paradigma; (iii) gli approcci di *knowledge distillation* tradizionale, che introducono le basi teoriche del trasferimento di conoscenza tra modelli; e (iv) le varianti di *knowledge distillation federata*, che estendono tale principio a contesti distribuiti, con o senza l'impiego di dataset sintetici. Questa rassegna costituisce la base teorica da cui muove il presente lavoro di tesi, volto a confrontare e integrare tali strategie in scenari clinici reali e caratterizzati da dati non-IID.

1.1 Apprendimento federato

L'apprendimento federato (FL) è un paradigma di *machine learning* distribuito introdotto da McMahan con l'obiettivo di addestrare modelli condivisi mantenendo i dati presso i dispositivi che li generano. In uno schema tipico, ciascun partecipante (*client*) esegue localmente l'aggiornamento del proprio modello utilizzando dati privati e invia al server centrale soltanto informazioni derivate dal processo di ottimizzazione, come pesi o gradienti. Il server aggrega tali aggiornamenti per formare un modello globale, che viene poi ridistribuito ai client in un processo iterativo di apprendimento collaborativo. Come osservato da Kairouz et al. [9], il FL può essere formalmente definito come un “*collaborative machine learning framework under privacy and communication constraints*”, in cui si cerca di massimizzare la prestazione globale rispettando vincoli di riservatezza dei dati e di capacità di comunicazione limitata.

La validità del paradigma risiede nella sua capacità di fornire una struttura metodologica generale per l'apprendimento distribuito, combinando efficienza, privacy e scalabilità. Rispetto al machine learning centralizzato, l'FL consente di ottenere prestazioni comparabili pur riducendo il rischio di esposizione dei dati e la necessità di infrastrutture di memorizzazione centralizzate.

Negli anni successivi alla sua introduzione, la comunità scientifica ha approfondito sia gli aspetti teorici sia le questioni ingegneristiche del paradigma, ponendo le basi per la vasta famiglia di algoritmi federati che verranno analizzati nelle sezioni successive. Tra i contributi più influenti si collocano la survey sistematica di Li et al. [12], che fornisce una visione complessiva e strutturata dei sistemi federati, e la rassegna di Kairouz et al. che ne formalizza i fondamenti teorici e individua le principali sfide ancora aperte. Le analisi di questi due lavori, complementari tra loro, rappresentano oggi i riferimenti di base per comprendere la solidità e le prospettive evolutive del federated learning.

Nel lavoro “*Federated Learning: Challenges, Methods, and Future Directions*”, Li offre una delle trattazioni più complete del federated learning, inquadrandolo non soltanto come un problema di ottimizzazione distribuita, ma come un vero e proprio *paradigma di sistema*. Gli autori adottano una prospettiva di tipo *system-oriented*, che considera l'intero ecosistema federato: dalla distribuzione dei dati ai protocolli di comunicazione, fino ai meccanismi di sicurezza e alla scala della cooperazione. In questa visione, il successo di un sistema federato dipende dall'interazione bilanciata tra componenti algoritmiche, infrastrutturali e di privacy.

Il contributo principale del lavoro consiste nella definizione di una tassonomia che sintetizza le dimensioni fondamentali del paradigma: la modalità di distribuzione dei dati (*cross-silo* o *cross-device*), la tipologia di modello di apprendimento, i meccanismi di privacy e sicurezza adottati, l'architettura di comunicazione e sincronizzazione, la scala della federazione e la motivazione della cooperazione (centralizzata o decentralizzata). Questa struttura concettuale, oggi ampiamente utilizzata nella letteratura successiva, ha consentito di organizzare in modo coerente contributi eterogenei e di fornire un linguaggio comune per la progettazione e la valutazione dei sistemi federati.

Secondo Li et al., la progettazione di un sistema federato efficace deve bilanciare tre obiettivi chiave: efficienza computazionale e comunicativa, capacità predittiva del modello globale e protezione della privacy dei dati locali. La survey mostra che, in condizioni

favorevoli — distribuzioni dei dati moderatamente eterogenee e connettività stabile —, le prestazioni del modello federato possono eguagliare quelle centralizzate. Tuttavia, in presenza di forti squilibri statistici o di risorse limitate, emergono sfide rilevanti legate alla convergenza, alla stabilità e ai costi di comunicazione, che costituiscono tuttora linee di ricerca aperte.

Il lavoro di Li et al. è di particolare interesse per il presente contesto poiché stabilisce la cornice sistemica entro cui collocare gli algoritmi analizzati nei paragrafi successivi, evidenziando come la progettazione di un sistema federato non possa prescindere da una visione integrata di modello, infrastruttura e vincoli di privacy.

La prospettiva di Kairouz, esposta nell’articolo “*Advances and Open Problems in Federated Learning*”, è invece complementare, di natura più teorica, volta a formalizzare le basi del federated learning e a delinearne i problemi ancora irrisolti. Gli autori definiscono l’FL come un paradigma di apprendimento collaborativo decentralizzato che mira a ottimizzare un obiettivo globale condiviso mantenendo la località dei dati, e ne analizzano in modo sistematico gli sviluppi più significativi.

La survey individua quattro assi principali di ricerca: (i) la *convergenza teorica*, con particolare attenzione all’impatto dell’eterogeneità statistica e della partecipazione parziale dei client; (ii) l’*efficienza comunicativa*, che rappresenta un limite pratico nei sistemi su larga scala; (iii) la *privacy e sicurezza*, in relazione alle minacce di *model inversion* e alle contromisure basate su privacy differenziale e crittografia omomorfa; (iv) la *personalizzazione*, intesa come adattamento locale dei modelli globali a specifici domini o utenti.

Gli autori sottolineano inoltre come il federated learning stia evolvendo verso un paradigma più ampio, capace di integrare tecniche di *transfer learning*, *knowledge distillation* e *continual learning*, superando la concezione originaria di semplice mediazione di pesi. Questa visione consolida il federated learning come infrastruttura concettuale per l’intelligenza artificiale distribuita, in grado di coniugare efficienza, equità e protezione della privacy.

Nel complesso, il lavoro di Kairouz et al. contribuisce a definire il fondamento teorico del paradigma e a individuare le sue principali direzioni di evoluzione — aspetti che risultano particolarmente rilevanti nel presente contesto, poiché molte delle innovazioni più recenti, come la distillazione federata, derivano proprio da queste linee di sviluppo.

1.2 Algoritmi federati classici

Il primo lavoro di riferimento nell'ambito dell'apprendimento federato è *Federated Averaging* (**FedAvg**), introdotto da McMahan et al. [15]. L'algoritmo nasce come risposta alla necessità di preservare la privacy dei dati in scenari decentralizzati, riducendo al tempo stesso il costo comunicativo tipico di approcci distribuiti sincroni. In particolare, FedAvg affronta due problematiche centrali: (i) la presenza di dati *non indipendenti e non identicamente distribuiti* (non-IID) tra i client, che possono portare a modelli divergenti, e (ii) l'eterogeneità delle risorse computazionali, che rende impraticabile una sincronizzazione frequente dei gradienti su larga scala.

L'approccio proposto combina più epoche di ottimizzazione locale tramite SGD con un'aggregazione centrale basata su media pesata. Formalmente, dato un modello globale $w^{(t)}$ e un sottoinsieme di client $\mathcal{S}_t$ selezionato al round t , ciascun client k addestra localmente i propri parametri $w_k^{(t+1)}$ per E epoche sui dati $\mathcal{D}_k$; il server centrale aggiorna quindi il modello globale secondo:

$$w^{(t+1)} = \sum_{k \in \mathcal{S}_t} \frac{n_k}{\sum_{j \in \mathcal{S}_t} n_j} w_k^{(t+1)},$$

dove $n_k = |\mathcal{D}_k|$ rappresenta la cardinalità del dataset locale. Questa semplice modifica consente di ridurre drasticamente il numero di comunicazioni necessarie, poiché ogni client esegue più passi di aggiornamento prima di sincronizzarsi con il server.

I risultati sperimentali riportati nel lavoro originale dimostrano l'efficacia di questo schema su modelli di diversa natura. Ad esempio, su CIFAR-10, con una CNN, per raggiungere l'80% di accuratezza in test FedAvg richiede circa 280 round, a fronte dei 18 000 round di un SGD centralizzato e dei 3 750 round di FedSGD; il guadagno si mantiene anche a soglie di accuratezza più elevate, con un risparmio fino a due ordini di grandezza. Su Shakespeare, con un LSTM per language modeling, FedAvg garantisce la convergenza anche in condizioni di non-IID estremo (ogni client corrisponde a un singolo autore), mentre su EMNIST, con una CNN, produce prestazioni comparabili a quelle centralizzate pur mantenendo la separazione dei dati tra scriventi.

Nonostante la sua efficacia e semplicità implementativa, FedAvg presenta alcune criticità, in particolare la sensibilità all'eterogeneità statistica tra i client. In scenari fortemente non-IID, gli aggiornamenti prodotti dai singoli dispositivi possono divergere dall'ottimo globale, dando luogo al fenomeno noto come *client drift*. Questo non solo rallenta la convergenza, ma può anche condurre a modelli instabili o sub-ottimali. Per affrontare questa criticità, Li et al. [13] hanno introdotto *Federated Proximal* (**FedProx**), una generalizzazione di FedAvg che aggiunge un termine di regolarizzazione prossimale all'obiettivo locale di ciascun client.

La funzione di ottimizzazione diventa:

$$\min_w F_k(w) + \frac{\mu}{2} \|w - w^{(t)}\|_2^2,$$

dove $F_k(w)$ è la loss empirica sui dati del client k , $w^{(t)}$ rappresenta il modello globale ricevuto dal server, e $\mu \geq 0$ controlla l'intensità del vincolo. Questo termine limita la distanza tra modello locale e globale, riducendo l'impatto di aggiornamenti troppo divergenti. L'aggregazione lato server resta invariata rispetto a FedAvg, ossia una media

pesata dei parametri locali.

Gli autori hanno validato FedProx su diversi benchmark federati con modelli coerenti al dominio dei dati: CNN su FEMNIST (riconoscimento di caratteri manoscritti), LSTM su Shakespeare (modellazione del linguaggio a livello di carattere) e regressione logistica su Sentiment140 (classificazione del sentiment su Twitter). I risultati mostrano che FedProx fornisce un miglioramento sostanziale in condizioni di forte eterogeneità: in scenari non-IID estremi, FedProx riduce le oscillazioni e le instabilità osservate con FedAvg, con incrementi medi di accuratezza fino a 22 punti percentuali rispetto alla baseline. In scenari più omogenei, come quelli con distribuzioni locali meno divergenti, FedAvg e FedProx ottengono prestazioni comparabili, a conferma che il principale vantaggio di FedProx emerge quando la disomogeneità statistica tra i client è elevata.

Parallelamente, Reddi et al. [19] hanno evidenziato come l'aggregazione basata su media pesata non sia sempre ottimale per gestire la variabilità tra client e la dinamica dei gradienti. Per affrontare queste criticità, hanno proposto la famiglia *Federated Optimization* (**FedOpt**), che estende FedAvg reinterpretando la differenza tra modello globale e modelli locali come uno *pseudo-gradiente* da ottimizzare tramite algoritmi adattivi lato server.

Formalmente, lo scostamento medio tra modello globale e modelli locali al round t viene definito come:

$$\Delta^{(t)} = \sum_{k \in \mathcal{S}_t} \frac{n_k}{\sum_{j \in \mathcal{S}_t} n_j} (w^{(t)} - w_k^{(t+1)}),$$

e l'aggiornamento globale non è più una media pesata, ma avviene tramite un ottimizzatore adattivo **Opt** applicato a tale pseudo-gradiente:

$$w^{(t+1)} = w^{(t)} - \eta \cdot \text{Opt}(\Delta^{(t)}).$$

Le due istanze principali di questa famiglia sono FedAdam, che sfrutta i momenti esponenziali di primo e secondo ordine tipici di Adam, e FedAdagrad, che accumula i gradienti quadrati storici per modulare dinamicamente il passo di aggiornamento.

Gli autori hanno validato FedOpt su una serie di benchmark federati, utilizzando modelli coerenti con il dominio dei dati: CNN per CIFAR-10 e CIFAR-100, CNN per EMNIST (classificazione di caratteri manoscritti), LSTM per Shakespeare (modellazione del linguaggio a livello di carattere) e reti fully connected per Stack Overflow (predizione dei tag). I risultati sperimentali mostrano che FedOpt migliora sia la qualità finale dei modelli sia la rapidità di convergenza rispetto a FedAvg. In particolare, su CIFAR-10 e CIFAR-100 gli ottimizzatori adattivi come FedAdam e FedYogi raggiungono accuratèzze superiori e richiedono meno round per convergere; su EMNIST, pur con guadagni più contenuti, FedOpt garantisce maggiore stabilità; su Shakespeare, riduce la perplexity rispetto a FedAvg; infine, su Stack Overflow, gli adattivi ottengono incrementi significativi di Recall@k, evidenziando la capacità di gestire domini ad alta dimensionalità e fortemente sbilanciati. Questi risultati confermano che l'introduzione di ottimizzatori lato server permette di mitigare le instabilità tipiche del training federato classico e di migliorare le prestazioni complessive su più domini.

Un contributo recente di grande interesse è quello di Alfonso Esposito, Yasamin Moghbelan, Ivan Zyrianoff e Marco Di Felice [2], che hanno introdotto il framework *Transfer-Learning Variational Autoencoder* (**TL-VAE**) per migliorare l'apprendimento federato

in scenari caratterizzati da distribuzioni non-IID e dataset sbilanciati. Il problema affrontato riguarda la difficoltà di garantire robustezza e capacità di generalizzazione in ambienti clinici e IoT, dove i dati sono distribuiti tra dispositivi diversi e non possono essere centralizzati per ragioni di privacy.

L'architettura proposta combina una rete MobileNetV2 [21] pre-addestrata e congelata, utilizzata come estrattore di feature, con un *Class-Informed Variational Autoencoder* (CI-VAE) che opera nello spazio latente. In questo modo, l'encoder del CI-VAE non lavora direttamente sulle immagini grezze, ma su embedding di alto livello già ricchi di informazione semantica, riducendo la complessità computazionale e migliorando la generalizzazione a domini eterogenei. Il training federato avviene tramite la strategia FedAvg, aggregando i pesi dei modelli locali e restituendo un modello globale condiviso. La funzione di perdita integra tre componenti: errore di ricostruzione, regolarizzazione tramite divergenza di Kullback–Leibler e cross-entropy sul classificatore latente, con l'obiettivo di ottenere rappresentazioni latenti al contempo generative e discriminative.

Gli esperimenti riportati dagli autori hanno valutato TL-VAE su due dataset non-IID: FEMNIST, benchmark di riferimento nel federated learning, e IRDS, corpus clinico di esercizi fisioterapici acquisiti da sette pazienti. In entrambi i casi i dati sono stati suddivisi con split 70:30 per train/test e i client corrispondevano a scriventi (FEMNIST) o pazienti (IRDS). La valutazione è stata condotta confrontando TL-VAE con un approccio di federated transfer learning basato sullo stesso estrattore MobileNetV2 ma con un semplice classificatore lineare al posto del VAE.

I risultati hanno mostrato che TL-VAE supera nettamente la baseline: su FEMNIST raggiunge il 100% di accuratezza globale, mentre su IRDS ottiene il 91%, a fronte di valori significativamente inferiori del classificatore lineare. Inoltre, TL-VAE mantiene prestazioni elevate al crescere del numero di client (3, 5 e 7), confermando la sua capacità di gestire la frammentazione e l'eterogeneità dei dati. Questi risultati dimostrano che l'integrazione di transfer learning e CI-VAE consente di ottenere rappresentazioni latenti più robuste e generalizzabili, rendendo TL-VAE una soluzione promettente per scenari federati complessi, in particolare in ambito clinico e IoT.

Infine, spostiamo l'attenzione dalle formulazioni algoritmiche alle effettive applicazioni del paradigma federato in ambito clinico e riabilitativo, un settore che negli ultimi anni ha registrato un crescente interesse grazie alla possibilità di sfruttare dati sensibili nel rispetto dei vincoli di privacy e sicurezza propri del dominio sanitario. Jiang et al. [8] hanno proposto un sistema federato per la cooperazione tra robot riabilitativi, dimostrando che la condivisione di conoscenza tra dispositivi consente di migliorare la stima dei parametri motori e la personalizzazione delle terapie post-ictus. Analogamente, Razfar et al. [18] hanno sviluppato un modello federato per la valutazione automatica delle capacità motorie residue in pazienti colpiti da ictus, evidenziando come la collaborazione tra centri clinici possa rafforzare la coerenza diagnostica senza compromettere la riservatezza dei dati.

Nel complesso, questi contributi confermano la crescente rilevanza del federated learning nel settore sanitario e motivano l'esplorazione di tecniche robuste e adattive, in grado di coniugare efficacia predittiva, rispetto della privacy e integrazione nei flussi operativi clinici reali.

1.3 Algoritmi di distillazione

Il concetto di Knowledge Distillation (KD) è stato introdotto da Hinton et al. [5], con l'obiettivo di trasferire conoscenza da un modello complesso (*teacher*) a un modello più compatto (*student*). A differenza dei tradizionali approcci di compressione, la KD non si limita a copiare i parametri del modello più grande, ma ne riproduce il comportamento probabilistico, permettendo allo studente di apprendere anche le relazioni tra classi e non solo le etichette corrette. Per ottenere ciò, le predizioni del teacher vengono ammorbidite attraverso un parametro di temperatura T , così da fornire uno spettro informativo più ricco durante l'addestramento.

Nel lavoro originale, gli autori dimostrano che modelli compressi tramite KD possono mantenere fino al 97% delle prestazioni del modello teacher su dataset come CIFAR-10 e ImageNet, con una riduzione del numero di parametri fino a 1/10. La funzione di perdita è definita come combinazione tra la cross-entropy supervisionata e la divergenza di Kullback–Leibler (KLD) tra le predizioni teacher e student:

$$\mathcal{L}_{KD} = (1 - \lambda) \mathcal{H}(y, P_s) + \lambda T^2 \mathcal{H}(\sigma(\frac{z_t}{T}), \sigma(\frac{z_s}{T})),$$

dove z_t e z_s sono i logit del teacher e dello student, e T controlla la morbidezza delle probabilità. L'efficacia di questo metodo ha reso la KD una delle strategie più influenti per la compressione di modelli e il trasferimento di conoscenza in deep learning.

Un importante passo avanti rispetto al framework originario di Hinton è rappresentato da *FitNets*, proposto da Romero et al. [20]. L'obiettivo di questo lavoro è estendere la KD oltre le sole predizioni finali, introducendo un meccanismo di trasferimento basato sulle attivazioni intermedie del modello teacher. Questa strategia, nota come *hint-based training*, consente al modello student di apprendere non solo la distribuzione delle uscite, ma anche la struttura gerarchica interna delle rappresentazioni apprese dal teacher, migliorando la capacità di generalizzazione e la velocità di convergenza.

L'idea alla base di FitNets è che le rappresentazioni intermedie del teacher contengano informazione semantica utile a guidare lo student durante le prime fasi di addestramento, quando i gradienti provenienti dalla loss finale possono risultare deboli o instabili. Per realizzare tale trasferimento, gli autori introducono una funzione di allineamento tra gli strati intermedi (*hints*) del teacher e i corrispondenti strati (*guided layers*) dello student. Formalmente, data una coppia di feature map F_t e F_s , la loss di distillazione intermedia è definita come:

$$\mathcal{L}_{\text{hint}} = \frac{1}{2} \| G(F_s) - F_t \|_2^2,$$

dove $G(\cdot)$ è una trasformazione lineare (ad esempio una convoluzione 1×1) che adatta la dimensionalità delle feature dello student a quella del teacher. Questa componente si combina poi con la loss di classificazione tradizionale per l'ottimizzazione congiunta:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{cls}} + \beta \mathcal{L}_{\text{hint}},$$

dove β bilancia il contributo del segnale intermedio rispetto a quello supervisionato.

Gli esperimenti condotti su benchmark standard come CIFAR-10, CIFAR-100 e SVHN mostrano che FitNets consente di addestrare reti student più profonde ma con un numero di parametri ridotto fino a un terzo rispetto al teacher, mantenendo prestazioni

uguali o superiori. In particolare, su CIFAR-10 gli autori riportano un incremento medio dell'1–2% in accuratezza rispetto al teacher, mentre su SVHN la rete student, con 50% dei parametri in meno, raggiunge prestazioni comparabili al modello completo. Un risultato interessante riguarda l'efficienza del training: l'uso degli *hints* accelera la convergenza di oltre il 30%, riducendo la necessità di regolarizzatori complessi o di inizializzazioni pre-addestrate.

Il contributo di FitNets è stato determinante nello sviluppo successivo della distillazione multi-livello e delle strategie di *self-distillation*, in cui lo stesso modello agisce come teacher e student a diversi stadi dell'addestramento. Questo lavoro ha inoltre aperto la strada a numerose varianti di KD per architetture convolutional e transformer, e viene tuttora considerato un riferimento fondamentale per la distillazione in modelli profondi e risorse-constrained.

1.4 Algoritmi distillazione federata

La distillazione della conoscenza rappresenta una soluzione efficace per ridurre la complessità dei modelli e favorire la cooperazione tra reti eterogenee. Tuttavia, la sua applicazione diretta in contesti decentralizzati presenta limiti legati alla necessità di dati condivisi e alla possibile esposizione di informazioni sensibili. Per risolvere tali criticità, la ricerca più recente ha introdotto il paradigma della *Federated Knowledge Distillation*, che sposta la cooperazione dal livello dei parametri a quello delle predizioni, garantendo un maggiore livello di privacy e una più ampia flessibilità architetturale. In tale schema, i modelli locali non condividono pesi o gradienti, ma soltanto informazioni derivate dalle proprie predizioni — ad esempio probabilità di classe o logit — che vengono aggregate e utilizzate per aggiornare un modello globale o per favorire l'apprendimento reciproco tra i partecipanti.

Diversi studi hanno validato l'efficacia generale di questo paradigma, evidenziandone sia l'efficienza sia la capacità di preservare la privacy dei dati locali. In particolare, Itahara et al. [7] hanno mostrato che la comunicazione basata su predizioni consente di ridurre significativamente la banda richiesta rispetto al federated learning tradizionale, mantenendo al contempo una buona stabilità del training anche in condizioni di partecipazione asincrona.

Successivamente, Li et al. [11] hanno introdotto il framework *Federated Model Distillation* (**FedMD**), che ha rappresentato un punto di svolta nel collegare apprendimento federato e distillazione e dimostra come la cooperazione tramite un piccolo dataset condiviso possa garantire prestazioni competitive pur supportando modelli eterogenei.

Il problema affrontato riguarda la rigidità degli algoritmi di aggregazione dei parametri, che richiedono architetture identiche su tutti i client e possono risultare inefficaci quando i modelli sono eterogenei o i dispositivi hanno risorse limitate. Inoltre, la condivisione diretta dei pesi può comportare rischi di privacy leakage, anche in assenza di accesso ai dati grezzi.

FedMD propone di spostare la cooperazione dal livello dei parametri a quello delle predizioni. Ciascun client addestra localmente il proprio modello su dati privati e produce predizioni su un *dataset pubblico* condiviso, $\mathcal{D}_{\text{pub}}$. Il server aggrega queste distribuzioni calcolando, per ciascun campione $x \in \mathcal{D}_{\text{pub}}$, una media delle probabilità predette:

$$\bar{P}(x) = \frac{1}{K} \sum_{k=1}^K P_k(x),$$

dove $P_k(x)$ rappresenta la predizione del modello locale f_k . Le distribuzioni aggregate vengono poi ridistribuite ai client, che aggiornano i propri modelli minimizzando la divergenza di Kullback–Leibler rispetto a $\bar{P}(x)$. Questo schema consente la cooperazione anche tra reti neurali con architetture molto diverse, senza condividere né dati né pesi.

I risultati sperimentali evidenziano che il framework FedMD consente miglioramenti significativi rispetto al semplice *transfer learning* locale. In particolare, su MNIST/FEM-NIST, dove ciascun partecipante dispone di dati limitati e potenzialmente non-IID (ad esempio lettere scritte da un singolo autore), la fase di distillazione collaborativa ha permesso ai modelli locali di ottenere incrementi di accuratezza test dell'ordine del 20%

rispetto al training isolato, riducendo sensibilmente il divario rispetto a uno scenario centralizzato. Un comportamento analogo è stato osservato su CIFAR-100/CIFAR-10, in cui i client ricevevano dati appartenenti solo a specifiche sottoclassi: in questo caso, la cooperazione ha consentito ai modelli di generalizzare anche verso sottoclassi mai viste durante il training locale, raggiungendo prestazioni superiori al transfer learning puro. In entrambi gli ambienti sperimentali, la distillazione iterativa ha mostrato una convergenza rapida e stabile, a conferma della capacità del metodo di combinare contributi eterogenei senza richiedere la condivisione diretta dei parametri dei modelli.

Un limite evidente di FedMD è la necessità di disporre di un dataset pubblico realmente condivisibile, condizione spesso irrealistica in scenari clinici o sensibili alla privacy. Per superare questa criticità, Lin et al. [14] hanno proposto il paradigma **FedDF**, che elimina la dipendenza da dati esterni sfruttando campioni sintetici generati a partire da modelli generativi.

L'idea alla base è di combinare la diversità dei modelli locali con la flessibilità di generatori neurali (ad esempio GAN o VAE) per costruire un dataset condiviso artificiale. Ogni client addestra il proprio modello discriminativo sui dati privati e produce predizioni su esempi sintetici. Il server aggrega queste predizioni ottenendo distribuzioni di consenso, che vengono poi utilizzate per addestrare o aggiornare un modello globale tramite knowledge distillation. In questo modo, la cooperazione non richiede più dati pubblici reali: il dataset sintetico diventa il canale informativo comune.

Gli autori hanno validato FedDF in scenari federati eterogenei utilizzando benchmark come CIFAR-10, CIFAR-100 e ImageNet, con client che adottavano architetture diverse (ResNet-20, ResNet-32 e ShuffleNetV2). I risultati mostrano che FedDF supera costantemente FedAvg sia in termini di accuratezza sia di stabilità, mantenendo una varianza sensibilmente inferiore tra le diverse esecuzioni. Su CIFAR-10 e CIFAR-100, FedDF produce un modello globale con prestazioni vicine a quelle dell'ensemble dei modelli locali (considerato upper bound), dimostrando la capacità del metodo di trasferire efficacemente conoscenza da architetture eterogenee. Nel caso di ImageNet, il modello distillato mantiene gran parte della qualità del modello centralizzato, dimostrando la scalabilità dell'approccio anche in scenari ad alta complessità. Un aspetto rilevante emerso dallo studio è la dipendenza dalla qualità del dataset di distillazione: quando si utilizzano sorgenti non etichettate più ricche (ad esempio STL-10 o ImageNet32), il modello globale distillato raggiunge prestazioni sensibilmente superiori rispetto all'uso di dataset meno rappresentativi. Complessivamente, FedDF conferma la validità della distillazione da ensemble come alternativa alle strategie classiche di aggregazione dei parametri, offrendo accuratezza più elevata e maggiore robustezza nei contesti non-IID.

Accanto a FedMD e FedDF, negli ultimi anni sono emerse varianti che mirano a ridurre ulteriormente i costi computazionali e migliorare la generalizzazione in scenari non-IID. Due contributi particolarmente rilevanti in questa direzione sono *Federated Learning via Group Knowledge Transfer* (**FedGKT**) [4] e *Federated Generation* (**FedGen**) [24].

FedGKT affronta il problema dell'addestramento di modelli complessi su dispositivi con risorse limitate. In questo schema, i client addestrano modelli leggeri e condividono rappresentazioni intermedie (attivazioni) con un server più potente, che esegue il training su modelli più grandi tramite meccanismi di distillazione.

In questo modo si riduce il carico computazionale e di memoria sui client, pur mantenendo

do la possibilità di beneficiare della capacità espressiva di modelli complessi lato server. Gli esperimenti riportati dagli autori su CIFAR-10, CIFAR-100 e CINIC-10 dimostrano che FedGKT raggiunge accuratezze comparabili a FedAvg, ma con una significativa riduzione del costo computazionale sui dispositivi periferici.

FedGen, invece, affronta la questione della generalizzazione in scenari federati caratterizzati da distribuzioni spurie o fortemente non-IID. Il framework introduce un generatore centralizzato che, addestrato a partire da informazioni statistiche condivise dai client, produce campioni sintetici utilizzati per regolarizzare l'addestramento dei modelli locali. Questa strategia riduce l'overfitting ai domini specifici dei client e migliora la capacità di generalizzare a nuovi dati. I risultati sperimentali, riportati su benchmark di classificazione testuale e sequenziale, mostrano che FedGen migliora la robustezza e la stabilità della convergenza rispetto a FedAvg e FedProx in presenza di dati altamente eterogenei.

1.5 Sintesi dello stato dell'arte

Dalla letteratura emerge come l'evoluzione dell'apprendimento federato abbia seguito due filoni complementari. Il primo è costituito dagli algoritmi di aggregazione classici (FedAvg, FedProx, FedOpt), che affrontano principalmente il problema del *client drift* e della convergenza lenta in scenari non-IID. Il secondo filone riguarda la *knowledge distillation* tradizionale (Hinton, Romero), che nasce come tecnica di trasferimento di conoscenza da modelli complessi a modelli più leggeri. Il terzo filone è rappresentato dai metodi di *knowledge distillation federata* (FedMD, FedDF, FedGen, FedGKT), che spostano la cooperazione dal livello dei parametri a quello delle predizioni, riducendo i vincoli architetturali e talvolta ricorrendo a dati sintetici per preservare la privacy. Accanto a questi approcci generali, lavori recenti come TL-VAE hanno esplorato architetture specifiche per domini clinici, integrando estrattori di feature pre-addestrati con modelli generativo-discriminativi.

La Tabella 1.1 riassume i principali contributi analizzati, specificando per ciascuno: il problema affrontato e l'approccio proposto. Questa sintesi consente di inquadrare le linee di ricerca esistenti senza ripetere nei singoli paragrafi la connessione diretta con il presente lavoro di tesi, che sarà discussa in maniera organica nel capitolo successivo.

Lavoro	Problema affrontato	Approccio proposto
McMahan et al. FedAvg	Preservare la privacy e ridurre i costi di comunicazione in scenari decentralizzati; garantire la convergenza anche con dati non-IID.	Media pesata dei modelli locali dopo più epoche di aggiornamento, riducendo la frequenza di sincronizzazione con il server.
Li et al. FedProx	Instabilità e <i>client drift</i> causati da distribuzioni locali eterogenee; rischio di modelli globali incoerenti.	Termine prossimale nella loss locale che vincola la distanza tra modello globale e locale, mantenendo gli aggiornamenti più stabili.
Reddi et al. FedOpt (FedAdam, FedAdagrad)	Lentezza di convergenza e inefficienza della media pesata in scenari non-IID complessi; necessità di adattività lato server.	Riformulazione della differenza locale-globale come pseudo-gradiente, ottimizzato tramite metodi adattivi (Adam, Adagrad).
Esposito et al. TL-VAE	Apprendimento federato su dati clinici non-IID e sbilanciati, con necessità di rappresentazioni latenti robuste.	Integrazione di MobileNetV2 preaddestrata come estrattore di feature con un CI-VAE federato tramite FedAvg.
Hinton et al. Knowledge Distillation	Ridurre la complessità dei modelli di deep learning mantenendo elevate prestazioni; trasferire conoscenza da modelli grandi a modelli compatti.	Introduzione della distillazione della conoscenza: lo <i>student</i> imita la distribuzione di probabilità del <i>teacher</i> mediante una loss combinata (cross-entropy + KLD) e un parametro di temperatura che ammorbidisce le predizioni.
Romero et al. Fit-Nets	Limitata profondità e capacità rappresentativa dei modelli student; difficoltà nel trasferimento efficace di conoscenza nelle reti profonde.	Estensione della KD attraverso il <i>hint-based training</i> : lo <i>student</i> apprende anche dalle attivazioni intermedie del <i>teacher</i> , migliorando convergenza e generalizzazione con una loss di allineamento tra feature map.
Li et al. FedMD	Rigidità degli algoritmi basati su aggregazione dei pesi; difficoltà a supportare modelli eterogenei; rischi di privacy leakage.	Distillazione basata su predizioni di ciascun client su un dataset pubblico condiviso, aggregate e redistribuite per l'allineamento locale.
Lin et al. FedDF	Mancanza di dataset pubblico realmente condivisibile in scenari sensibili alla privacy.	Uso di modelli generativi (GAN/VAE) per creare un dataset sintetico su cui distillare predizioni aggregate.
He et al. FedGKT	Limitata capacità computazionale dei client, che non riescono a eseguire l'intero training backpropagation.	Delegare al server la backward pass, mentre i client eseguono solo la forward pass sui dati locali.
Zhu et al. FedGen	Necessità di ridurre la dipendenza da dati pubblici condivisi e migliorare la stabilità in scenari non-IID.	Generatore centrale di campioni sintetici costruito da statistiche aggregate, distribuiti ai client per la distillazione collaborativa.

Tab. 1.1: Sintesi dei principali contributi nello stato dell'arte: problemi affrontati e approcci proposti.

Capitolo 2

Progettazione

Il capitolo descrive in modo dettagliato la fase di progettazione del lavoro, con l'obiettivo di delineare le scelte metodologiche, architetturali e sperimentali che hanno guidato lo sviluppo dei diversi paradigmi di apprendimento considerati. L'intento è fornire una visione chiara del percorso seguito, evidenziando come i modelli, i dataset e le strategie di addestramento siano stati concepiti per affrontare il problema della classificazione automatica di routine fisioterapiche in scenari distribuiti.

Dopo una sezione introduttiva dedicata agli obiettivi generali del lavoro e alla descrizione dei dataset impiegati, il capitolo approfondisce tre differenti approcci di apprendimento. Il primo è basato sul paradigma del Federated Learning classico, nel quale l'addestramento dei modelli avviene in modo distribuito e cooperativo tra più client, coordinati da un server centrale. In questa parte vengono presentati il modello CI-VAE utilizzato, le diverse strategie di aggregazione lato server e la pipeline sperimentale adottata per valutarne le prestazioni in scenari non-IID.

Segue il paradigma della Knowledge Distillation centralizzata, che introduce un modello teacher addestrato su un dataset aggregato e più modelli student che apprendono dalle sue predizioni, permettendo di analizzare il trasferimento di conoscenza in condizioni ideali e di definire una baseline di riferimento. Infine, il capitolo si concentra sulla Knowledge Distillation federata, in cui i principi della distillazione vengono estesi a un contesto distribuito, preservando la riservatezza dei dati locali. Sono descritte le due varianti considerate, FedMD e FedKD, entrambe basate sulla cooperazione tra modelli tramite la condivisione di predizioni su un dataset sintetico generato da statistiche aggregate.

L'intero capitolo mira a costruire un quadro organico e comparabile dei tre paradigmi proposti, illustrando per ciascuno le motivazioni teoriche, l'architettura del sistema e la progettazione sperimentale, che costituiranno la base per le analisi e i risultati discussi nei capitoli successivi.

2.1 Obiettivo del lavoro

Il presente lavoro si concentra sul problema della classificazione automatica di routine fisioterapiche in ambienti distribuiti, con particolare attenzione agli scenari di apprendimento federato. In questo contesto, i dati dei pazienti sono raccolti localmente e non possono essere centralizzati per motivi di privacy, rendendo necessario lo sviluppo di tecniche che consentano l'apprendimento collaborativo in presenza di dati non IID e distribuzioni squilibrate tra i client.

Il punto di partenza è rappresentato dal lavoro di Esposito et al., che ha introdotto il framework TL-VAE per l'apprendimento federato basato su rappresentazioni latenti. Questo approccio combina una rete MobileNetV2 pre-addestrata (congelata), usata per l'estrazione delle feature, con un modello CI-VAE addestrato localmente su ciascun client. Nel lavoro originale, la federazione dei modelli avviene tramite la classica strategia *FedAvg*, in cui i pesi dei modelli locali vengono mediati centralmente per aggiornare un modello globale.

In questo studio, si estende l'analisi del framework TL-VAE esplorando diverse strategie di aggregazione nel contesto federato, andando oltre *FedAvg*. L'obiettivo è valutare se tecniche alternative possano portare benefici in termini di accuratezza e stabilità dell'addestramento.

In aggiunta a questo filone di ricerca, il lavoro propone un confronto con il paradigma della *Knowledge Distillation*, esplorato sia in forma centralizzata che federata. Nella variante centralizzata, si assume la disponibilità di un dataset etichettato aggregato, su cui viene addestrato un modello teacher; i suoi output vengono poi utilizzati per distillare la conoscenza su modelli student locali. In ambito federato, vengono invece adottati due approcci distinti: *FedMD* e *FedKD*, che utilizzano dati sintetici generati a partire da statistiche locali.

Tutti i paradigmi considerati — federato standard, distillazione centralizzata, e distillazione federata — sono costruiti su un input coerente: vettori di feature a 1280 dimensioni estratti tramite MobileNetV2 congelata. Le valutazioni sperimentali saranno condotte su due dataset: FEMNIST, benchmark visivo ampiamente utilizzato nell'apprendimento federato, e IRDS, un dataset specifico per la classificazione di esercizi fisioterapici raccolti da soggetti reali.

Nelle sezioni successive verranno descritti in dettaglio i tre paradigmi considerati — TL-VAE con strategie federate, distillazione centralizzata e distillazione federata — con particolare attenzione alle rispettive architetture e meccanismi di cooperazione.

2.1.1 Dataset utilizzati

Alla base delle sperimentazioni proposte nel presente studio vi sono due dataset con caratteristiche significative per il contesto federato: **FEMNIST**, una variante gerarchica del celebre MNIST costruita per scenari non-IID, e **IRDS**, un corpus clinico di movimenti fisioterapici acquisiti da sensori 3D.

Dataset FEMNIST (Federated Extended MNIST)

Il dataset **FEMNIST** (Federated Extended MNIST) [3] costituisce uno dei benchmark più diffusi e consolidati per la valutazione di algoritmi di *Federated Learning* (FL). Esso fa parte del framework LEAF, un insieme di dataset progettati per scenari decentralizzati e non-IID, sviluppato con l'obiettivo di fornire ambienti di test realistici per il training federato. FEMNIST estende il classico dataset MNIST includendo non soltanto le cifre da 0 a 9, ma anche lettere maiuscole e minuscole, per un totale di 62 classi distinte ($10 + 26 + 26$). Questo incremento di complessità rende il compito di classificazione più articolato rispetto al benchmark originario, introducendo sfide legate alla variabilità tipografica e stilistica della scrittura manuale.

Ogni campione è rappresentato da un'immagine in scala di grigi di dimensione 28×28 pixel, derivata dal dataset NIST SD-19 opportunamente preprocessato per garantire compattezza e uniformità. In aggiunta all'immagine, sono disponibili metadati fondamentali quali l'identificativo dello scrivente (`writer_id`) e l'etichetta corrispondente al carattere (`character`). Tra questi, la chiave `writer_id` svolge un ruolo cruciale, poiché abilita il partizionamento naturale dei dati per autore. In scenari federati, ciò consente di assegnare a ciascun client i campioni prodotti da un singolo scrivente, riproducendo così in maniera fedele un contesto non-IID in cui i dati locali riflettono stili di scrittura eterogenei.

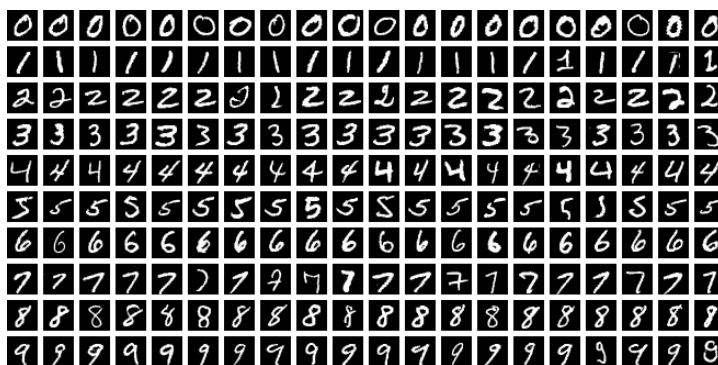


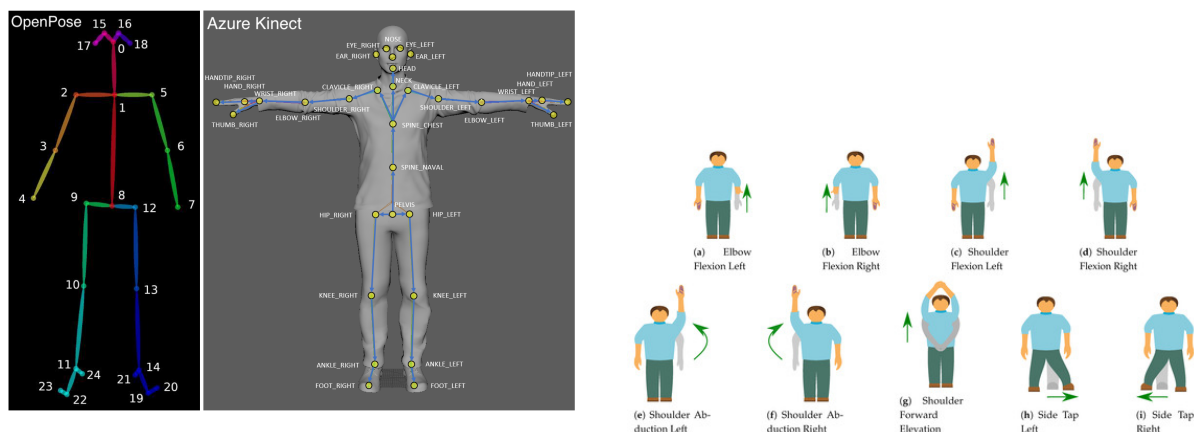
Fig. 2.1: Esempi tratti dal dataset FEMNIST. Ogni immagine è in scala di grigi e rappresenta un carattere scritto a mano (cifre e lettere maiuscole e minuscole). Il dataset include 62 classi e partizioni non-IID basate sullo scrivente, per un totale di oltre 805 000 campioni. Fonte: Hugging Face ([flwrlabs/femnist](https://huggingface.co/datasets/flwrlabs/femnist)).

Dal punto di vista quantitativo, FEMNIST raccoglie circa 814 227 esempi provenienti da oltre 3.500 scrittori unici. Ogni scrittore contribuisce con una media di circa 226 campioni, con una deviazione standard pari a 89, a testimonianza dell'elevata variabilità tra le distribuzioni locali. Questa disomogeneità intrinseca costituisce un banco di prova significativo per i protocolli di Federated Learning, nei quali è necessario garantire la convergenza globale nonostante la diversità dei dati locali.

FEMNIST si distingue per il suo equilibrio tra semplicità delle rappresentazioni visive (immagini a bassa risoluzione e in scala di grigi) e complessità strutturale legata alla natura non-IID del partizionamento. La possibilità di simulare scenari federati realistici, unita all'ampiezza del numero di classi, ne fa un dataset di riferimento consolidato per la ricerca nel campo del FL.

Il dataset IRDS: IntelliRehabDS

Il dataset *IntelliRehabDS* (IRDS) [16] è stato progettato per fornire una base empirica alla valutazione automatica dell'esecuzione di esercizi fisioterapici, con particolare riferimento al contesto della riabilitazione motoria degli arti superiori. A differenza di altri dataset presenti in letteratura, spesso ottenuti mediante acquisizioni su soggetti sani in scenari controllati, IRDS è costruito a partire da dati raccolti in un ambiente clinico reale, su pazienti affetti da condizioni ortopediche o neurologiche, e include annotazioni effettuate da fisioterapisti professionisti. Tale caratteristica conferisce al dataset una maggiore validità, rendendolo adatto all'addestramento e alla valutazione di modelli in contesti ad alta variabilità inter- e intra-soggetto.



(a) Esempio di keypoints scheletrici estratti da OpenPose (a sinistra) e da Azure Kinect (a destra). I diversi sistemi di acquisizione presentano differenze nella granularità e nella disposizione dei punti articolari.

(b) Esempi delle esercitazioni riabilitative contenute nel dataset IRDS, comprendenti movimenti di flessione ed abduzione degli arti superiori e laterali.

Fig. 2.2: Illustrazione del dataset IRDS: a sinistra, i sistemi di acquisizione scheletrica (OpenPose e Azure Kinect); a destra, una selezione delle esercitazioni motorie annotate nel dataset.

La campagna di raccolta dati si è svolta presso il centro riabilitativo *Pusat Rehabilitasi Perkeso* in Malesia e ha coinvolto 29 individui, di cui 15 pazienti effettivi in trattamento e 14 soggetti sani, questi ultimi inclusi come gruppo di controllo per consentire analisi comparative. L'acquisizione è stata effettuata tramite il dispositivo Microsoft Kinect v2, che fornisce simultaneamente segnali RGB, dati di profondità e tracciamento tridimensionale dei giunti articolari, alla frequenza costante di 30 fotogrammi al secondo. Le informazioni raccolte comprendono, per ogni frame, le coordinate 3D di 25 giunti scheletrici, la mappa di profondità della scena, il tipo di esercizio eseguito, lo stato di correttezza del gesto, la posizione del soggetto (seduto o in piedi) e l'identificativo individuale.

Il protocollo sperimentale include nove esercizi riabilitativi di uso comune nella terapia degli arti superiori. Ogni gesto è stato registrato in più condizioni: in posizione seduta e in piedi, ed eseguito sia correttamente sia in maniera deliberatamente errata, al fine di simulare errori frequenti nei pazienti con limitazioni funzionali. Ogni combinazione è stata ripetuta almeno tre volte per garantire una certa densità osservazionale. Le annotazioni di correttezza sono state prodotte manualmente da fisioterapisti esperti, sulla base di criteri clinici standardizzati, e rappresentano l'etichetta di riferimento per le successive fasi di classificazione automatica.

I dati sono memorizzati in formato testuale strutturato (.txt o .csv) secondo una convenzione sistematica che codifica in maniera univoca tutte le informazioni rilevanti. Ciascun file corrisponde a una singola esecuzione e contiene una sequenza temporale di frame descritti dalle coordinate spaziali. L'organizzazione del dataset permette analisi sia frame-wise sia sequence-wise, abilitando approcci supervisionati e auto-supervisionati basati su architetture deep learning.

Nel presente lavoro, IRDS è stato impiegato come benchmark clinico per la valutazione di metodi di apprendimento. La sua struttura eterogenea, che riflette realisticamente condizioni non identicamente distribuite tra i soggetti, risulta particolarmente adatta per simulare scenari federati in cui i client possiedono dati fortemente non-IID.

2.1.2 MobileNetV2 come estrattore di feature

In tutti i paradigmi analizzati in questo lavoro, la fase di pre-elaborazione delle immagini è affidata a MobileNetV2 quantizzata, utilizzata esclusivamente come estrattore di feature. Questa architettura è stata progettata per dispositivi mobili e scenari con risorse limitate, con l'obiettivo di garantire un buon compromesso tra accuratezza e costo computazionale.

La prima generazione di MobileNet [6] si basa sul concetto di *depthwise separable convolutions*, che rappresentano una fattorizzazione della convoluzione standard. Invece di applicare un unico filtro tridimensionale su tutte le dimensioni spaziali e sui canali, l'operazione viene suddivisa in due passaggi distinti (Fig. 2.3):

- la **depthwise convolution**, che applica un kernel spaziale indipendente su ciascun canale di input, estraendo le caratteristiche locali;
- la **pointwise convolution** (convoluzione 1×1), che combina linearmente i canali risultanti per generare nuove rappresentazioni.

Questa decomposizione riduce in modo significativo la complessità computazionale, passando da un costo pari a $\mathcal{O}(D_K^2 \cdot M \cdot N)$ per una convoluzione standard a $\mathcal{O}(D_K^2 \cdot M + M \cdot N)$ per la variante depthwise separable, dove D_K è la dimensione del kernel, M il numero di canali in input e N quello in output.

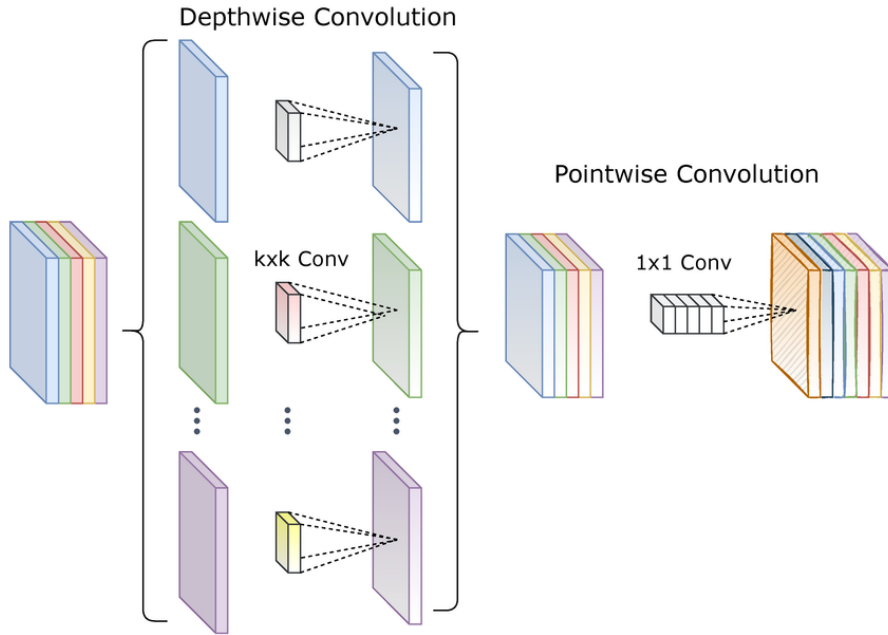


Fig. 2.3: Convoluzioni depthwise separable introdotte in MobileNet. (a) convoluzione standard: un kernel tridimensionale applicato simultaneamente su tutti i canali; (b) depthwise convolution: un kernel per ciascun canale; (c) pointwise convolution: combinazione lineare dei risultati.

La principale innovazione di MobileNetV2 [21] è l'introduzione dei blocchi *inverted residual* con *linear bottleneck*. A differenza dei residual block classici (ad esempio in ResNet), che comprimono le feature per poi espanderle, il blocco invertito adotta una logica opposta:

1. **Espansione:** l'input $H \times W \times M$ viene proiettato in uno spazio a dimensionalità maggiore (tM , con tipicamente $t = 6$) tramite una convoluzione 1×1 . Ciò aumenta la capacità espressiva del blocco.
2. **Depthwise convolution:** sui canali espansi viene applicata una convoluzione $k \times k$, operando separatamente su ciascun canale e mantenendo basso il costo computazionale.
3. **Compressione lineare (bottleneck):** un'ulteriore convoluzione 1×1 riduce i canali a N . Questo strato è lineare, privo di attivazioni non lineari, per preservare l'informazione ed evitare perdite dovute alla compressione.

Quando il numero di canali in ingresso e in uscita coincide ($M = N$) e lo stride è pari a 1, l'input originale viene sommato all'output del blocco tramite una connessione residua. Il termine *inverted* fa riferimento proprio a questa architettura: l'espansione avviene all'inizio e la compressione al termine, in contrasto con i residual block tradizionali.

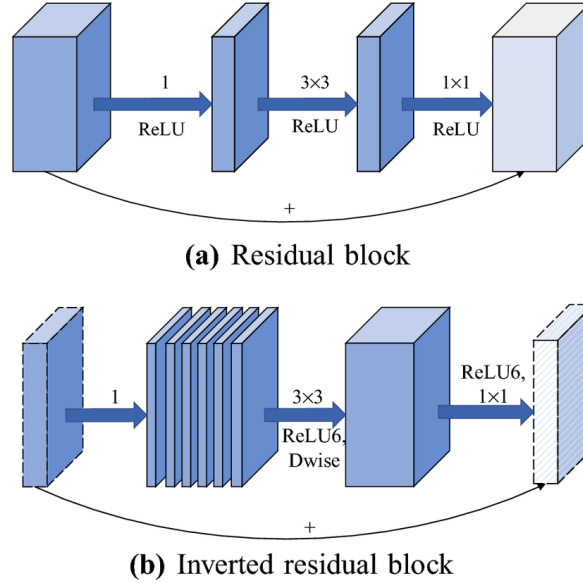


Fig. 2.4: Confronto tra un residual block tradizionale (a) e un *inverted residual block* (b) di MobileNetV2. Nel blocco invertito, l'input viene prima espanso tramite una convoluzione 1×1 , elaborato da una depthwise convolution 3×3 , e infine compresso linearmente con una convoluzione 1×1 . Se $\text{stride}=1$ e $M = N$, è presente una connessione residua.

Nel presente lavoro, MobileNetV2 quantizzata è utilizzata in modalità *frozen*. Le immagini RGB vengono normalizzate secondo i parametri standard di ImageNet e processate dalla rete fino allo strato finale, che produce un embedding di dimensione 1280. Questo vettore, denotato come $\mathbf{z}_{\text{feat}} \in \mathbb{R}^{1280}$, costituisce la rappresentazione comune fornita come input a tutti i modelli considerati (federati, distillati centralmente o distillati in ambito federato).

2.2 Paradigma 1: Federated Learning classico

2.2.1 Fondamenti teorici e motivazioni

L'apprendimento federato è un paradigma distribuito per l'addestramento collaborativo di modelli di machine learning, in cui più nodi periferici, detti *client*, partecipano al processo di ottimizzazione condividendo esclusivamente gli aggiornamenti dei parametri del modello, e non i dati locali su cui avviene l'addestramento. Un'entità centrale, il *server*, coordina il processo aggregando periodicamente i contributi ricevuti dai client e aggiornando iterativamente il modello globale.

Questo approccio, introdotto da McMahan et al. [15] con il protocollo FedAvg, si configura come risposta a tre principali sfide dell'intelligenza artificiale moderna: la preservazione della privacy, la distribuzione eterogenea dei dati tra client, e l'efficienza nella comunicazione in ambienti decentralizzati. A differenza dei modelli centralizzati, l'FL si basa sul principio di *data minimization*, per cui i dati sensibili non lasciano mai i dispositivi di origine, minimizzando il rischio di violazioni della privacy o esposizioni non autorizzate.

Il Federated Learning è stato progettato per operare anche in scenari in cui i dati siano *non indipendenti e non identicamente distribuiti*. Questo è particolarmente rilevante in contesti applicativi reali, dove ogni client può osservare solo un sottoinsieme parziale e potenzialmente sbilanciato del dominio, con classi mancanti o rappresentate in modo disomogeneo. McMahan et al. evidenziano come questa eterogeneità possa ostacolare la convergenza del modello globale, ma dimostrano empiricamente che approcci come FedAvg possono comunque garantire performance competitive.

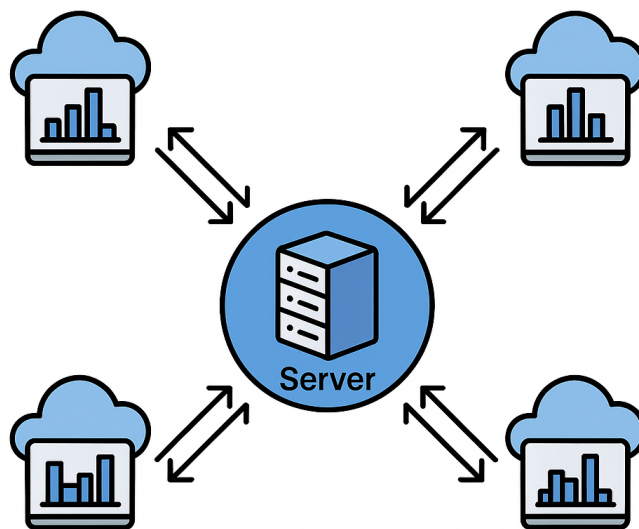


Fig. 2.5: Rappresentazione schematica dell'apprendimento federato. I dati restano locali presso ciascun client, rappresentati da distribuzioni non identicamente distribuite (non-IID), mentre solo i parametri del modello vengono condivisi con il server centrale per l'aggregazione e successiva redistribuzione.

In questo lavoro, il paradigma FL viene applicato all'analisi distribuita di routine fisioterapiche, un ambito caratterizzato dalla presenza di dati sensibili, fortemente eterogenei,

acquisiti da fonti differenti e soggetti a vincoli normativi rilevanti.

Nonostante i numerosi vantaggi offerti dal paradigma, l'apprendimento federato presenta alcune criticità ben documentate in letteratura. Tra queste, si evidenziano la lentezza nella convergenza rispetto all'apprendimento centralizzato, l'eterogeneità sia statistica che computazionale tra client, i costi di comunicazione in ambienti reali, nonché la possibile vulnerabilità a client avversari o compromessi.

2.2.2 Architettura del sistema federato

L'architettura federata adottata in questo lavoro si basa su un paradigma *client-server* sincrono, in cui un server centrale coordina l'addestramento distribuito di un modello globale. Il framework riprende il TL-VAE [2], che integra MobileNet pre-addestrato (Sez. 2.1.2) con un CI-VAE (Sez. 2.2.3) per costruire uno spazio latente condiviso tra i client. Nel nostro contesto, le feature vengono estratte localmente da MobileNetV2 quantizzata e fornite al CI-VAE, il quale apprende una rappresentazione latente utile per la classificazione distribuita. Il processo si articola in una sequenza di round federati, ciascuno dei quali prevede una fase locale eseguita in parallelo dai client partecipanti e una fase di aggregazione centralizzata.

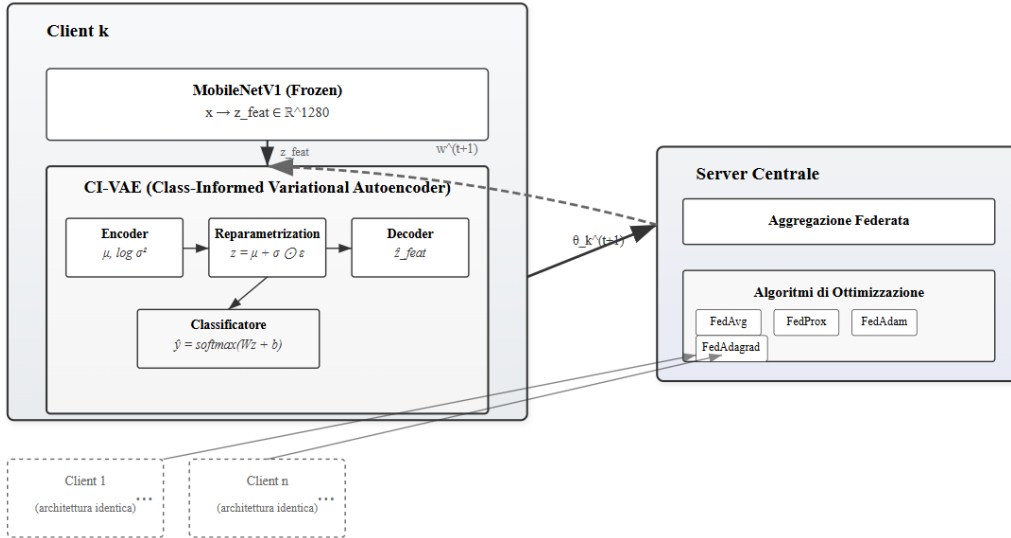


Fig. 2.6: Architettura del sistema federato: ciascun client estrae feature tramite MobileNetV1 e le fornisce in input a un CI-VAE con decoder e classificatore latente. I pesi aggiornati vengono inviati al server, che esegue l'aggregazione e distribuisce il modello globale aggiornato.

Fase locale (client). Ogni client elabora i dati grezzi tramite una MobileNetV2 pre-addestrata e congelata, ottenendo un vettore di feature $\mathbf{z}_{\text{feat}} \in \mathbb{R}^{1280}$. Questo vettore viene fornito in input a un modello CI-VAE, che esegue simultaneamente ricostruzione nel dominio delle feature e classificazione latente. L'ottimizzazione locale avviene minimizzando la loss composita $\mathcal{J}_{\text{loc}}$, definita come somma di tre termini: errore di ricostruzione, regolarizzazione KL e cross-entropy sul classificatore (Sez. 2.2.3). Al termine dell'addestramento per un numero fisso di epoche locali E , il client restituisce al server i parametri aggiornati del proprio modello.

Fase centrale (server). Il server raccoglie gli aggiornamenti dai client selezionati e li aggrega per produrre una nuova versione del modello globale $w^{(t+1)}$. A seconda della strategia adottata, l'aggregazione può essere una semplice media pesata dei parametri (FedAvg), una versione regolarizzata (FedProx) o un aggiornamento adattivo ispirato ad algoritmi come Adam o Adagrad (FedOpt). I dettagli delle strategie di aggregazione sono approfonditi in Sez. 2.2.4.

Ciclo federato. Il processo di training prosegue per un numero fissato di round T oppure fino al soddisfacimento di un criterio di arresto basato su metrica globale di validazione (ad esempio, accuratezza stazionaria). In ogni round, viene selezionato un sottoinsieme $\mathcal{S}_t \subseteq \mathcal{C}$ dei client disponibili, su cui viene distribuito il modello corrente $w^{(t)}$. Il ciclo generale è riportato nell'Alg. 1.

Orchestrazione e infrastruttura. La comunicazione tra entità federate è gestita tramite il framework *Flower*, che offre astrazioni per il controllo dei round, la raccolta delle metriche, la gestione delle connessioni client e la serializzazione dei modelli.

Algorithm 1 Federated Learning con CI-VAE e MobileNet

- 1: **Input:** Modello iniziale $w^{(0)}$, numero di round T , epoche locali E , insieme dei client $\mathcal{C}$
 - 2: **for** $t = 0, 1, \dots, T - 1$ **do**
 - 3: Il server seleziona $\mathcal{S}_t \subseteq \mathcal{C}$
 - 4: **for all** client $k \in \mathcal{S}_t$ **in parallelo do**
 - 5: Riceve $w^{(t)}$, inizializza $(\theta, \phi, \psi) \leftarrow w^{(t)}$
 - 6: Estrae $x = \mathbf{z}_{\text{feat}}$ via MobileNet (frozen)
 - 7: Addestra il CI-VAE su (x, y) per E epoche minimizzando $\mathcal{J}_{\text{loc}}$
 - 8: Invia $w_k^{(t+1)}$ al server
 - 9: **end for**
 - 10: Il server aggrega $\{w_k^{(t+1)}\}$ usando FedAvg, FedProx, FedAdam o FedAdagrad
 - 11: Ridistribuisce $w^{(t+1)}$ ai client
 - 12: **end for**
 - 13: **Output:** Modello globale $w^{(T)}$
-

2.2.3 CI-VAE: encoder, decoder e classificatore

Il *Class-Informed Variational Autoencoder* adottato in questo lavoro estende il paradigma dei *Variational Autoencoder* (VAE) [10] con l'obiettivo di apprendere una rappresentazione latente che sia *discriminativa* rispetto alle classi di interesse. A differenza dei Conditional VAE (CVAE) [22], che incorporano la label come variabile condizionante direttamente nell'encoder e nel decoder, il CI-VAE introduce l'informazione di classe in maniera indiretta, aggiungendo una testa di classificazione applicata allo spazio latente. Il CI-VAE riceve in input un vettore denso $\mathbf{z}_{\text{feat}} \in \mathbb{R}^{1280}$ estratto localmente da MobileNet (si veda §2.1.2), riducendo così il costo computazionale e semplificando l'ottimizzazione in scenari federati.

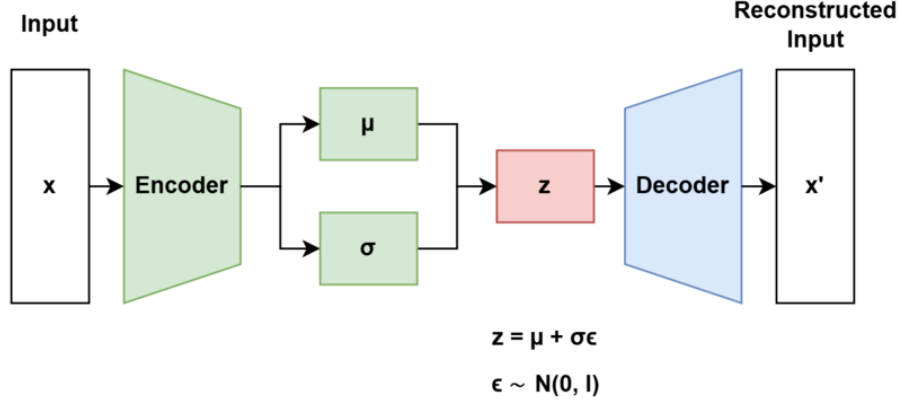


Fig. 2.7: Architettura schematica di un Variational Autoencoder (VAE). L'encoder proietta l'input nello spazio latente producendo media e varianza, da cui si campiona z tramite la reparameterization trick; il decoder ricostruisce l'input.

Modello probabilistico. Dato un input $x \in \mathbb{R}^d$ (con $d = 1280$) e una variabile latente $z \in \mathbb{R}^m$ con prior isotropo $p(z) = \mathcal{N}(0, I)$, l'encoder approssima il posterior con:

$$q_\phi(z | x) = \mathcal{N}(\mu_\phi(x), \text{diag } \sigma_\phi^2(x)),$$

mentre il decoder modella $p_\theta(x | z)$ come distribuzione gaussiana con varianza fissa. Il campionamento latente è reso differenziabile tramite la *reparameterization trick*:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

L'ottimizzazione è guidata dalla *Evidence Lower Bound* (ELBO):

$$\mathcal{L}_{\text{ELBO}}(x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x | z)] - D_{\text{KL}}(q_\phi(z | x) \| p(z)),$$

dove, assumendo prior $p(z) = \mathcal{N}(0, I)$ e posterior gaussiano diagonale, il termine KL ammette forma chiusa:

$$D_{\text{KL}}(q_\phi(z | x) \| p(z)) = \frac{1}{2} \sum_{j=1}^m (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1).$$

Iniezione di informazione di classe. Per garantire una rappresentazione latente utile alla classificazione, si introduce una testa $h_\psi : \mathbb{R}^m \rightarrow \Delta^{C-1}$, implementata come livello lineare seguito da `softmax`, che restituisce una distribuzione di probabilità sulle C classi:

$$\hat{y} = h_\psi(z) = \text{softmax}(Wz + b).$$

Questo meccanismo supervisionato guida il latente ad assumere una struttura separabile tra classi, migliorando la convergenza nei contesti federati.

Funzione obiettivo. La loss locale su un campione (x, y) è una combinazione di ricostruzione generativa e classificazione supervisionata:

$$\mathcal{J}_{\text{loc}}(x, y; \theta, \phi, \psi) = -\mathcal{L}_{\text{ELBO}}(x) + \lambda_{\text{cls}} \text{CE}(h_\psi(z), y),$$

dove CE è la cross-entropy e z è ottenuto tramite reparameterization.

Integrazione nella pipeline federata. Durante l'addestramento federato, ogni client riceve il modello globale, estrae $x = \mathbf{z}_{\text{feat}}$ e aggiorna localmente i parametri (θ, ϕ, ψ) minimizzando $\mathcal{J}_{\text{loc}}$. I pesi aggiornati vengono poi trasmessi al server, che li aggrega secondo le strategie definite (cfr. §2.2.4). L'utilizzo di uno spazio delle feature condiviso tra i client migliora la stabilità e la compatibilità dei gradienti aggregati, specialmente in scenari non-IID.

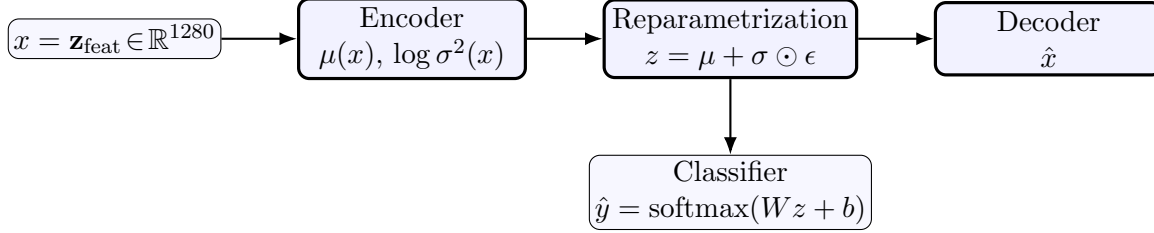


Fig. 2.8: Schema del CI-VAE. L'encoder produce i parametri del posterior gaussiano; il campione z è ottenuto tramite reparameterization; il decoder ricostruisce x , mentre una testa di classificazione applicata al latente restituisce $\hat{y}$

2.2.4 Algoritmi di aggregazione lato server

Nel contesto dell'apprendimento federato, la fase di aggregazione lato server rappresenta il fulcro dell'intero processo. Dopo che i client hanno completato l'addestramento locale, inviano i parametri aggiornati al server, che li combina per aggiornare il modello globale w . Il problema di ottimizzazione federata può essere formalizzato come segue:

$$\min_{w \in \mathbb{R}^d} F(w) = \sum_{k=1}^K p_k F_k(w), \quad p_k = \frac{n_k}{\sum_{j=1}^K n_j},$$

dove $F_k(w)$ è la funzione di perdita locale del client k su dati $\mathcal{D}_k$ di cardinalità n_k , e p_k rappresenta la frazione relativa di dati locali. Al round t , il server invia il modello globale $w^{(t)}$ a un sottoinsieme di client $\mathcal{S}_t$, raccoglie i modelli locali aggiornati $w_k^{(t+1)}$, e applica una regola di aggregazione **Agg** per ottenere il nuovo modello $w^{(t+1)}$.

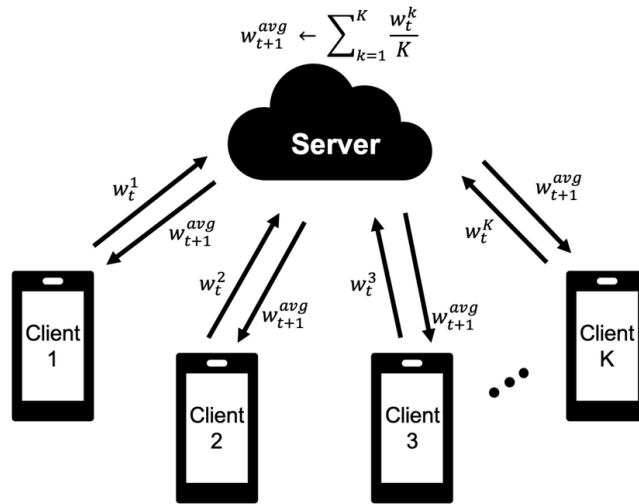


Fig. 2.9: Schema del funzionamento dell'algoritmo *Federated Averaging* (FedAvg). Ogni client k aggiorna localmente i pesi w_t^k sui propri dati e invia il modello al server centrale. Il server aggrega i pesi ricevuti calcolandone la media $\frac{1}{K} \sum_{k=1}^K w_t^k$, ottenendo il modello globale w_{t+1}^{avg} , che viene poi redistribuito ai client per il round successivo.

Di seguito vengono illustrate le strategie di aggregazione adottate.

Federated Averaging (FedAvg). Proposto da McMahan, FedAvg è l'algoritmo standard del FL. Ogni client esegue uno o più step di discesa del gradiente locale e restituisce il modello aggiornato. Il server esegue una media pesata rispetto alla quantità di dati:

$$w^{(t+1)} = \sum_{k \in \mathcal{S}_t} \frac{n_k}{n_{\mathcal{S}_t}} w_k^{(t+1)}, \quad \text{con} \quad n_{\mathcal{S}_t} = \sum_{k \in \mathcal{S}_t} n_k.$$

FedAvg è efficiente in termini di comunicazione, ma può soffrire di instabilità in scenari non-IID a causa del fenomeno noto come *client drift*, dove gli aggiornamenti locali divergono tra loro.

Federated Proximal (FedProx). Per affrontare il *drift*, Li et al. propongono FedProx, che modifica il problema locale includendo un termine di regolarizzazione prossimale:

$$\min_w F_k(w) + \frac{\mu}{2} \|w - w^{(t)}\|_2^2,$$

dove $\mu \geq 0$ controlla la penalizzazione della distanza dal modello globale corrente. In questo modo, gli aggiornamenti locali sono vincolati a non discostarsi troppo da $w^{(t)}$. L'aggregazione lato server resta una media pesata come in FedAvg.

Ottimizzazione adattiva: FedOpt. La famiglia di algoritmi *FedOpt*, introdotta da Reddi et al., generalizza FedAvg interpretando la differenza media tra modelli globali e locali come uno pseudo-gradiente:

$$\Delta^{(t)} = \sum_{k \in \mathcal{S}_t} \frac{n_k}{n_{\mathcal{S}_t}} \left(w^{(t)} - w_k^{(t+1)} \right),$$

e aggiornando il modello secondo

$$w^{(t+1)} = w^{(t)} - \eta \cdot \text{Opt}(\Delta^{(t)}),$$

dove η è il learning rate lato server e **Opt** rappresenta un ottimizzatore adattivo. Sono stati adottati due metodi:

- **FedAdam**, che applica momenti esponenzialmente decrescenti:

$$\begin{aligned} m^{(t+1)} &= \beta_1 m^{(t)} + (1 - \beta_1) \Delta^{(t)}, \\ v^{(t+1)} &= \beta_2 v^{(t)} + (1 - \beta_2) \Delta^{(t)} \odot \Delta^{(t)}, \\ \hat{m}^{(t+1)} &= \frac{m^{(t+1)}}{1 - \beta_1^{t+1}}, \quad \hat{v}^{(t+1)} = \frac{v^{(t+1)}}{1 - \beta_2^{t+1}}, \\ w^{(t+1)} &= w^{(t)} - \eta \frac{\hat{m}^{(t+1)}}{\sqrt{\hat{v}^{(t+1)} + \epsilon}}, \end{aligned}$$

dove:

- $m^{(t)}$ e $v^{(t)}$ sono rispettivamente i momenti del primo e secondo ordine dei gradienti;

- $\beta_1, \beta_2 \in [0, 1)$ controllano il decadimento dei momenti;
- $\hat{m}^{(t)}, \hat{v}^{(t)}$ correggono il bias iniziale dei momenti;
- $\epsilon > 0$ serve per stabilizzare la divisione.

- **FedAdagrad**, che accumula il quadrato dei gradienti passati:

$$v^{(t+1)} = v^{(t)} + \Delta^{(t)} \odot \Delta^{(t)},$$

$$w^{(t+1)} = w^{(t)} - \eta \frac{\Delta^{(t)}}{\sqrt{v^{(t+1)}} + \epsilon},$$

dove $v^{(t)}$ rappresenta l'accumulo dei quadrati dei pseudo-gradienti, che serve a modulare l'ampiezza degli aggiornamenti in maniera adattiva.

2.2.5 Progettazione sperimentale

La fase di progettazione ha avuto come obiettivo principale la definizione di una pipeline sperimentale chiara e riproducibile, in grado di valutare in modo sistematico l'impatto delle diverse strategie federate in scenari non-IID. Le scelte progettuali sono state guidate dalla necessità di bilanciare tre aspetti: la complessità dei modelli, la natura eterogenea dei dati e la confrontabilità tra paradigmi.

Il sistema federato è stato concepito per riprodurre fedelmente un contesto clinico: i *client* a centri di raccolta, ognuno dei quali possiede dati parziali, sbilanciati e potenzialmente incompleti; il *server* centrale, dotato di maggiori risorse computazionali, coordina l'orchestrazione dei round, aggrega gli aggiornamenti ricevuti e monitora le metriche globali. Il framework *Flower* è stato scelto per la sua capacità di supportare questo scenario, grazie a un'architettura modulare che consente la gestione trasparente dei client, il logging avanzato e la riproducibilità degli esperimenti.

Scelta dei modelli. L'adozione di un approccio ibrido basato su MobileNetV2 e CI-VAE risponde a due esigenze principali. Da un lato, l'uso di una MobileNetV2 pre-addestrata e congelata come estrattore di feature riduce il carico computazionale sui client. Dall'altro, l'impiego di un CI-VAE consente di costruire rappresentazioni latenti discriminative. L'encoder apprende una distribuzione gaussiana nello spazio latente e il classificatore latente guida la separazione tra classi. In particolare, l'obiettivo del CI-VAE è duplice: (i) garantire che lo spazio latente catturi variazioni intra-classe rilevanti per la ricostruzione, e (ii) imporre una struttura discriminativa utile alla classificazione cross-client.

Dal punto di vista architetturale, l'encoder è composto da tre livelli completamente connessi, ciascuno seguito da operazioni di normalizzazione batch, attivazione ReLU e dropout. La dimensionalità viene ridotta progressivamente per 3 livelli, fino a ottenere un quarto della dimensione iniziale. A valle dell'encoder, due proiezioni lineari producono i parametri della distribuzione latente: il vettore delle medie μ e quello delle varianze $\log \sigma^2$, che definiscono una distribuzione gaussiana diagonale nello spazio latente.

Il decoder segue una struttura speculare, articolata in quattro livelli densi che ricostruiscono progressivamente lo spazio delle feature originarie, anch'essi arricchiti da normalizzazione e attivazioni non lineari.

Parallelamente al decoder è stato introdotto un classificatore nello spazio latente, costituito da due livelli lineari con attivazione ReLU intermedia e dropout. Questo modulo supervisionato proietta il vettore latente z nello spazio delle classi e induce una struttura discriminativa che migliora la separabilità tra domini eterogenei.

La dimensionalità latente e il numero di neuroni dei layer nascosti sono stati definiti per mantenere un compromesso tra leggerezza computazionale ed espressività del modello. In particolare, sono state esplorate diverse configurazioni dello spazio latente (tra 7 e 15 dimensioni) per valutare l’impatto della compattezza rappresentazionale sulla generalizzazione.

Strategie di ottimizzazione. Le strategie di aggregazione sono state selezionate per coprire tre approcci complementari al problema del federated learning. *FedAvg* rappresenta la baseline consolidata, adottata per la sua semplicità ed efficienza comunicativa. *FedProx* è stato introdotto per verificare se l’aggiunta di un termine prossimale potesse ridurre il fenomeno del *client drift*, particolarmente rilevante in IRDS. Infine, *FedAdam* e *FedAdagrad*, appartenenti alla famiglia FedOpt, sono stati selezionati per valutare il contributo di ottimizzatori adattivi lato server, che modulano dinamicamente l’ampiezza degli aggiornamenti globali. Questa scelta riflette l’ipotesi che l’adattività possa stabilizzare la convergenza del CI-VAE in presenza di distribuzioni locali molto divergenti, riducendo il rischio di instabilità tipico del solo FedAvg.

Gestione dei dataset. Per la valutazione sono stati considerati due dataset: FEM-NIST, costituito da immagini scritte a mano, e IRDS, relativo a segnali di routine fisioterapiche. Entrambi i dataset presentano distribuzioni intrinsecamente non-IID. In fase di progettazione ho scelto di: (i) partizionare FEMNIST per *writer_id*, riducendo il problema a dieci classi per semplificare il confronto, con split 70% training e 30% test; (ii) suddividere IRDS per paziente, mantenendo le quattro classi di movimento, con separazione 70% training e 30% validation. Questa scelta riflette un approccio realistico, in cui ciascun client dispone di dati parziali e potenzialmente sbilanciati.

Gestione degli esperimenti. Gli esperimenti sono stati progettati per isolare l’impatto delle diverse strategie di aggregazione. Per ogni strategia è stata condotta una *grid search* sistematica sugli iperparametri chiave, includendo il learning rate locale, il numero di epoche di addestramento, la dimensionalità dello spazio latente, i parametri dello scheduler (gamma e step) e gli iperparametri specifici degli ottimizzatori adattivi (ad esempio η , β_1 , β_2 per FedAdam, oppure η per FedAdagrad). Tale approccio ha permesso di esplorare in modo esaustivo lo spazio delle configurazioni e di individuare, per ciascun contesto, i compromessi più favorevoli tra stabilità, accuratezza e capacità di generalizzazione.

Ogni esecuzione è stata tracciata in maniera strutturata tramite file `.csv` contenenti sia i parametri di configurazione sia le metriche di performance (accuracy, precision, recall, F1-score). Questa scelta ha reso possibile una valutazione quantitativa e comparabile tra modelli, garantendo al contempo riproducibilità e trasparenza del processo sperimentale.

2.3 Paradigma 2: Knowledge Distillation

2.3.1 Motivazioni e principi concettuali

In scenari distribuiti e sensibili alla privacy, come quelli tipici del Federated Learning, l'aggregazione di modelli locali attraverso pesi o gradienti può risultare problematica. I metodi classici, come Federated Averaging, si basano sull'ipotesi che tutti i client adottino la medesima architettura di rete e siano in grado di trasferire regolarmente i parametri appresi. Tuttavia, questa assunzione risulta spesso irrealistica in presenza di dispositivi eterogenei, con capacità computazionali, framework e vincoli di sicurezza differenti.

In risposta a tali limitazioni, il paradigma della *distillazione della conoscenza* [5] rappresenta una strategia promettente. In questa sezione, la distillazione viene applicata in forma *centralizzata*, assumendo la disponibilità di un dataset aggregato accessibile al server. In questo scenario, il modello *teacher* viene addestrato sul dataset centralizzato, acquisendo una rappresentazione globale dei dati, e produce predizioni morbide (*soft labels*) che catturano informazioni sulle relazioni tra le classi. Successivamente, tali predizioni vengono utilizzate per addestrare uno o più modelli *student*, ciascuno associato a un client, senza che sia necessario condividere direttamente né i dati originali né i pesi del *teacher*.

Va sottolineato che questo approccio, pur violando il principio di non condivisione dei dati tipico dell'FL, viene adottato come scenario ipotetico per studiare il comportamento dei modelli nel caso in cui fosse disponibile un dataset pubblico centralizzato, fornendo un riferimento teorico per confrontare successivamente strategie decentralizzate.

2.3.2 Distillazione della conoscenza: formulazione matematica

La *distillazione della conoscenza* (Knowledge Distillation) è una tecnica di trasferimento della conoscenza in cui un modello di riferimento (*teacher*) guida l'addestramento di un modello più semplice (*student*). L'idea centrale consiste nel far sì che lo *student* imiti non solo le etichette corrette (*hard labels*), ma anche la distribuzione predetta dal *teacher* (*soft labels*), che contiene informazioni aggiuntive sulle relazioni tra classi.

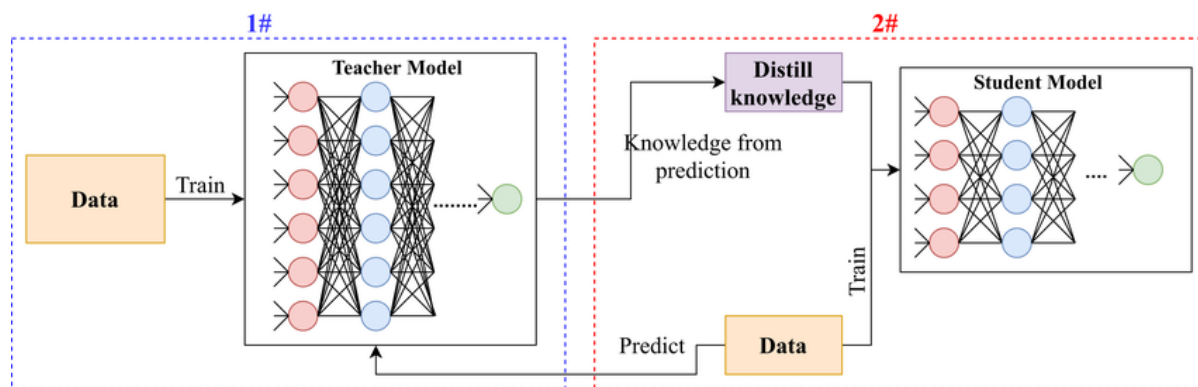


Fig. 2.10: Schema concettuale della *Knowledge Distillation* centralizzata. Il modello *teacher*, precedentemente addestrato, trasferisce conoscenza al modello *student* tramite la minimizzazione di una funzione di perdita combinata, che include la *cross-entropy* supervisionata e la divergenza di Kullback–Leibler tra le distribuzioni di probabilità ammorbidite (*soft targets*) generate dai due modelli.

In particolare, il teacher, per ciascun campione di input x , produce un vettore di logit $z \in \mathbb{R}^C$, dove C è il numero di classi. Applicando una *softmax* a temperatura $T > 1$, si ottiene la distribuzione ammorbidita:

$$p_i^{(T)} = \frac{\exp(z_i/T)}{\sum_{j=1}^C \exp(z_j/T)}, \quad i = 1, \dots, C. \quad (2.1)$$

Aumentare T ha l'effetto di rendere la distribuzione meno spigolosa, appiattendola e rendendo più evidenti le correlazioni tra classi meno probabili. Queste informazioni secondarie sono spesso ricche di conoscenza semantica e permettono allo student di apprendere pattern più generali rispetto a un semplice training supervisionato con hard labels.

L'addestramento dello student minimizza una loss combinata, che integra sia le hard label sia le soft label fornite dal teacher. La funzione di perdita è:

$$\mathcal{L}_{\text{KD}}(x) = (1 - \alpha) \cdot \mathcal{H}(y, q(x)) + \alpha \cdot T^2 \cdot \text{KL}(p^{(T)}(x) \parallel q^{(T)}(x)) \quad (2.2)$$

dove:

- $\mathcal{H}(y, q(x))$ è la cross-entropy tra le predizioni dello student $q(x)$ e l'etichetta reale y , detta *hard label*;
- $\text{KL}(p^{(T)} \parallel q^{(T)})$ misura la divergenza di Kullback-Leibler tra la distribuzione ammorbidita del teacher e quella dello student, cioè quanto lo student si discosta dal teacher nelle predizioni soft;
- $\alpha \in [0, 1]$ bilancia l'importanza relativa tra hard e soft labels;
- T^2 è un fattore di scaling che compensa la riduzione dei gradienti introdotta dalla softmax ad alta temperatura, garantendo stabilità numerica durante l'ottimizzazione.

In questo modo, lo student non apprende soltanto la classe corretta, ma anche le relazioni probabilistiche tra tutte le classi, migliorando la generalizzazione del modello e permettendo di trasferire la conoscenza globale del teacher anche a modelli più compatti e leggeri.

2.3.3 Architettura del paradigma di distillazione centralizzata

Il paradigma di distillazione centralizzata sviluppato in questo lavoro si articola in tre componenti principali: costruzione del dataset centralizzato, ciclo di addestramento e distillazione della conoscenza.

Per simulare la disponibilità di un dataset pubblico centralizzato, da ciascun client si preleva una porzione dei dati, che viene inclusa nel dataset centralizzato destinato all'addestramento del teacher. Il teacher viene quindi addestrato su questo insieme aggregato di feature provenienti da tutti i client, acquisendo una rappresentazione globale dei dati. I modelli student, invece, vengono addestrati localmente sulle rimanenti feature presenti nel client, che non sono state utilizzate dal teacher.

Ciclo di addestramento

L'addestramento centralizzato tramite distillazione opera direttamente sulle *feature* $\mathbf{z}_{\text{feat}} \in \mathbb{R}^{1280}$ estratte da MobileNetV2, piuttosto che sui dati grezzi. Il processo si articola in tre fasi principali:

1. **Addestramento del teacher:** il modello teacher viene addestrato sul dataset centralizzato, ottenuto aggregando porzioni di dati dai client. L'obiettivo è produrre predizioni globali accurate, catturando le relazioni tra classi tramite le cosiddette *soft labels*.
2. **Distribuzione delle soft labels:** le soft labels generate dal teacher mediante la 2.1 vengono comunicate a ciascun client. Queste rappresentano la conoscenza globale acquisita dal teacher senza condividere i dati originali.
3. **Addestramento degli student:** ogni client utilizza le soft labels ricevute per aggiornare localmente il proprio modello student. L'aggiornamento avviene minimizzando una loss di knowledge distillation che combina l'informazione delle soft labels con eventuali etichette locali, utilizzando la formula 2.2.

Questo processo, schematizzato in figura 2.11 permette di trasferire conoscenza globale ai modelli locali, mantenendo coerenza delle rappresentazioni anche in presenza di architetture eterogenee e di dati distribuiti in maniera non IID. L'algoritmo è indipendente dal tipo di modello utilizzato, sia esso un CI-VAE o una rete neurale fully connected.

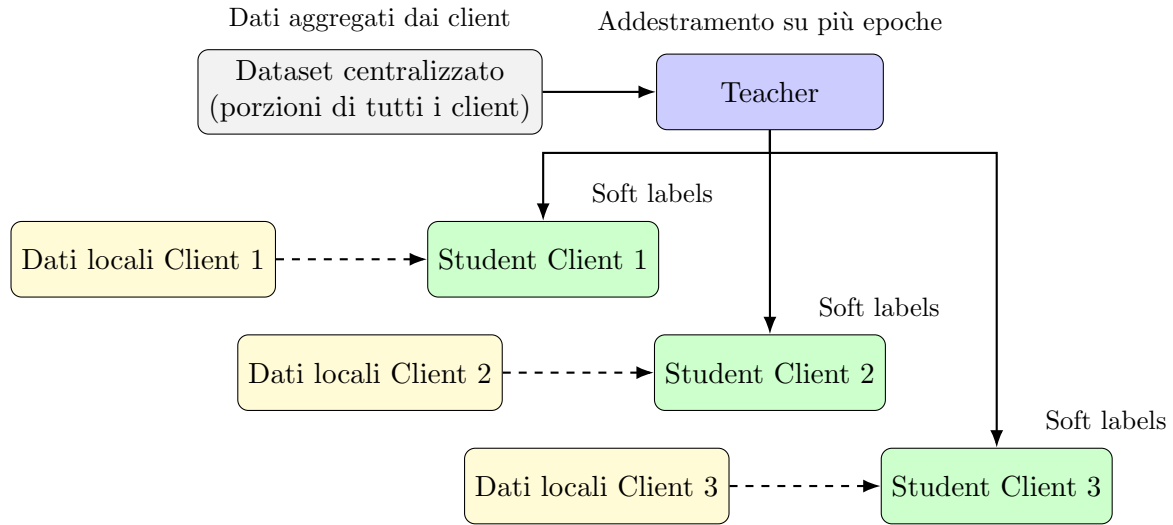


Fig. 2.11: Schema della distillazione centralizzata con più client. Il teacher è addestrato su un dataset aggregato proveniente da tutti i client e genera predizioni ammorbidite (*soft labels*) utilizzate dai modelli student locali. Ogni student utilizza inoltre i propri dati locali rimanenti, non inclusi nel dataset del teacher (freccie tratteggiate).

L'intero processo è schematizzato nell'Algoritmo 2, che mostra il flusso completo dalla costruzione del dataset centralizzato all'aggiornamento dei modelli student.

Algorithm 2 Distillazione centralizzata multi-client (teacher prima, student dopo)

Require: Dataset di ciascun client $\{\mathcal{D}_k\}_{k=1}^K$, numero di epoche $E_{\text{teacher}}, E_{\text{student}}$, teacher f_{teacher} , student locali $\{f_{\text{student},k}\}_{k=1}^K$

Ensure: Modelli student addestrati

- 1: Costruisci il dataset centralizzato $\mathcal{D}_{\text{central}}$ aggregando porzioni dei dati di tutti i client
- 2: **Addestramento del teacher:**
- 3: **for** $e = 1$ to E_{teacher} **do**
- 4: Aggiorna f_{teacher} minimizzando la loss sul dataset centralizzato $\mathcal{D}_{\text{central}}$
- 5: **end for**
- 6: Genera le soft labels $p^{(T)}(x)$ per tutti i dati da fornire ai client
- 7: **Addestramento dei modelli student:**
- 8: **for** ogni client $k = 1, \dots, K$ **in parallelo do**
- 9: **for** $e = 1$ to E_{student} **do**
- 10: Aggiorna lo student $f_{\text{student},k}$ minimizzando la loss di distillazione:

$$\mathcal{L}_{\text{KD},k} = (1 - \alpha) \mathcal{H}(y, q_k(x)) + \alpha T^2 \text{KL}(p^{(T)}(x) \parallel q_k^{(T)}(x))$$

- 11: **end for**
 - 12: **end for**
 - 13: **return** Modelli student addestrati $\{f_{\text{student},k}\}_{k=1}^K$
-

2.3.4 Progettazione sperimentale

La distillazione centralizzata è stata progettata come baseline di riferimento per quantificare il potenziale di trasferimento di conoscenza in assenza di vincoli di privacy. L'obiettivo principale è analizzare: (i) la capacità di generalizzazione dei modelli student su domini non visti in addestramento; (ii) l'impatto della differenza tra architetture feed-forward e modelli generativi (CI-VAE); (iii) la sensibilità ai parametri caratteristici della distillazione (temperatura T , coefficiente α della loss definita in (2.2)).

Dataset e partizionamento. La distillazione centralizzata è stata applicata su entrambi i dataset introdotti in Sez. 2.1.1, ossia FEMNIST e IRDS, già partizionati rispettivamente per *writer_id* e per paziente. In tali scenari, i dati locali di ciascun client sono rimasti invariati, ma la funzione di caricamento è stata adattata per distinguere chiaramente tra i dati utilizzati dal modello *teacher* (centralizzato) e quelli riservati agli *student* locali (cfr. Sez. 3.4.4).

In particolare:

- Nel caso **IRDS**, ogni client presenta una suddivisione a priori in training (70%), validation (15%) e test (15%). Per la distillazione centralizzata, il dataset del teacher è stato costruito aggregando le porzioni di validation di tutti i client, successivamente suddivise tramite split stratificato in sottoinsiemi di training e validation. Gli student hanno invece continuato a utilizzare esclusivamente i dati del proprio client (train e test), senza alcuna condivisione inter-client.
- Nel caso **FEMNIST**, ogni partizione locale è stata originariamente divisa 70/30 tra training e test. Per il teacher, i test set di tutti i client sono stati aggregati in

un unico dataset centralizzato, a sua volta suddiviso in training e validation. Gli student hanno mantenuto i propri dati di addestramento, ripartiti internamente in train (80%) e validation (20%), con seed fissato per garantire riproducibilità.

In entrambi i casi, le immagini originali sono state preprocessate come descritto in Sez. 2.1.1: conversione in RGB, ridimensionamento a 224×224 , normalizzazione con le statistiche di ImageNet e trasformazione in rappresentazioni numeriche a 1280 dimensioni mediante MobileNetV2 pre-addestrata.

Architetture dei modelli. Sono state considerate due famiglie principali di modelli neurali, con l’obiettivo di studiare la distillazione sia in scenari puramente discriminativi, sia in contesti generativi più complessi.

1. **Modelli Multilayer Perceptron (MLP).** Il modello teacher f_T è implementato come una rete feed-forward profonda composta da tre hidden layer con dimensioni (512, 256, 128), ciascuno seguito da attivazione ReLU, batch normalization e dropout. Questa architettura, sufficientemente espressiva, è in grado di apprendere rappresentazioni globali dai dati centralizzati. I modelli student $f_{S,k}$ adottano invece una struttura più compatta, con due soli hidden layer (256, 128), così da simulare scenari di dispositivi con risorse limitate. Entrambe le architetture terminano con uno strato lineare e funzione di attivazione softmax, che produce una distribuzione di probabilità sulle C classi.

2. Modelli CI-VAE.

Oltre ai modelli feed-forward, la distillazione è stata applicata anche a modelli di tipo generativo-discriminativo, basati sul CI-VAE descritto in dettaglio in Sez. 2.2.3. Tale modello combina le proprietà di un VAE — encoder, campionamento latente, decoder simmetrico — con una testa di classificazione nello spazio latente, che guida l’apprendimento di rappresentazioni discriminative.

Nel contesto della distillazione centralizzata, abbiamo distinto due varianti:

- **Teacher.** Il teacher è un CI-VAE ad alta capacità, caratterizzato da un encoder profondo a tre livelli, un decoder simmetrico di pari complessità e un classificatore latente multi-strato. Tale configurazione consente al modello di apprendere rappresentazioni latenti ricche e ben strutturate, adatte a catturare la distribuzione dei dati aggregati.
- **Student.** Gli student sono versioni semplificate dello stesso schema architetturale: l’encoder è ridotto a due livelli, il decoder è simmetrico all’encoder e il classificatore latente è costituito da due layer lineari. Questa riduzione di complessità riflette scenari realistici di client con risorse computazionali ridotte e rende più difficile mantenere la stessa accuratezza del teacher, evidenziando l’efficacia del trasferimento di conoscenza tramite distillazione.

In entrambi i casi, l’architettura mantiene la stessa pipeline funzionale (encoder $\rightarrow$ campionamento latente $\rightarrow$ classificatore e decoder), ma la differenza di profondità e numero di parametri introduce un divario di capacità che la distillazione mira a colmare.

Pipeline sperimentale. Ciascun esperimento segue una pipeline strutturata in quattro fasi principali. Nella prima fase, il modello teacher f_T viene addestrato sul dataset centralizzato $\mathcal{D}_{\text{central}}$ mediante ottimizzazione AdamW, con scheduler StepLR; il checkpoint finale è selezionato tramite *early stopping* sull’accuratezza di validazione. Successivamente, il teacher genera per ogni input x dei client le distribuzioni di probabilità $p^{(T)}(x)$ (Eq. 2.1), che fungono da *soft labels*. Nella terza fase, ogni client k addestra localmente il proprio modello $f_{S,k}$ minimizzando la funzione di distillazione $\mathcal{L}_{\text{KD}}$ (Eq. 2.2), combinando etichette reali e predizioni morbide del teacher. Per i modelli CI-VAE, la loss è ulteriormente arricchita dalla componente generativa (ricostruzione, classificazione e KLD), come descritto in Sez. 2.2.3. Infine, nella fase di valutazione, ciascun modello studente viene testato sia sul proprio dominio sia sui dataset degli altri client, realizzando una *cross-client evaluation*.

Valutazione cross-client. La capacità di generalizzazione dei modelli è misurata costruendo una matrice di accuratezza $M \in \mathbb{R}^{K \times K}$, in cui

$$M_{i,j} = \text{Acc}(f_{S,i}, \mathcal{D}_j),$$

dove $f_{S,i}$ indica lo studente addestrato sul client i e $\mathcal{D}_j$ il test set del client j . La diagonale $M_{i,i}$ rappresenta la performance sul dominio di addestramento, mentre gli elementi fuori diagonale quantificano il trasferimento di conoscenza verso domini eterogenei. Oltre all’accuratezza, vengono riportate metriche complementari di classificazione (F1-score, precision, recall) per una valutazione più completa.

Strutturazione degli esperimenti. Gli esperimenti sono stati progettati con l’obiettivo di valutare la distillazione centralizzata in scenari non-IID, tenendo conto sia della complessità dei dataset sia delle differenze architetturali tra teacher e student. In particolare, su FEMNIST è stata adottata un’architettura MLP, mentre su IRDS sono stati testati sia modelli fully connected sia modelli generativi-discriminativi basati su CI-VAE, così da analizzare la distillazione in contesti con diversa capacità rappresentativa.

Per ciascuno scenario è stata predisposta una griglia di iperparametri, variando sistematicamente i valori di α e della temperatura T , poiché questi due parametri giocano un ruolo centrale nel determinare il compromesso tra accuratezza locale e capacità di generalizzazione cross-client. Oltre a questi, sono stati esplorati anche i tassi di apprendimento e le dimensioni del batch, mantenendo fissi altri parametri di addestramento per garantire confronti controllati.

Le epoche di addestramento sono state fissate a valori sufficientemente elevati per permettere la convergenza, con il rischio di overfitting mitigato dall’impiego di *early stopping* e dalla selezione automatica del miglior checkpoint sia per il teacher sia per gli student.

Questa impostazione ha consentito di indagare in maniera sistematica: (i) la capacità di generalizzazione della distillazione al variare di α e T ; (ii) le differenze di comportamento tra dataset con diversa natura e complessità (scrittura manuale in FEMNIST vs dati clinici in IRDS); (iii) l’impatto della capacità architetturale confrontando MLP e CI-VAE. Il dettaglio delle configurazioni esplorate e dei risultati migliori è riportato nel Capitolo 4.3.

Riproducibilità e tracciamento. Per garantire rigore sperimentale, tutti i seed sono fissati e l'intero processo genera un file CSV contenente: configurazione degli esperimenti (dataset, architettura, iperparametri), metriche locali e cross-client, matrici di accuratezza e statistiche aggregate (medie e deviazioni standard). Questo protocollo consente di effettuare analisi comparative sistematiche e di confrontare in modo diretto la distillazione centralizzata con le strategie federate presentate nei capitoli successivi.

2.4 Paradigma 3: Knowledge Distillation Federata

Dopo aver analizzato il paradigma di distillazione centralizzata, in cui un teacher globale supervisiona i modelli student locali tramite un dataset centralizzato, emerge naturalmente la necessità di estendere tali principi a contesti distribuiti e sensibili alla privacy. Nella knowledge distillation federata, l'idea fondamentale è spostare la collaborazione dal livello dei parametri a quello delle predizioni: invece di sincronizzare pesi o gradienti, i client condividono informazioni derivate dai propri modelli attraverso un dataset pubblico o sintetico, generando distribuzioni di output che veicolano conoscenza globale. In questo contesto, non disponendo di un dataset pubblico reale, si ricorre alla costruzione di un dataset sintetico, generato a partire da statistiche aggregate dei dati locali, che funge da canale informativo condiviso per la distillazione.

Questo approccio consente di preservare la riservatezza dei dati, ridurre il carico comunicativo e favorire la convergenza dei modelli anche in presenza di dati non-IID. Nel presente lavoro vengono esplorate due varianti principali:

- **Federated Model Distillation (FedMD)**, in cui ogni client mantiene il proprio modello locale e apprende attraverso l'allineamento alle predizioni aggregate calcolate su un dataset sintetico condiviso. Non viene prodotto un modello globale finale, ma ogni dispositivo ottiene un proprio modello adattato tramite distillazione collaborativa.
- **Federated Knowledge Distillation (FedKD)**, in cui i client contribuiscono all'addestramento di un unico modello globale, che viene supervisionato tramite le predizioni aggregate dei modelli locali su un dataset sintetico. Il risultato del processo è quindi un modello centrale condiviso da tutti.

2.4.1 Architettura e flusso operativo (FedMD)

Il paradigma di FedMD, introdotto da Li et al., rappresenta una delle prime estensioni sistematiche della distillazione della conoscenza al contesto federato. A differenza degli approcci tradizionali come FedAvg, che prevedono la sincronizzazione periodica dei pesi dei modelli, FedMD si basa esclusivamente sullo scambio di predizioni. In questo schema, i client non condividono mai né i dati privati né i parametri dei modelli, ma cooperano tramite le distribuzioni di output calcolate su un dataset pubblico condiviso, $\mathcal{D}_{\text{pub}}$.

Il protocollo FedMD si sviluppa in round federati, ciascuno composto da due fasi distinte e cicliche. In primo luogo, ogni client esegue un addestramento supervisionato sul proprio dataset privato $\mathcal{D}_k$, ottimizzando la funzione di cross-entropy rispetto alle etichette vere. Questa fase permette al modello locale f_k di adattarsi alle caratteristiche specifiche dei propri dati, mantenendo l'indipendenza strutturale del modello.

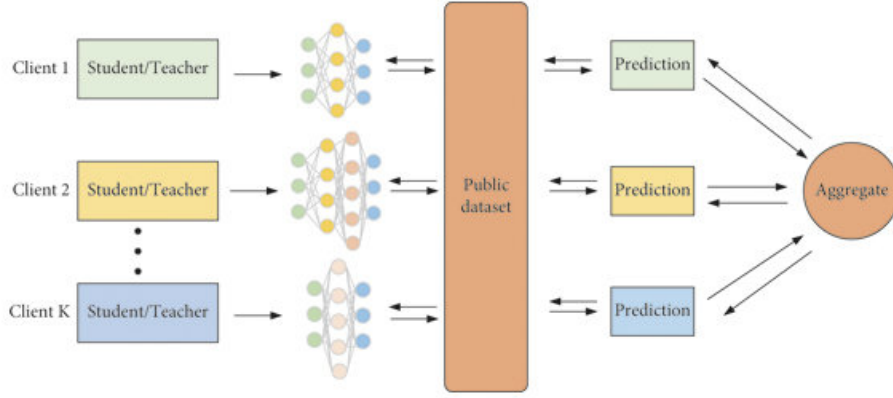


Fig. 2.12: Architettura operativa del paradigma FedMD. Ogni client genera predizioni sui dati pubblici, che vengono aggregate dal server centrale per produrre una distribuzione consensuale. I client aggiornano localmente i propri modelli allineandosi a queste predizioni tramite distillazione.

Successivamente, i client utilizzano il dataset pubblico $\mathcal{D}_{\text{pub}}$ per generare predizioni sotto forma di vettori di logit ammorbiditi (tramite softmax a temperatura $T > 1$), che vengono inviati al server centrale. Il server aggrega queste predizioni calcolando, per ogni campione $x \in \mathcal{D}_{\text{pub}}$, una distribuzione di consenso $\bar{P}(x)$, ottenuta mediante una media pesata rispetto alla dimensione dei dataset locali dei client.

La distribuzione aggregata viene poi redistribuita a tutti i partecipanti, che aggiornano localmente i propri modelli utilizzando le *soft-label* $\bar{P}(x)$ come target di distillazione. In questa fase, ciascun client ottimizza la loss $\mathcal{L}_k$, che combina l'informazione proveniente dalle soft-labels aggregate con quella etichette del dataset pubblico, secondo la formula Eq. 2.2. L'intero processo è iterato per un numero prefissato di round, consentendo a tutti i modelli di convergere verso una rappresentazione coerente e informata collettivamente, senza scambio diretto di pesi o dati sensibili.

Il processo di FedMD si articola secondo la procedura illustrata nell'Algoritmo 3, in cui ciascun client alterna addestramento locale supervisionato e aggiornamento tramite distillazione.

Algorithm 3 Federated Model Distillation (FedMD)

- 1: **Input:** Dataset pubblico $\mathcal{D}_{\text{pub}}$, dataset privati $\{\mathcal{D}_k\}_{k=1}^K$, modelli locali $\{f_k\}_{k=1}^K$, numero di round T , epoche locali E
 - 2: **for** $t = 1$ to T **do**
 - 3: **for all** client $k \in \mathcal{C}$ **in parallelo do**
 - 4: Addestra localmente f_k su $\mathcal{D}_k$ per E epoche minimizzando $\mathcal{H}(y, f_k(x))$
 - 5: Calcola logit $P_k(x) = f_k(x)$ per ogni $x \in \mathcal{D}_{\text{pub}}$ e invia al server
 - 6: **end for**
 - 7: Il server calcola $\bar{P}(x) = \frac{1}{K} \sum_{k=1}^K P_k(x)$ per ogni $x \in \mathcal{D}_{\text{pub}}$
 - 8: **for all** client $k \in \mathcal{C}$ **in parallelo do**
 - 9: Riceve $\bar{P}(x)$ dal server
 - 10: Aggiorna f_k minimizzando $\text{KL}(\bar{P}^{(T)}(x) \parallel f_k^{(T)}(x))$ su $\mathcal{D}_{\text{pub}}$
 - 11: **end for**
 - 12: **end for**
 - 13: **return** Modelli distillati $\{f_k\}_{k=1}^K$
-

Generazione del dataset sintetico

Nel paradigma FedMD, il dataset pubblico rappresenta uno strumento essenziale per abilitare la distillazione collaborativa tra client senza scambio diretto di dati sensibili. In scenari in cui non è disponibile un dataset pubblico reale è necessario adottare una strategia alternativa per fornire ai modelli un terreno comune di confronto.

In questa implementazione, il dataset pubblico è costruito artificialmente mediante un processo di generazione sintetica nel dominio delle rappresentazioni intermedie, ovvero nello spazio delle feature vettoriali prodotte da una rete MobileNetV2 pre-addestrata e congelata. Gli embedding rappresentano uno spazio semantico già strutturato, condiviso da tutti i client, che cattura l'informazione discriminante in forma vettoriale e astratta, mantenendo al contempo una maggiore neutralità rispetto ai dati originali.

Nel presente lavoro, il dataset pubblico è stato progettato come un campionamento controllato da una distribuzione *Gaussian Mixture Model* (GMM), costruita a partire da statistiche aggregate sulle distribuzioni dei dati privati. Formalmente, siano K i client partecipanti, ognuno con un insieme di feature locali $\mathcal{D}_k = \{(x_i^{(k)}, y_i^{(k)})\}$. Per ogni classe $c \in \{1, \dots, C\}$, viene calcolata una media μ_c e una deviazione standard σ_c aggregando i momenti statistici delle distribuzioni locali, ossia:

$$\mu_c = \frac{1}{K_c} \sum_{k \in \mathcal{C}_c} \mu_c^{(k)}, \quad \sigma_c^2 = \frac{1}{K_c} \sum_{k \in \mathcal{C}_c} (\sigma_c^{(k)})^2,$$

dove $\mathcal{C}_c \subseteq \{1, \dots, K\}$ denota l'insieme dei client che possiedono dati della classe c , e $\mu_c^{(k)}, \sigma_c^{(k)}$ sono i momenti statistici calcolati localmente sul client k .

Il dataset sintetico viene generato campionando vettori $x \in \mathbb{R}^d$ secondo una distribuzione gaussiana multivariata per ciascuna classe:

$$x \sim \mathcal{N}(\mu_c, \Sigma_c),$$

dove $\Sigma_c = \text{diag}(\sigma_c^2)$ è una matrice diagonale che incorpora la varianza per dimensione. I vettori così generati vengono normalizzati e clampati entro un intervallo prefissato, per riflettere le proprietà osservate nei dati reali.

Sebbene le etichette non derivino da osservazioni reali, esse forniscono una partizione coerente dello spazio semantico, utile per supportare l'allineamento dei modelli locali. La distillazione che ne deriva sfrutta sia queste etichette sintetiche, sia la distribuzione di consenso ottenuta dall'aggregazione delle predizioni dei client, secondo una funzione di perdita combinata (cfr. Sez. 3.5.4).

2.4.2 Progettazione sperimentale

La progettazione degli esperimenti con FedMD è stata guidata da due obiettivi principali: da un lato riprodurre scenari realistici in cui i dati siano distribuiti in maniera non-IID tra client diversi, dall'altro garantire un confronto equo e controllato con i paradigmi di federated learning classico e di distillazione centralizzata. Le scelte effettuate su dataset, modelli e protocolli operativi mirano a bilanciare queste esigenze, mantenendo al tempo stesso semplicità e riproducibilità.

Dataset e partizionamento. Gli esperimenti sono stati condotti sui due dataset non-IID introdotti in Sez. 2.1.1, ossia FEMNIST, partizionato per *writer_id*, e IRDS, partizionato per paziente. In entrambi i casi sono stati considerati scenari con $K = \{3, 5, 7\}$ client, così da analizzare l'impatto della numerosità dei partecipanti e della frammentazione dei dati sulla capacità di generalizzazione. Per FEMNIST il dataset è stato ristretto a dieci classi di caratteri manoscritti, con suddivisione 70% training e 30% test per ciascun client. Per IRDS, ridotto a quattro classi di movimenti riabilitativi, ogni client corrisponde a un paziente e i dati sono stati divisi a priori in 70% training, 15% validation e 15% test; validation e test sono stati poi aggregati in un unico insieme di valutazione. Poiché FedMD richiede la disponibilità di un dataset pubblico, è stato inoltre generato un dataset sintetico nello spazio delle rappresentazioni intermedie (embedding a 1280 dimensioni ottenuti da MobileNetV2 pre-addestrata). I campioni sono stati estratti da un modello a Gaussian Mixture parametrizzato a partire da statistiche aggregate dei client, fornendo così un terreno comune per la cooperazione senza violare l'indipendenza dei dati privati.

Modelli locali. I client utilizzano un MLP compatto, progettato per operare sugli embedding a 1280 dimensioni forniti da MobileNetV2. L'architettura, costituita da tre layer completamente connessi con dimensioni (512, 256, 128) e un classificatore finale con softmax, è stata scelta per due motivi: da un lato ridurre il carico computazionale lato client, dall'altro isolare l'efficacia del protocollo FedMD evitando che differenze architetturali introducano variabilità nei risultati. In questo modo è possibile attribuire con maggiore confidenza le differenze osservate alle caratteristiche del paradigma e non alla capacità espressiva dei modelli.

Gestione degli esperimenti. La valutazione di FedMD è stata organizzata come una ricerca sistematica degli iperparametri tramite *grid search*. Questa scelta, seppur più onerosa rispetto ad approcci euristici come il random search, consente un'esplorazione esaustiva e controllata dello spazio delle configurazioni, permettendo di isolare con maggiore affidabilità l'effetto dei singoli parametri sul comportamento del modello. Sono stati quindi definiti insiemi discreti di valori per tasso di apprendimento, temperatura T , numero di epoche locali e di consenso, batch size e coefficiente α ; il prodotto cartesiano di tali insiemi ha costituito la griglia da esplorare.

Ogni esperimento ha seguito una pipeline strutturata in round federati: i modelli locali sono stati inizialmente addestrati in maniera supervisionata sui dati privati, successivamente hanno prodotto predizioni sul dataset sintetico condiviso, le quali sono state aggregate centralmente per generare una distribuzione di consenso. Tale distribuzione è stata poi utilizzata per riallineare i modelli tramite distillazione, prima di procedere alla valutazione sia sui dati propri sia su quelli degli altri client. L'intero processo è stato monitorato round per round e dotato di un meccanismo di *early stopping* basato sull'andamento delle metriche di validazione, con selezione del miglior checkpoint per garantire stabilità e ridurre il rischio di overfitting.

Per ciascuna configurazione, la procedura è stata ripetuta considerando scenari con tre, cinque e sette client, così da analizzare la sensibilità del protocollo rispetto al grado di eterogeneità e frammentazione dei dati. Ogni run ha prodotto un file CSV contenente la configurazione, le metriche locali e cross-client (accuratezza, precision, recall e F1-score)

e la matrice di accuratezza inter-client, utile per individuare nel dettaglio i pattern di trasferibilità.

In questo modo è stato possibile caratterizzare in maniera sistematica l’impatto degli iperparametri chiave di FedMD, con particolare attenzione alla temperatura e al coefficiente α , sul bilanciamento tra apprendimento locale e trasferimento di conoscenza. Inoltre, l’analisi su differenti livelli di numerosità dei client ha consentito di valutare la resilienza del paradigma in scenari più o meno frammentati. Infine, l’impostazione sperimentale ha fornito una baseline solida e riproducibile, direttamente confrontabile con la distillazione centralizzata e con gli altri paradigmi federati.

2.4.3 Architettura e flusso operativo (FedKD)

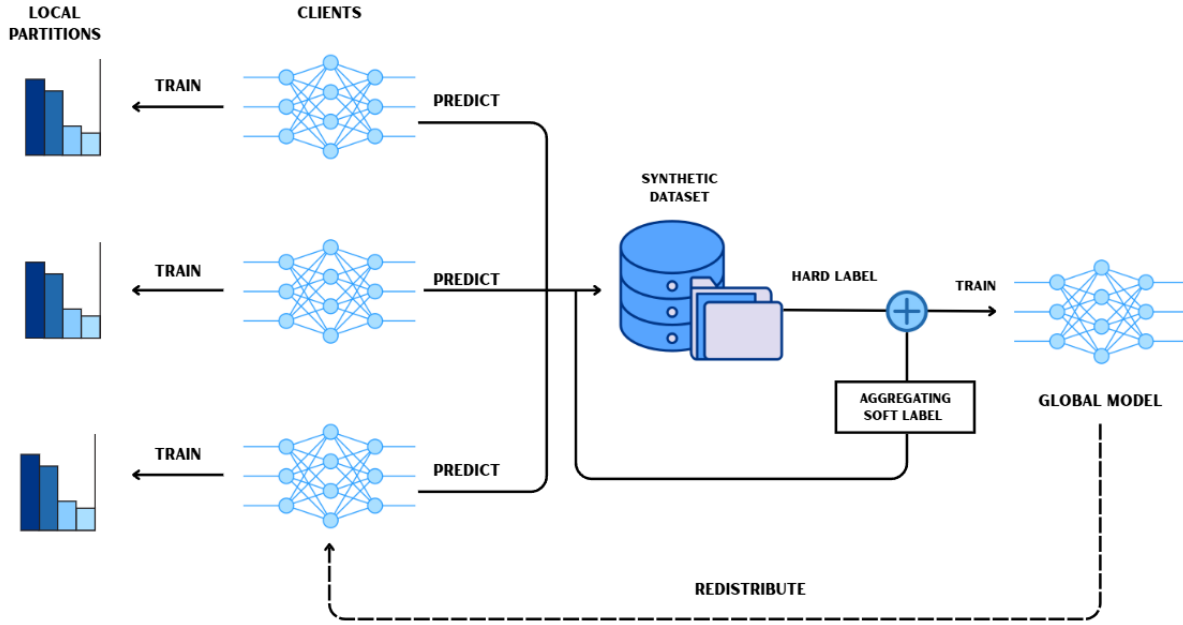


Fig. 2.13: Schema del processo di FedKD. Ogni client si addestra localmente sul proprio dataset non-IID (*local partitions*) e utilizza il modello risultante per generare predizioni (*soft labels*) su un dataset sintetico condiviso. Le predizioni dei client vengono aggregate dal server, che combina le *soft labels* con le *hard labels* del dataset sintetico per addestrare il modello globale mediante distillazione. Il modello aggiornato viene infine ridistribuito ai client, chiudendo il ciclo federato.

In questo paradigma di distillazione federata, la cooperazione tra client avviene non tramite la condivisione di pesi, gradienti o dati privati, bensì attraverso l'aggregazione di predizioni probabilistiche (*soft labels*) ottenute su un dataset condiviso. Tuttavia, in assenza di un dataset pubblico reale, tale base comune viene generata in forma sintetica a partire da statistiche aggregate e anonimizzate, seguendo una strategia ispirata al paradigma FedDF.

L'architettura segue una struttura sincrona di tipo *client-server* organizzata in round federati successivi. Ad ogni round, ogni client k dispone di un dataset privato $\mathcal{D}_k$ e di un modello locale f_k , inizializzato a partire dal modello globale corrente f_g . Il client addestra localmente il proprio modello per un numero predefinito di epoche, utilizzando le rappresentazioni intermedie estratte da una rete MobileNetV2 pre-addestrata e congelata. Al termine dell'addestramento, ciascun client calcola le statistiche di primo e secondo ordine (media e deviazione standard) delle proprie feature, suddivise per classe, e le invia al server.

Il server utilizza queste informazioni per generare un dataset sintetico condiviso $\mathcal{D}_{\text{syn}}$, campionando vettori latenti da distribuzioni gaussiane multivariate calibrate sulle statistiche ricevute. Il processo di generazione sintetica adotta una strategia analoga a quella descritta nella sezione 2.4.1.

Una volta generato $\mathcal{D}_{\text{syn}}$, ciascun client calcola le predizioni $P_k(x)$ del proprio modello su tutti i campioni $x \in \mathcal{D}_{\text{syn}}$, producendo logit normalizzati mediante softmax. Tali predizioni vengono trasmesse al server, che ne esegue un'aggregazione ponderata per ottenere una distribuzione consensuale $\bar{P}(x)$. I pesi dell'aggregazione possono essere assegnati in

funzione della cardinalità dei dataset locali.

Il server utilizza le distribuzioni aggregate $\bar{P}(x)$ come etichette morbide di riferimento per aggiornare il modello globale f_g . L'ottimizzazione viene effettuata minimizzando una funzione di perdita che combina la cross-entropy supervisionata sulle etichette sintetiche y e una penalità di distillazione basata sulla divergenza di Kullback–Leibler tra le distribuzioni $\bar{P}^{(T)}(x)$ e $f_g^{(T)}(x)$, ammorbidite da una temperatura $T > 1$. La loss complessiva adottata è:

$$\mathcal{L}_{\text{FedKD}}(x, y) = (1 - \alpha) \mathcal{H}(y, f_g(x)) + \alpha T^2 \text{KL}(\bar{P}^{(T)}(x) \parallel f_g^{(T)}(x)),$$

dove $\alpha \in [0, 1]$ è un iperparametro che regola il bilanciamento tra supervisione simbolica e distillazione collaborativa.

Il modello globale viene quindi ridistribuito ad ogni client. Il ciclo viene ripetuto per un numero predefinito di round federati ed è mostrato nell'algoritmo 4. A differenza di algoritmi come FedAvg, il modello globale f_g non viene ridistribuito ai client dopo ogni round: l'apprendimento avviene esclusivamente lato server, sulla base delle conoscenze aggregate via soft labels. Sebbene il framework FedDF originale consenta sia modalità con che senza ridistribuzione, in questo lavoro si adotta la variante con ridistribuzione per facilitare la convergenza.

Rispetto alla formulazione originale di FedDF, che prevede la generazione di feature sintetiche tramite generatori neurali o GAN, il metodo qui proposto si basa su un approccio statistico più interpretabile e facilmente controllabile.

Algorithm 4 Federated Knowledge Distillation (FedKD)

```

1: Input: Dataset locali  $\{\mathcal{D}_k\}_{k=1}^K$ , modelli locali  $\{f_k\}_{k=1}^K$ , parametri  $\alpha, T$ , epoche locali  $E$ , round globali  $R$ 
2: Output: Modello globale distillato  $f_g$ 
3: for  $r = 1$  to  $R$  do
4:   for all client  $k$  in parallelo do
5:     Addestra  $f_k$  su  $\mathcal{D}_k$  per  $E$  epoche
6:     Calcola  $(\mu_c^{(k)}, \sigma_c^{(k)})$  per ogni classe  $c$  e invia al server
7:   end for
8:   Il server aggrega le statistiche  $(\mu_c, \sigma_c)$  e genera  $\mathcal{D}_{\text{syn}}$ 
9:   for all client  $k$  in parallelo do
10:    Calcola  $P_k(x) = f_k(x)$  per ogni  $x \in \mathcal{D}_{\text{syn}}$  e invia al server
11:   end for
12:   Calcola  $\bar{P}(x) = \sum_k w_k \cdot P_k(x)$ 
13:   Aggiorna  $f_g$  minimizzando  $\mathcal{L}_{\text{FedKD}}(x, y)$ 
14:   Distribuisci  $f_g$  aggiornato a tutti i client
15: end for
16: return  $f_g$ 

```

2.4.4 Progettazione sperimentale

La progettazione degli esperimenti con FedKD è stata guidata dall'obiettivo di valutare in maniera sistematica l'efficacia della distillazione federata basata su dati sintetici,

confrontandola con i paradigmi alternativi analizzati nei capitoli precedenti. Poiché FedKD rappresenta un adattamento del paradigma FedDF con un generatore statistico più leggero e interpretabile, le scelte metodologiche mirano da un lato a riflettere scenari clinici realistici (dati distribuiti, eterogenei e sensibili), dall'altro a mantenere condizioni di confronto eque con FedAvg, FedProx, FedOpt e FedMD. Le decisioni riguardanti dataset, modelli, procedure di addestramento e metriche sono quindi state motivate dalla necessità di bilanciare tre aspetti fondamentali: (i) preservare la privacy dei dati clinici evitando qualsiasi condivisione diretta, (ii) ridurre la complessità computazionale per favorire la scalabilità, (iii) garantire riproducibilità e trasparenza dei risultati tramite un protocollo sperimentale controllato.

Gestione dei dataset. Gli esperimenti con FedKD sono stati condotti sui dataset non-IID *FEMNIST* e *IRDS* (cfr. Sez. 2.1.1), scelti perché rappresentano due scenari complementari: un benchmark ampiamente utilizzato per valutare metodi federati e un corpus clinico reale, caratterizzato da forte eterogeneità tra i client. Per analizzare l'impatto della frammentazione, sono stati considerati scenari con $K \in \{3, 5, 7\}$ client. In *FEMNIST* i dati sono stati partizionati per *writer_id*, limitandosi a dieci classi, così da simulare il caso in cui ciascun client disponga solo di un sottoinsieme del dominio e garantire confronti più interpretabili. In *IRDS*, invece, ogni client corrisponde a un paziente, riflettendo la distribuzione naturale dei dati clinici. La suddivisione è stata fissata a 70% training, 15% validation e 15% test, unendo validation e test nella fase di valutazione. In assenza di un dataset pubblico reale, la fase di distillazione è stata supportata da un dataset sintetico nello spazio delle rappresentazioni intermedie: si sono utilizzati embedding a 1280 dimensioni da MobileNetV2 congelata, generando 1000 campioni tramite un Gaussian Mixture stimato su statistiche aggregate per classe. Questa scelta fornisce un terreno comune neutro, preservando la separazione dei dati privati e rendendo controllabile la qualità del segnale di consenso.

Scelta dei modelli. Sia i client sia il server condividono lo stesso classificatore, implementato come un *MLP* progettato per operare direttamente sugli embedding di dimensione 1280 estratti da MobileNetV2. L'architettura è volutamente compatta ma sufficientemente espressiva per catturare pattern discriminativi nello spazio latente. In particolare, il modello prevede una prima proiezione lineare da 1280 a 512 dimensioni, seguita da normalizzazione batch, funzione di attivazione ReLU e un dropout con probabilità pari a 0.3, allo scopo di favorire la generalizzazione. La seconda trasformazione riduce ulteriormente la dimensionalità a 256 neuroni e impiega nuovamente batch normalization, ReLU e un dropout più leggero (0.2), così da ottenere una rappresentazione più compatta e regolarizzata. Un ulteriore livello nascosto da 128 neuroni consolida le feature in uno spazio intermedio più astratto, mentre lo strato finale lineare mappa le rappresentazioni nello spazio delle classi, producendo logit non normalizzati (10 classi per *FEMNIST* e 4 per *IRDS*).

Questa configurazione architetturale rappresenta un compromesso mirato: da un lato la leggerezza del modello consente l'esecuzione distribuita su più client senza eccessivi costi computazionali, dall'altro la presenza di tre livelli nascosti con dropout e batch normalization conferisce al classificatore sufficiente capacità espressiva per trarre vantaggio dalle fasi di distillazione. Inoltre, l'adozione di un'architettura omogenea tra client e server

elimina la variabilità dovuta al modello e permette di attribuire le differenze prestazionali esclusivamente alle dinamiche del protocollo FedKD.

Gestione degli esperimenti. La valutazione è stata organizzata con una *grid search* esaustiva su insiemi discreti di iperparametri, ripetendo ogni configurazione per $K \in \{3, 5, 7\}$. Sono stati esplorati: *learning rate*, *temperatura* T (per modulare l’ammorbidimento dei logit), numero di *epoche locali* e di *epoche di distillazione* (che definiscono, rispettivamente, la spinta alla specializzazione sul dominio privato e la profondità dell’allineamento al consenso) e il *coefficiente* $\alpha \in [0, 1]$ della loss (peso relativo tra cross-entropy sulle etichette sintetiche e distillazione via KL). La scelta di una griglia, pur più onerosa di una random search, garantisce copertura sistematica e consente di misurare in modo controllato l’effetto marginale di ciascun iperparametro in condizioni di non-IID. Tutti gli esperimenti sono stati eseguiti per $R = 70$ round federati, con monitoraggio round-by-round e *early stopping* sul tracciato globale: il training si arresta se non si osserva un miglioramento minimo dopo una finestra di pazienza prefissata, mentre il modello selezionato è il checkpoint con migliore metrica di validazione aggregata. Ogni run produce file CSV con configurazione, metriche globali e per-client, e, ove pertinente, matrici di accuratezza inter-client, così da abilitare confronti diretti e analisi post-hoc riproducibili.

Metriche e obiettivi. Le metriche considerate includono accuratezza globale, accuratezza per client, precision, recall e F1-score. Questo set è stato scelto per fornire una valutazione completa: non solo della performance media, ma anche della coerenza del modello tra domini eterogenei. L’obiettivo è verificare se FedKD riesca a bilanciare specializzazione locale e trasferibilità, chiarendo come parametri chiave quali T , α , il numero di round e le epoche di distillazione influenzino il punto di equilibrio tra apprendimento guidato da etichette sintetiche e allineamento alle distribuzioni di consenso. In questo modo si può valutare se un approccio statistico leggero e interpretabile possa costituire un’alternativa praticabile ad altre forme più complesse di distillazione federata.

Capitolo 3

Implementazione

In questo capitolo vengono descritte le scelte tecnologiche e le soluzioni implementative adottate per realizzare i paradigmi presentati nella fase di progettazione. La sezione è organizzata in base ai diversi approcci considerati, con particolare attenzione al flusso operativo, all'infrastruttura software e agli aspetti sperimentali.

3.1 Tecnologie e librerie utilizzate

La realizzazione dell'intero framework federato si è basata sull'utilizzo del linguaggio Python, integrato con un insieme di librerie open-source consolidate per il deep learning, il calcolo scientifico e la simulazione federata. Di seguito vengono descritte le principali tecnologie adottate, motivandone l'impiego in funzione dei requisiti sperimentali e implementativi del progetto.

3.1.1 Python

Python è un linguaggio di programmazione ad alto livello, interpretato e dinamico, progettato con l'obiettivo di promuovere leggibilità e produttività. Nato nei primi anni '90, è oggi uno degli strumenti principali nello sviluppo di applicazioni scientifiche, data science e sistemi di apprendimento automatico. La sua sintassi essenziale, l'ampio ecosistema di librerie e il supporto a paradigmi di programmazione imperativa, funzionale e orientata agli oggetti lo rendono particolarmente adatto alla prototipazione rapida e alla sperimentazione su larga scala.

Nel presente lavoro, Python costituisce la base dello stack tecnologico adottato. Tutti i componenti del sistema federato sono stati implementati in ambiente Python, incluse le componenti di addestramento locale (client), orchestrazione server, generazione dei dataset sintetici e analisi dei risultati sperimentali. Inoltre, la modularità del linguaggio ha permesso la separazione chiara tra le diverse componenti funzionali del progetto, facilitando la manutenzione e l'estensibilità del codice.

Le principali librerie scientifiche utilizzate comprendono: `PyTorch`, per la definizione e l'addestramento dei modelli neurali; `Flower`, per la simulazione e il coordinamento del processo federato; `NumPy` e `Pandas`, per la manipolazione di array e strutture dati complesse; `scikit-learn`, per operazioni statistiche e metriche di valutazione; `Matplotlib` e `Seaborn`, per la visualizzazione grafica dei risultati.

Python si è rivelato uno strumento efficace non solo per l'implementazione degli algoritmi, ma anche per la gestione del ciclo completo di sviluppo sperimentale, garantendo portabilità, riproducibilità e integrazione con ambienti di calcolo distribuito.

3.1.2 PyTorch

PyTorch [17] rappresenta il framework di riferimento per l'implementazione delle reti neurali profonde in questo lavoro. Si tratta di una libreria open-source sviluppata da Facebook AI Research (FAIR), basata su grafi computazionali dinamici (*define-by-run*) che garantiscono flessibilità ed efficienza nella sperimentazione e nel deployment.

Nel contesto di questa tesi, PyTorch è stato utilizzato nelle sue principali componenti:

- `torch`: costituisce il nucleo della libreria, fornendo le primitive per la manipolazione di tensori multidimensionali, il supporto a operazioni lineari e non lineari e l'integrazione nativa con GPU attraverso CUDA.
- `torch.nn`: modulo fondamentale per la definizione delle architetture neurali. Esso consente di comporre modelli come sottoclassi di `nn.Module`, integrando strati standard (lineari, convoluzionali, dropout, batch normalization) e funzioni di attivazione. In questo lavoro è stato impiegato per implementare sia classificatori leggeri che architetture più complesse come il CI-VAE.
- `torch.nn.functional`: fornisce un accesso diretto alle funzioni matematiche e alle trasformazioni comunemente usate nei layer, come `F.relu`, `F.softmax` o `F.cross_entropy`. Questo approccio consente una maggiore flessibilità nella definizione di forward pass personalizzati, senza la necessità di costruire esplicitamente layer modulari.
- `torch.utils.data`: modulo utilizzato per la gestione dei dataset e dei processi di campionamento. In particolare, `Dataset` e `DataLoader` permettono di strutturare i dati locali (client) e pubblici (distillazione) in maniera scalabile, garantendo *batch processing*, shuffle dei campioni e parallelizzazione del caricamento.

Queste componenti, integrate tra loro, hanno consentito la realizzazione delle pipeline federate e distillative sperimentate: dalla costruzione dei modelli neurali locali alla gestione dei dati distribuiti.

3.1.3 Framework Flower (flwr)

`Flower` [1] è un framework open-source per la ricerca e lo sviluppo di sistemi di *Federated Learning* concepito per essere modulare. Il modello architetturale separa chiaramente le responsabilità tra lato *server* (orchestrazione dei round, selezione dei client, aggregazione) e lato *client* (addestramento locale, valutazione, serializzazione del modello). La letteratura e la documentazione tecnica di `Flower` enfatizzano: (i) il supporto a differenti strategie di aggregazione lato server, (ii) il disaccoppiamento tra logica di addestramento locale e protocolli di federazione, e (iii) un ecosistema di utilità per il benchmarking (ad es. `flwr_datasets`) [1].

Componenti principali impiegati. Nel nostro progetto utilizziamo in modo esplicito le seguenti astrazioni/utility di `flwr`:

- `ClientApp`, `NumPyClient` (`flwr.client`): interfacce per definire il ciclo locale di *fit* e *evaluate* con scambio di pesi/metriche in formati leggeri; `NumPyClient` semplifica l'integrazione quando il modello può essere serializzato in array `NumPy`.
- `Context` (`flwr.common`): veicola metadati di esecuzione (configurazioni, *seed*, parametri di run) e consente comportamenti riproducibili tra round.
- `ServerApp`, `ServerAppComponents`, `ServerConfig` (`flwr.server`): definiscono l'orchestrazione di un esperimento FL (numero di round, frazione di client partecipanti, timeout) e compongono gli elementi del server (selezionatore di client, strategia, storage).
- Strategie lato server (`flwr.server.strategy`): `FedAvg`, `FedProx`, `FedAdagrad`, `FedAdam`, tutte con parametri configurabili (p. es. frazioni di selezione, funzioni di valutazione globale, *accept_failures*).
- `flwr_datasets` (`FederatedDataset`, `IidPartitioner`): modulo ausiliario per caricare dataset e partizionarli in più *client shards*. `FederatedDataset` fornisce un'API uniforme per reperire split (train/val/test) e partizioni; `IidPartitioner` crea suddivisioni i.i.d. controllando il numero di client e la riproducibilità.

3.1.4 Pandas

`Pandas` è una libreria open-source per il linguaggio Python, progettata per la manipolazione e l'analisi di dati etichettati e strutturati. Introdotta da Wes McKinney nel 2008, si è rapidamente affermata come standard de facto nell'ambito della data science grazie alla combinazione tra efficienza computazionale e semplicità d'uso.

La libreria si fonda su due strutture dati principali:

- la `Series`, un array unidimensionale indicizzato;
- il `DataFrame`, una tabella bidimensionale con indici e colonne etichettate.

Queste astrazioni consentono di eseguire operazioni avanzate di filtraggio, selezione, aggregazione, fusione, trasformazione e gestione di valori mancanti, rendendo `pandas` particolarmente adatta per l'analisi esplorativa dei dati e la preparazione di dataset complessi.

3.1.5 Libreria NumPy

`NumPy` (*Numerical Python*) è una libreria fondamentale per il calcolo scientifico in Python, sviluppata a partire dal progetto `Numeric` di Jim Hugunin negli anni '90 e consolidata in seguito da Travis Oliphant con la creazione di `NumPy` nel 2006. Essa fornisce un'infrastruttura efficiente per la gestione di array multidimensionali (`ndarray`), su cui si basano gran parte delle librerie di machine learning e deep learning moderne, tra cui `PyTorch`, `TensorFlow` e `scikit-learn`.

Le caratteristiche principali includono:

- implementazione ottimizzata di operazioni vettoriali e matriciali, che riduce drasticamente l'overhead rispetto a cicli espliciti in Python puro;
- ampio supporto per funzioni matematiche e statistiche, tra cui decomposizioni algebriche, trasformate di Fourier, operazioni randomiche e interpolazioni;
- interoperabilità con linguaggi di basso livello come C e Fortran, che permette di raggiungere prestazioni prossime a quelle del codice compilato;
- compatibilità con i principali framework di data science e machine learning, grazie a un'API stabile e standardizzata.

3.1.6 Libreria `scikit-learn`

`scikit-learn` è una delle librerie più utilizzate per il *machine learning* in Python, introdotta inizialmente come estensione di `SciPy` e successivamente sviluppata come progetto indipendente. La libreria fornisce un'ampia gamma di strumenti per la modellazione predittiva e l'analisi dei dati, tra cui algoritmi supervisionati e non supervisionati, metodi di riduzione dimensionale, tecniche di validazione incrociata e metriche di valutazione standardizzate.

3.1.7 Modulo `itertools`

Il modulo `itertools`, parte integrante della libreria standard di Python, fornisce un insieme di strumenti per la manipolazione efficiente di iteratori. Gli iteratori consentono di generare sequenze di dati in modo *lazy*, evitando il caricamento in memoria di strutture complete e garantendo così una gestione più efficiente delle risorse computazionali. Le funzioni principali di `itertools` includono operatori per la combinatoria (`product`, `permutations`, `combinations`), per l'elaborazione sequenziale (`chain`, `cycle`, `repeat`) e per il filtraggio (`filterfalse`, `islice`). In contesti di *machine learning*, queste funzioni si rivelano utili per la generazione controllata di sottoinsiemi, la gestione di batch o la costruzione di dataset sintetici.

Nel presente lavoro, `itertools` è stato impiegato come supporto alla generazione di strutture iterative complesse all'organizzazione dei task federati. Grazie alla sua natura ottimizzata in C, il modulo consente di ridurre l'overhead computazionale rispetto a implementazioni puramente Python, migliorando la scalabilità del sistema.

3.1.8 Libreria `matplotlib`

`matplotlib` rappresenta una delle librerie fondamentali per la visualizzazione scientifica in Python. Essa fornisce un insieme completo di strumenti per la creazione di grafici bidimensionali e tridimensionali, consentendo un controllo dettagliato sugli elementi visivi, quali assi, etichette, colori e annotazioni.

Nel contesto della presente tesi, `matplotlib` è stato utilizzato principalmente per la rappresentazione grafica delle metriche di performance e per la visualizzazione comparativa dei risultati ottenuti dai diversi paradigmi.

3.1.9 Libreria `seaborn`

`seaborn` è una libreria di data visualization costruita sopra `matplotlib`, che ne estende le funzionalità introducendo un'interfaccia ad alto livello e stili grafici ottimizzati. Essa integra in maniera nativa la compatibilità con `pandas`, consentendo di generare rappresentazioni statistiche complesse direttamente a partire da `DataFrame`. Tra le sue caratteristiche principali vi sono il supporto per heatmap, distribuzioni di densità, boxplot e pairplot, strumenti fondamentali per l'analisi esplorativa dei dati.

In questo lavoro, `seaborn` è stato impiegato come complemento a `matplotlib` per produrre visualizzazioni statistiche più interpretative e di immediata leggibilità.

3.1.10 Altre librerie ausiliarie

Tra le altre librerie:

- `math` per operazioni matematiche di base (es. logaritmi, radici);
- `random` per la gestione dei seed o per la selezione casuale dei client;
- `csv` per registrare i risultati degli esperimenti in formato leggibile e portabile.

Tali librerie, pur essendo standard di Python, risultano essenziali per garantire stabilità, riproducibilità e modularità al codice.

3.2 Preprocessing, partizionamento e caricamento dei dati

In questa sezione descriviamo la pipeline adottata per il preprocessing e la gestione federata dei due dataset utilizzati: FEMNIST e IRDS. L'obiettivo è uniformare i formati dei dati e fornire un'interfaccia di caricamento coerente per i modelli locali.

3.2.1 Estrazione delle feature con MobileNetV2

Per entrambi i dataset, le immagini grezze sono state preprocessate tramite MobileNetV2 pre-addestrata su ImageNet. Il classificatore finale è stato sostituito con un layer identità (`nn.Identity`), ottenendo un vettore di feature semantiche di dimensione 1280 per ogni campione. È stata utilizzata la variante quantizzata `MobileNet_V2_QuantizedWeights.IMAGENET1K_QNNPACK_V1`, che opera in bassa precisione (`int8`) tramite backend QNN-PACK. Le feature risultanti sono quindi leggere da memorizzare e veloci da processare, con una perdita di accuratezza trascurabile.

3.2.2 Partizionamento del dataset FEMNIST

Il dataset FEMNIST, composto da caratteri manoscritti, è stato suddiviso secondo lo *writer_id*, sfruttando il partizionatore `NaturalIdPartitioner` della libreria `flwr-datasets`. In questa tesi sono stati considerati client, ciascuno associato a un sottoinsieme disgiunto di scrittori, così da ottenere una distribuzione fortemente *non-IID*. Il problema di classificazione è stato limitato a 10 classi, selezionate tra quelle disponibili, per rendere il task computazionalmente gestibile (Figura 3.1).

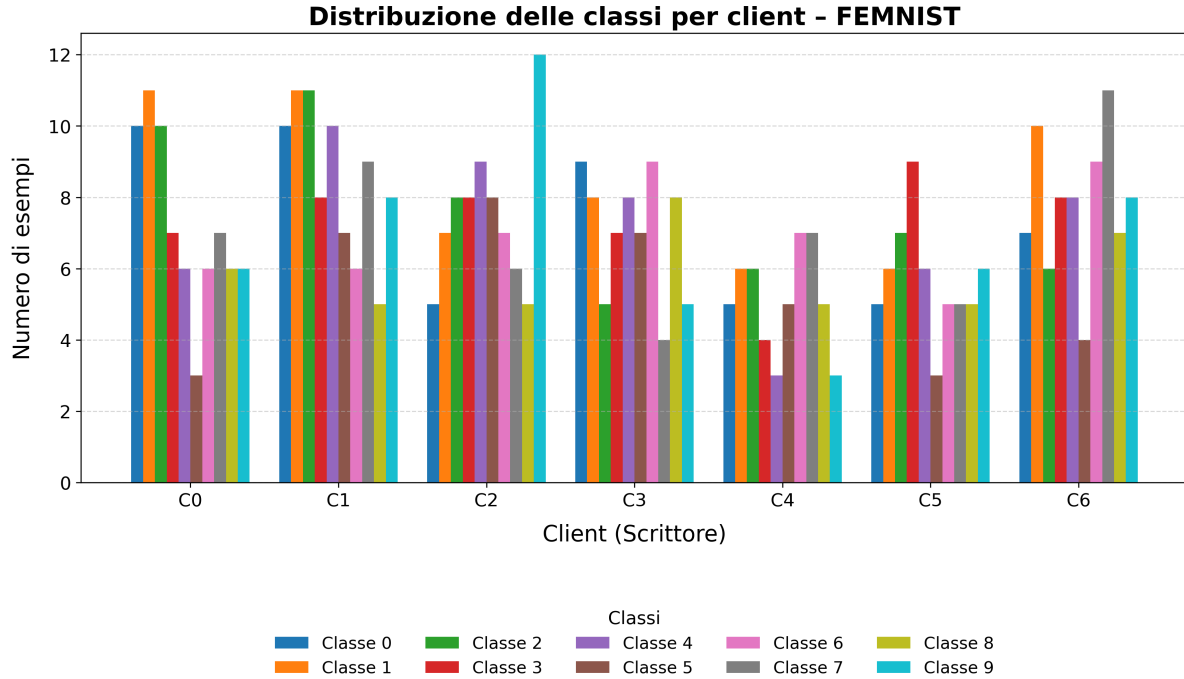


Fig. 3.1: Distribuzione del numero di esempi per classe nei 7 client FEMNIST (10 classi).

Ogni partizione è stata divisa localmente in training (70%) e test (30%), mantenendo il seed costante per garantire riproducibilità. Le immagini, originariamente in scala di grigi, sono state convertite in RGB, ridimensionate a 224×224 , normalizzate con le statistiche di ImageNet e trasformate in feature con MobileNetV2. Per ciascun client sono stati salvati quattro file: `train_features.pt`, `train_labels.pt`, `test_features.pt`, `test_labels.pt`, organizzati in cartelle separate.

3.2.3 Partizionamento del dataset IRDS

Il dataset IRDS contiene immagini di pazienti durante esercizi riabilitativi. Il dataset è stato ristretto a 4 classi di movimento, selezionate per semplificare il problema di classificazione e ridurre la complessità computazionale. Ogni sottocartella (101, 102, ...) corrisponde a un paziente e quindi a un client indipendente, introducendo naturalmente una distribuzione fortemente non bilanciata. In questa tesi sono stati selezionati 7 client, corrispondenti ai pazienti 101–107, così da ottenere un sistema federato eterogeneo ma gestibile dal punto di vista computazionale (Figura 3.2).

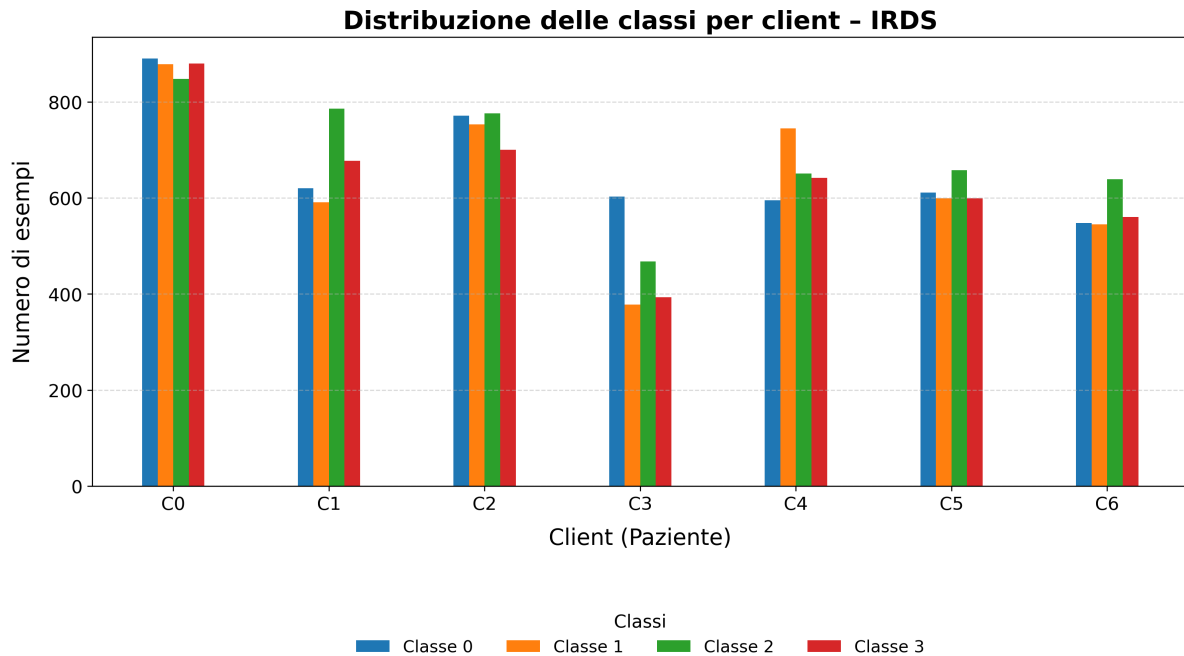


Fig. 3.2: Distribuzione del numero di esempi per classe nei client IRDS, con eterogeneità evidente.

Per ciascun client i dati sono stati suddivisi a priori in training (70%), validation (15%) e test (15%). Durante l'addestramento federato, test e validation vengono concatenati. Le feature sono state estratte con MobileNetV2 quantizzata e salvate come file `.pt`, accompagnate da mappe JSON che associano ciascun file all'etichetta corrispondente. La struttura delle directory prevede quindi tre sottocartelle (`train`, `validation`, `test`) e i rispettivi file di mapping.

3.2.4 Funzione di caricamento unificata

Per entrambi i dataset è stata implementata una funzione `load_data` che restituisce due `DataLoader`: uno per l'addestramento e uno per il test.

- Nel caso FEMNIST, i tensori di feature ed etichette sono caricati direttamente dai file binari e aggregati in un `TensorDataset`.
- Nel caso IRDS, la funzione costruisce dinamicamente i dataset leggendo i file `.pt` dalle directory e mappando le etichette tramite i file `JSON`. Test e validation vengono uniti in un unico `DataLoader`.

Questa interfaccia unificata permette di astrarre completamente il formato dei dati, garantendo compatibilità con tutte le strategie di addestramento federato implementate.

3.3 Paradigma 1: Federated Learning classico

3.3.1 Architettura generale del sistema federato

L'implementazione del paradigma di *Federated Learning* si basa su un'architettura distribuita sincrona client-server, realizzata tramite il framework **Flower (FLWR)** [1]. Il sistema è composto da due componenti principali:

- **Server centrale**, che coordina i round federati, seleziona i client e aggrega i modelli locali;
- **Client periferici**, ciascuno dei quali addestra localmente il modello CI-VAE sui dati privati e restituisce gli aggiornamenti al server.

Il ciclo di addestramento segue lo schema descritto in fase di progettazione (cfr. Sez. 2.2.2): il server distribuisce i pesi iniziali, i client eseguono training locale, e i risultati vengono aggregati fino al raggiungimento della convergenza.

L'intero processo si articola in round sincroni:

1. Il **server** centrale inizializza il modello globale e seleziona un sottoinsieme di client disponibili.
2. Ogni **client** riceve i pesi del modello, li ottimizza localmente sui propri dati privati, e restituisce al server i parametri aggiornati.
3. Il **server** esegue l'aggregazione secondo una strategia scelta (FedAvg, FedProx, FedAdam, FedAdagrad).
4. Il ciclo si ripete fino al numero massimo di round o fino al soddisfacimento di un criterio di arresto.

3.3.2 Server e Strategie di aggregazione

La funzione `server_fn()` costituisce il punto di ingresso del nodo centrale ed è responsabile della configurazione del server federato. Essa riceve come argomento il contesto esecutivo di Flower (`Context`), dal quale vengono letti i parametri di configurazione. All'interno della funzione viene inizializzato il modello globale `ImprovedCI_VAE_NUMERICAL` con le dimensioni e gli iperparametri definiti; successivamente i pesi vengono serializzati in un formato compatibile con FLWR. A questo punto viene costruita la strategia di aggregazione tramite la funzione `create_strategy()`, alla quale possono essere passati parametri specifici relativi alla strategia selezionata (ad esempio FedProx, FedAdam o FedVAE). Infine, viene definita la configurazione del server (come il numero di round da eseguire e la frazione di client da coinvolgere) e il tutto è incapsulato in un oggetto `ServerAppComponents`, che viene restituito a Flower per l'orchestrazione dei round federati. Un estratto semplificato del codice è riportato in Listing 3.1:

```
1 def server_fn(context: Context):  
2     # seed per riproducibilit   
3     fix_random(42)  
4
```

```

5  # Lettura configurazione da context
6  num_rounds    = context.run_config["num-server-rounds"]
7  local_epochs  = context.run_config["local-epochs"]
8  ...
9  strategy_name = context.run_config["strategy"]
10
11 # Inizializzazione modello globale
12 net = ImprovedCI_VAE_NUMERICAL(...)
13 parameters = ndarrays_to_parameters(get_weights(net))
14
15 ...
16
17 # Creazione strategia
18 strategy = create_strategy(strategy_name,
19                             fraction_fit,
20                             parameters=parameters, ...)
21
22 # Configurazione server
23 config = ServerConfig(num_rounds=num_rounds)
24
25 ...
26 # Composizione dei componenti
27 return ServerAppComponents(strategy=strategy, config=
    config)

```

Listing 3.1: Sketch rappresentativo della funzione `server_fn()`

Lancio del server. Il server federato viene avviato istanziando un oggetto `ServerApp` e passando come factory proprio la `server_fn()`. Il lancio avviene dunque con poche righe di codice:

```

1 app = ServerApp(server_fn=server_fn)
2 app.run()

```

Listing 3.2: Avvio del server federato

Strategie di aggregazione Come discusso in fase di progettazione (cfr. Sez. 2.2.4), la fase di aggregazione è il cuore del *Federated Learning*. L'implementazione sfrutta le classi di strategia incluse in FLWR, incapsulate in una strategia base `BaseStrategyWithEarlyStopping` per gestire la logica di early stopping. Questa classe introduce i parametri `patience` e `min_delta`: se l'accuracy aggregata non migliora per un numero di round consecutivi superiore a `patience`, l'addestramento federato viene interrotto automaticamente.

Le strategie sono istanziate mediante funzione factory a partire dal nome specificato in input:

```

1 def create_strategy(strategy_name: str, ...):
2     if strategy_name.lower() == "fedavg":

```

```

3         return FedAvg(...)
4     elif strategy_name.lower() == "fedprox":
5         return FedProx(...)
6     elif strategy_name.lower() == "fedadam":
7         return FedAdam(...)
8     elif strategy_name.lower() == "fedadagrad":
9         return FedAdagrad(...)

```

Listing 3.3: Factory delle strategie di aggregazione

FedAvg. La classe `flwr.server.strategy.FedAvg` è la strategia standard di FLWR, utilizzata come baseline. Essa implementa la media pesata dei parametri locali in base alla cardinalità dei dataset.

FedProx. La classe `flwr.server.strategy.FedProx` estende `FedAvg` introducendo una penalizzazione prossimale lato client. In implementazione, viene istanziata specificando il parametro `proximal_mu` per controllare l'intensità della regolarizzazione.

FedAdam. La classe `flwr.server.strategy.FedAdam`, appartenente alla famiglia `FedOpt`, adotta un aggiornamento adattivo lato server. L'istanza richiede la configurazione degli iperparametri `eta`, `beta_1`, `beta_2`, che vengono passati dal file di configurazione.

FedAdagrad. Analogamente, la classe `flwr.server.strategy.FedAdagrad` è una variante adattiva che accumula lo storico degli aggiornamenti. Anch'essa è supportata nativamente da FLWR e istanziata passando i parametri `eta` ed ϵ .

3.3.3 Costruzione e comportamento del client federato

Ogni nodo federato rappresenta un'entità indipendente che riceve, addestra e valuta localmente una copia del modello neurale `ImprovedCI.VAE.NUMERICAL`. Il client estende l'interfaccia `flwr.client.NumPyClient`, come previsto dal framework Flower, e implementa i tre metodi fondamentali richiesti: `get_parameters()`, `fit()`, e `evaluate()`.

Costruttore del client federato. La classe `FlowerClient` definisce nel suo costruttore tutti gli attributi necessari per l'esecuzione del training locale. In particolare, riceve in input il modello CI-VAE, i dataloader di training e validazione, oltre a una serie di iperparametri tra cui: learning rate, gamma dello scheduler, step di aggiornamento, numero di epoche, strategia di ottimizzazione, funzione di loss, batch size, e dimensione dello spazio latente. Tutti questi vengono assegnati come attributi dell'istanza e il modello è spostato sul dispositivo disponibile (GPU o CPU), come mostrato in Listing 3.4.

```

1 class FlowerClient(NumPyClient):
2     def __init__(self, net, trainloader, valloader, lr, gamma,
3                 step_size, local_epochs, strategy_name,
4                 loss_function,

```

```

4         batch_size, latent_dim, optimizer_name):
5
6         self.net = net
7         self.trainloader = trainloader
8         self.valloader = valloader
9         self.lr = lr
10        self.gamma = gamma
11        self.step_size = step_size
12        self.local_epochs = local_epochs
13        self.strategy_name = strategy_name
14        self.loss_function = loss_function
15        self.batch_size = batch_size
16        self.latent_dim = latent_dim
17        self.optimizer_name = optimizer_name
18        self.device = torch.device("cuda" if torch.cuda.
19            is_available() else "cpu")
        self.net.to(self.device)

```

Listing 3.4: Costruttore del client federato

Registrazione e avvio del client. L’inizializzazione del client avviene tramite la funzione `client_fn(context)`, che legge i parametri dal contesto esecutivo (file di configurazione FLWR), costruisce dinamicamente il modello CI-VAE e i dataloader associati al nodo, e infine restituisce un oggetto compatibile con Flower. Questo è ottenuto tramite la chiamata a `.to_client()`, che trasforma l’istanza in un oggetto serializzabile da FLWR.

```

1 def client_fn(context: Context):
2     client_config = context.client_config
3     client_id = context.node_id
4
5     ...
6
7     net = ImprovedCI_VAE_NUMERICAL(...)
8     trainloader, valloader = get_dataloader_fl(client_id,
9                                                client_config)
10
11     ...
12
13     client = FlowerClient(net=net,
14                           trainloader=trainloader,
15                           valloader=valloader,
16                           lr=client_config["lr"],
17                           gamma=client_config["gamma"],
18                           step_size=client_config["step_size"],
19                           local_epochs=client_config["local_epochs"],
20                           strategy_name=client_config["strategy_name"],

```

```

21         loss_function=client_config["loss_function"],
22         batch_size=client_config["batch_size"],
23         latent_dim=client_config["latent_dim"],
24         optimizer_name=client_config["optimizer_name"])
25
26     return client.to_client()

```

Listing 3.5: Registrazione del client nel contesto FLWR

Funzione `fit()`: training locale. Il metodo `fit()` viene chiamato ad ogni round federato. Esso imposta i pesi globali ricevuti dal server tramite la funzione `set_weights`, esegue il training locale per un numero di epoche pari a `self.local_epochs` tramite la funzione `improved_train()` (3.15), e infine restituisce i nuovi parametri del modello insieme al numero di esempi usati e alla loss media locale.

```

1 def fit(self, parameters, config):
2     set_weights(self.net, parameters)
3     train_loss = improved_train(
4         self.net, self.trainloader, self.lr,
5         self.gamma, self.step_size, self.local_epochs,
6         self.loss_function, self.strategy_name,
7         self.latent_dim, self.optimizer_name
8     )
9
10    return get_weights(self.net), len(self.trainloader.dataset
11        ), {
12        "train_loss": train_loss
13    }

```

Listing 3.6: Training federato lato client

Funzione `evaluate()`: valutazione e logging. Il metodo `evaluate()` è responsabile della validazione del modello locale, tramite la funzione `test()` (3.3.7).

```

1 def evaluate(self, parameters, config):
2     set_weights(self.net, parameters)
3     metrics = test(self.net, self.valloader, self.device,
4                   self.loss_function, self.latent_dim)
5
6     return metrics["loss"], len(self.valloader.dataset),
7         metrics

```

Listing 3.7: Valutazione e logging delle metriche

Interfaccia `NumPyClient` e orchestrazione. L'intera classe `FlowerClient` implementa l'interfaccia `NumPyClient` definita da FLWR. Durante ogni round, il server

centrale invoca `fit()` e `evaluate()` sui client selezionati, i quali rispondono esclusivamente con i parametri aggiornati e le metriche aggregate, in linea con i principi di *privacy preservation*.

3.3.4 Costruzione dei modelli locali

Il modello locale utilizzato in ciascun client è un *Class-Informed Variational Autoencoder* (CI-VAE), denominata `ImprovedCI_VAE_NUMERICAL`. L'architettura è progettata per apprendere rappresentazioni latenti generative ma semanticamente organizzate, attraverso la combinazione di un autoencoder variational e un classificatore applicato allo spazio latente.

Inizializzazione e iperparametri. Il costruttore del modello accetta diversi iperparametri, forniti dinamicamente a ogni client tramite il contesto esecutivo (`Context.run_config`). Questi includono: la dimensione dell'input (`input_size`), la dimensione dei layer nascosti (`hidden_size`), la dimensionalità dello spazio latente (`latent_dim`) e il numero di classi (`num_classes`). Tutti i layer successivi si adattano proporzionalmente a questi valori, consentendo un controllo fine della complessità del modello.

Encoder profondo (3 livelli). L'encoder è composto da una sequenza di tre layer lineari completamente connessi, ciascuno seguito da `BatchNorm1d`, attivazione `ReLU` e dropout. La dimensionalità si riduce progressivamente a ogni passaggio, fino a raggiungere un quarto di quella iniziale. Questo consente di estrarre una rappresentazione compatta ma informativa dell'input:

```
1 self.encoder_layers = nn.Sequential(  
2     nn.Linear(input_size, hidden_size),  
3     nn.BatchNorm1d(hidden_size),  
4     nn.ReLU(inplace=True),  
5     nn.Dropout(0.3),  
6  
7     nn.Linear(hidden_size, hidden_size // 2),  
8     nn.BatchNorm1d(hidden_size // 2),  
9     nn.ReLU(inplace=True),  
10    nn.Dropout(0.3),  
11  
12    nn.Linear(hidden_size // 2, hidden_size // 4),  
13    nn.BatchNorm1d(hidden_size // 4),  
14    nn.ReLU(inplace=True),  
15    nn.Dropout(0.2)  
16 )
```

Listing 3.8: Encoder multilivello

Al termine dell'encoder, due layer lineari separati proiettano l'output nei vettori dei parametri della distribuzione latente:

```
1 self.mu_layer = nn.Linear(hidden_size // 4, latent_dim)  
2 self.logvar_layer = nn.Linear(hidden_size // 4, latent_dim)
```

Decoder simmetrico (4 livelli). Il decoder segue una struttura speculare rispetto all'encoder. A partire dal vettore latente z , ricostruisce progressivamente l'input originario attraverso quattro layer lineari con normalizzazione, ReLU e dropout:

```

1 self.decoder_layers = nn.Sequential(
2     nn.Linear(latent_dim, hidden_size // 4),
3     nn.BatchNorm1d(hidden_size // 4),
4     nn.ReLU(inplace=True),
5     nn.Dropout(0.2),
6
7     nn.Linear(hidden_size // 4, hidden_size // 2),
8     nn.BatchNorm1d(hidden_size // 2),
9     nn.ReLU(inplace=True),
10    nn.Dropout(0.3),
11
12    nn.Linear(hidden_size // 2, hidden_size),
13    nn.BatchNorm1d(hidden_size),
14    nn.ReLU(inplace=True),
15    nn.Dropout(0.3),
16
17    nn.Linear(hidden_size, input_size)
18 )

```

Listing 3.10: Decoder simmetrico

Classificatore latente (2 livelli). Parallelamente al decoder, un piccolo classificatore agisce direttamente sullo spazio latente z . Il modulo è composto da due layer lineari con una ReLU intermedia e dropout, e produce in output le probabilità per ciascuna classe:

```

1 self.classifier = nn.Sequential(
2     nn.Linear(latent_dim, latent_dim * 2),
3     nn.ReLU(inplace=True),
4     nn.Dropout(0.2),
5     nn.Linear(latent_dim * 2, num_classes)
6 )

```

Listing 3.11: Classificatore nello spazio latente

Forward pass. Il metodo `forward()` coordina l'intero flusso di calcolo: esegue la codifica dell'input, genera la variabile latente z tramite reparametrization trick, applica il classificatore latente e infine ricostruisce l'input originale con il decoder. Il modello restituisce la ricostruzione $\hat{x}$, i parametri μ e $\log \sigma^2$ della distribuzione latente, e le predizioni del classificatore:

```

1 def forward(self, x):
2     mu, logvar = self.encoder(x)
3     z = self.reparameterize(mu, logvar)
4     class_out = self.latent_classifier(z)
5     x_hat = self.decoder(z)
6     return x_hat, mu, logvar, class_out

```

Listing 3.12: Forward pass del modello

3.3.5 Training e ottimizzazione locale

Il training locale del modello `ImprovedCI_VAE_NUMERICAL` è gestito dalla funzione `improved_train()`. Ogni client addestra il modello sui propri dati privati per un numero di epoche prefissato, restituendo poi i pesi aggiornati al server centrale. Il processo adotta tecniche standardizzate di ottimizzazione e regolarizzazione per garantire stabilità e capacità di generalizzazione.

Ottimizzazione con AdamW. Come ottimizzatore viene utilizzato AdamW, una variante di Adam che introduce una regolarizzazione esplicita sui pesi (*weight decay*), riducendo il rischio di overfitting:

```

1 optimizer = torch.optim.AdamW(
2     net.parameters(),
3     lr=learning_rate,
4     weight_decay=1e-4
5 )

```

Listing 3.13: Ottimizzatore AdamW

Scheduler StepLR. Il learning rate viene gestito dinamicamente tramite StepLR, che lo riduce in maniera geometrica ogni step epoche, migliorando la stabilità della convergenza:

```

1 scheduler = torch.optim.lr_scheduler.StepLR(
2     optimizer, step_size=step, gamma=gamma
3 )

```

Listing 3.14: Scheduler StepLR

Pipeline di training. Il training locale viene gestito dalla funzione `improved_train()`, che integra le pratiche di ottimizzazione appena descritte. Dopo aver spostato il modello sul dispositivo disponibile ed impostato la modalità `train`, viene inizializzato l'ottimizzatore AdamW con penalizzazione L2 e lo scheduler StepLR per la regolazione dinamica del learning rate. Per ogni epoca, i batch di dati vengono elaborati con forward pass e calcolo della loss composita, seguiti da backpropagation, applicazione del gradient clipping e aggiornamento dei pesi. Al termine del ciclo, la funzione restituisce la loss media sulle epoche eseguite.

```

1 def improved_train(net, trainloader, epochs, device,
2                     learning_rate=0.01, gamma=0.95, step=10):
3     net.to(device)
4     net.train()
5     optimizer = torch.optim.AdamW(
6         net.parameters(), lr=learning_rate,
7         weight_decay=1e-4, betas=(0.9, 0.999)
8     )
9     scheduler = torch.optim.lr_scheduler.StepLR(
10        optimizer, step_size=step, gamma=gamma
11    )
12    max_grad_norm, total_loss = 1.0, 0.0
13
14    for epoch in range(epochs):
15        epoch_loss = 0.0
16        for features, labels in trainloader:
17            features, labels = features.to(device), labels.to(
18                device)
19            optimizer.zero_grad()
20            x_hat, mu, logvar, class_out = net(features)
21            loss, _, _, _ = fixed_loss_fn(
22                x_hat, features, logvar, mu, class_out, labels
23                , alpha=3.0
24            )
25            loss.backward()
26            torch.nn.utils.clip_grad_norm_(net.parameters(),
27                max_grad_norm)
28            optimizer.step()
29            epoch_loss += loss.item()
30        scheduler.step()
31        total_loss += epoch_loss / len(trainloader)
32    return total_loss / epochs

```

Listing 3.15: Funzione di training locale migliorata

3.3.6 Funzione di loss composita

L'addestramento del modello ImprovedCI-VAE-NUMERICAL utilizza una funzione di loss composita, tramite la funzione `fixed_loss_fn()`. Questa loss integra tre componenti fondamentali: ricostruzione, classificazione e regolarizzazione latente. Per garantire stabilità numerica ed evitare che una singola componente domini il processo di ottimizzazione, i tre termini sono opportunamente bilanciati tramite scaling esplicito.

```

1 def fixed_loss_fn(recon_x, x, logvar, mu, class_out, labels,
2                 alpha=1.0):
3     # ricostruzione

```

```

4  mse_recon = nn.MSELoss(reduction='mean')
5  recon_loss = mse_recon(recon_x, x)
6
7  # classificazione
8  class_criterion = nn.CrossEntropyLoss()
9  class_loss = class_criterion(class_out, labels.long())
10
11 # Regularizzazione
12 kld_loss = torch.mean(-0.5 * torch.sum(
13     1 + logvar - mu ** 2 - logvar.exp(), dim=1
14 ))
15
16 scaled_recon_loss = recon_loss * 10.0
17 total_loss = scaled_recon_loss + alpha * class_loss +
18     kld_loss
19
20 return total_loss, recon_loss, class_loss, kld_loss

```

Listing 3.16: Implementazione della funzione di loss composita

Ricostruzione. La prima componente misura la capacità del modello di ricostruire l'input originale a partire dal vettore latente, utilizzando l'errore quadratico medio (MSELoss). Poiché la scala di questo termine è tipicamente inferiore rispetto agli altri, viene moltiplicato per un fattore.

Classificazione. La seconda componente agisce sullo spazio latente z , imponendo che esso conservi informazioni discriminative rispetto alle classi. È calcolata come cross-entropy tra l'output del classificatore latente e le etichette reali. Un iperparametro α consente di regolarne l'influenza relativa.

Regularizzazione latente (KL). Infine, la divergenza KL forza la distribuzione appresa ad avvicinarsi a una prior gaussiana standard.

Utilizzo nel training. La funzione `fixed_loss_fn()` è invocata all'interno della routine di addestramento locale (`improved_train()`, cfr. Listing 3.15), che logga anche le singole componenti per consentire analisi e monitoraggio durante gli esperimenti.

3.3.7 Validazione e metriche federate

Nel contesto del *Federated Learning*, la valutazione del modello non può avvenire in modo centralizzato, poiché i dati rimangono distribuiti sui client. Pertanto, la validazione del modello globale è eseguita localmente da ciascun nodo, e le metriche vengono successivamente aggregate dal server per fornire una stima globale delle prestazioni del modello federato.

Validazione locale e calcolo delle metriche. La validazione lato client è realizzata dalla funzione `test()`, definita in `task.py`. Essa valuta il modello su un dataset di validazione locale, calcolando la **loss media**, la **accuracy**, e metriche più informative come **precision**, **recall** e **F1-score**. Il calcolo avviene come segue:

```
1 def test(net, testloader, device):
2     net.to(device)
3     criterion = torch.nn.CrossEntropyLoss()
4     correct, total_loss = 0, 0.0
5     y_pred = torch.tensor([], requires_grad=True).to(device)
6     y_true = torch.tensor([], requires_grad=True).to(device)
7     net.eval()
8
9     with torch.no_grad():
10         for batch in testloader:
11             features, labels = batch
12             features, labels = features.to(device), labels.to(
13                 device)
14             x_hat, mu, logvar, class_out = net(features)
15             loss, _, _, _ = loss_fn(x_hat, features, logvar,
16                 mu, class_out, labels)
17             total_loss += loss.item()
18             _, predicted = torch.max(class_out.data, 1)
19             correct += (predicted == labels).sum().item()
20             y_pred = torch.cat((y_pred, predicted), 0)
21             y_true = torch.cat((y_true, labels), 0)
22
23         # Compute accuracy and average loss
24         accuracy = correct / len(testloader.dataset)
25         avg_loss = total_loss / len(testloader)
26
27         # Compute F1-score, precision, and recall
28         y_pred = y_pred.cpu().detach().numpy()
29         y_true = y_true.cpu().detach().numpy()
30         f1score = f1_score(y_true, y_pred)
31         precision = precision_score(y_true, y_pred)
32         recall = recall_score(y_true, y_pred)
33
34     return avg_loss, accuracy, f1score, precision, recall
```

Listing 3.17: Funzione di validazione lato client

Questa funzione restituisce un insieme completo di metriche che vengono poi inserite nella risposta del client tramite il metodo `evaluate()`, all'interno della classe `FlowerClient`.

Aggregazione sul server. Il server centrale riceve i risultati di validazione da ciascun client e li aggrega all'interno del metodo `aggregate_evaluate()` delle strategie federate.

3.3.8 Early stopping nel server federato

Nel contesto di addestramento federato, l'early stopping consente di interrompere il processo globale di ottimizzazione quando la metrica di validazione aggregata non mostra più miglioramenti significativi. A differenza del training locale (cfr. Sez. 3.3.5), nel sistema proposto l'early stopping viene gestito dal **server centrale**, attraverso l'estensione personalizzata delle strategie di aggregazione del framework Flower.

Integrazione nelle strategie. Tutte le strategie supportate dal sistema (FedAvg, FedProx, FedAdam, ecc.) sono derivate da quelle native di `flwr.server.strategy`, ma arricchite tramite mixin personalizzati che implementano la logica di early stopping. In particolare, viene sovrascritto il metodo `aggregate_evaluate()`, che riceve le metriche aggregate dai client alla fine di ogni round. L'early stopping si basa sull'accuratezza media dei client validanti:

```
1 def aggregate_evaluate(self, server_round, results, failures):
2     aggregated_loss, aggregated_metrics = super().
3         aggregate_evaluate(server_round, results, failures)
4
5     # Accuratezza del round
6     accuracy = float(aggregated_metrics.get("accuracy", 0.0))
7
8     # early stoppying
9     self.check_early_stopping(server_round, accuracy)
10    return aggregated_loss, aggregated_metrics
```

Listing 3.18: Controllo early stopping nel server

Logica di arresto anticipato. La funzione `check_early_stopping()` implementa una variante del classico criterio con pazienza, confrontando l'accuratezza corrente con il miglior valore finora osservato. Se la metrica non migliora per un certo numero di round consecutivi, l'addestramento viene interrotto:

```
1 def check_early_stopping(self, server_round: int, metric:
2     float):
3     if self.best_metric is None or metric > self.best_metric +
4         self.min_delta:
5         self.best_metric = metric
6         self.patience_counter = 0
7     else:
8         self.patience_counter += 1
9
10    # addestramento fermato
11    if self.patience_counter >= self.patience:
12        print(f"Early stopping triggered at round {
13            server_round}")
14        self.stop_training = True
```

Listing 3.19: Implementazione di `check_early_stopping`

3.3.9 Logging avanzato dei risultati sperimentali

Durante l'addestramento federato, è fondamentale registrare in modo accurato sia le metriche di performance globali sia i parametri sperimentali associati a ciascuna esecuzione. Il server registra in un file `.csv` sia i parametri sperimentali (configurazione dei client, iperparametri di training e strategia adottata) sia le metriche globali aggregate, tra cui `accuracy`, `F1-score`, `precision`, `recall` e `loss`. Questo consente di tracciare ogni esperimento e confrontare facilmente le diverse configurazioni.

3.4 Paradigma 2: distillazione centralizzata

In questa sezione si descrive l'implementazione del paradigma di *centralized knowledge distillation* 2.3.3, utilizzato come baseline per confrontare i risultati ottenuti con l'apprendimento federato. L'obiettivo è sfruttare un *teacher model* addestrato su un dataset centralizzato, costituito dall'unione delle porzioni di validazione dei singoli client, per guidare l'addestramento di modelli studente più leggeri e distribuiti.

3.4.1 Struttura generale dell'esperimento

La funzione `run_single_experiment` rappresenta il punto di ingresso operativo del paradigma di distillazione centralizzata. Essa definisce un intero ciclo sperimentale che comprende l'addestramento di un modello teacher su un dataset centralizzato, seguito dalla distillazione della conoscenza appresa verso una serie di modelli student, addestrati localmente su partizioni eterogenee dei dati.

In modo analogo agli altri esperimenti, l'input è costituito da rappresentazioni numeriche a 1280 dimensioni, ottenute tramite estrazione delle feature da un backbone MobileNet. Il flusso della funzione può essere suddiviso in quattro fasi principali:

1. Preparazione dati e inizializzazione Il primo blocco della funzione si occupa del fissaggio del seed, della configurazione del dispositivo e del caricamento dei dati:

```
1 fix_all_seeds(seed)
2 if device is None:
3     device = torch.device("cuda" if torch.cuda.is_available() else "
4         cpu")
5 # caricamento dei dati del teacher
6 teacher_train_loader, teacher_val_loader =
7     load_teacher_data_with_split(
8         user_path, params['batch_size']
9     )
```

Listing 3.20: Inizializzazione e caricamento dei dati

La funzione `load_teacher_data_with_split` carica i dati necessari per l'addestramento del teacher, poi esegue uno split stratificato tra training e validation.

2. Addestramento del modello teacher Il modello teacher è una rete neurale profonda (tre hidden layer) progettata per catturare relazioni generali tra le classi. Viene addestrato sul dataset centralizzato e validato ad ogni epoca per selezionare il miglior checkpoint:

```
1 teacher_model = TeacherNetwork(
2     input_size=1280,
3     hidden_sizes=[512, 256, 128],
4     num_classes=4,
5     dropout_rate=params['teacher_dropout'],
6     device=device
7 )
```

```

8
9 teacher_model, teacher_results =
    train_teacher_with_best_model_selection(
10     teacher_model,
11     teacher_train_loader,
12     teacher_val_loader,
13     device,
14     epochs=params['teacher_epochs'],
15     lr=params['teacher_lr'],
16     gamma=params['teacher_gamma'],
17     step=params['teacher_step']
18 )

```

Listing 3.21: Addestramento del teacher

La funzione di training utilizza ottimizzazione con AdamW, scheduler di learning rate e early stopping basato sull'accuratezza di validazione.

3. Ciclo di distillazione verso gli studenti Una volta completato l'addestramento del teacher, la funzione avvia un ciclo su ciascun client per addestrare un modello studente. Ogni studente apprende tramite knowledge distillation, combinando le predizioni soft del teacher con le etichette reali locali:

```

1 for client_id in range(num_clients):
2     train_loader = all_client_data[client_id]['train_loader']
3     val_loader = all_client_data[client_id]['val_loader']
4
5     student_model = StudentNetwork(
6         input_size=1280,
7         hidden_sizes=[256, 128],
8         num_classes=4,
9         dropout_rate=params['student_dropout'],
10        device=device
11    )
12
13    student_model, client_results =
        train_student_with_best_model_selection(
14        student_model, teacher_model, train_loader, val_loader,
15        device,
16        epochs=params['student_epochs'],
17        lr=params['student_lr'],
18        temperature=params['temperature'],
19        alpha=params['alpha'],
20        gamma=params['student_gamma'],
21        step=params['student_step'],
22        client_id=client_id
23    )

```

Listing 3.22: Ciclo di addestramento degli studenti

La funzione di training dello studente implementa la loss di distillazione come combinazione di cross-entropy (etichetta reale) e KL-divergence (output del teacher), secondo il bilanciamento specificato dal parametro `alpha`.

4. Valutazione incrociata e metriche aggregate Dopo l'addestramento, ogni modello studente viene testato su tutti gli altri dataset client per misurare la capacità di generalizzazione cross-client. I risultati vengono raccolti per costruire matrici e medie:

```
1 cross_client_results = evaluate_student_on_other_clients(  
2     student_model, client_id, all_client_data, device, num_clients  
3 )  
4 client_results['cross_client_evaluation'] = cross_client_results  
5 all_client_results.append(client_results)
```

Listing 3.23: Valutazione cross-client e metriche

3.4.2 Architetture dei modelli

Nel paradigma di distillazione centralizzata vengono impiegate due classi distinte di modelli neurali: un *teacher model* complesso, in grado di apprendere rappresentazioni generali a partire da un dataset aggregato, e uno o più *student model* più leggeri, destinati a essere addestrati localmente con l'ausilio della conoscenza trasferita dal teacher.

Modello Teacher Il modello teacher è implementato come una rete neurale profonda composta da tre layer completamente connessi, intervallati da funzioni di attivazione ReLU, batch normalization e dropout. È progettato per essere sufficientemente espressivo da apprendere dai dati aggregati e rappresentare una conoscenza globale del task.

```
1 class TeacherNetwork(nn.Module):  
2     """Standard deep neural network for teacher model"""  
3  
4     def __init__(self, input_size=1280, hidden_sizes=[512, 256,  
5         128], num_classes=4,  
6         dropout_rate=0.3, device=torch.device("cuda" if  
7             torch.cuda.is_available() else "cpu")):  
8         super(TeacherNetwork, self).__init__()  
9  
10        self.device = device  
11        self.num_classes = num_classes  
12        self.input_size = input_size  
13  
14        # Build layers dynamically  
15        layers = []  
16        prev_size = input_size  
17  
18        for i, hidden_size in enumerate(hidden_sizes):  
19            layers.append(nn.Linear(prev_size, hidden_size))  
20            layers.append(nn.ReLU(inplace=True))  
21            layers.append(nn.BatchNorm1d(hidden_size))
```

```

20         layers.append(nn.Dropout(dropout_rate))
21         prev_size = hidden_size
22
23         # Final classification layer
24         layers.append(nn.Linear(prev_size, num_classes))
25
26         self.network = nn.Sequential(*layers)
27
28     def forward(self, x):
29         return self.network(x)

```

Listing 3.24: Architettura del modello teacher

La combinazione di batch normalization e dropout permette al modello di generalizzare meglio su domini multipli, riducendo il rischio di overfitting sul dataset centralizzato.

Modello Studente Il modello studente condivide la stessa struttura generale del teacher, ma con una profondità e una capacità ridotte.

```

1 class StudentNetwork(nn.Module):
2     def __init__(self, input_size=1280, hidden_sizes=[256, 128],
3                 num_classes=4,
4                 dropout_rate=0.2, device=torch.device("cuda" if
5                 torch.cuda.is_available() else "cpu")):
6         super(StudentNetwork, self).__init__()
7
8         self.device = device
9         self.num_classes = num_classes
10        self.input_size = input_size
11
12        # Build layers dynamically
13        layers = []
14        prev_size = input_size
15
16        for i, hidden_size in enumerate(hidden_sizes):
17            layers.append(nn.Linear(prev_size, hidden_size))
18            layers.append(nn.ReLU(inplace=True))
19            layers.append(nn.BatchNorm1d(hidden_size))
20            layers.append(nn.Dropout(dropout_rate))
21            prev_size = hidden_size
22
23        # Final classification layer
24        layers.append(nn.Linear(prev_size, num_classes))
25
26        self.network = nn.Sequential(*layers)
27
28    def forward(self, x):
29        return self.network(x)

```

Listing 3.25: Architettura del modello studente

L'obiettivo della distillazione è quello di trasferire le conoscenze del teacher a questo modello più semplice, preservando il più possibile le capacità discriminative nonostante la minore complessità.

Adattamento per il modello CI-VAE La procedura di distillazione è stata estesa anche a modelli neurali più complessi rispetto alle semplici reti fully connected, in particolare al modello `CI_VAE_NUMERICAL` descritto nella Sezione 3.3.4.

Per poter applicare la distillazione anche in questo contesto, è stato introdotto un modello studente denominato `StudentCI_VAE_NUMERICAL`, progettato come una versione strutturalmente compatibile ma meno profonda e meno capiente del CI-VAE originale. La struttura complessiva rimane quella tipica di un VAE condizionato — composta da un encoder, un decoder e un classificatore — ma ciascun componente è stato semplificato per adattarsi a uno scenario client con risorse limitate.

L'encoder è stato ridotto a due layer completamente connessi, mantenendo comunque la normale pipeline di batch normalization, attivazione ReLU e dropout. Anche il decoder segue una struttura simmetrica più leggera, con meno neuroni e meno profondità.

Un estratto dell'implementazione del modello studente è mostrato di seguito:

```
1 class StudentCI_VAE_NUMERICAL(nn.Module):
2     def __init__(self, input_size=1280, hidden_size=256, latent_dim
3         =7, num_classes=4, ...):
4         ...
5         # ENCODER piu' leggero
6         self.encoder_layers = nn.Sequential(
7             nn.Linear(self.input_size, self.hidden_size),
8             nn.BatchNorm1d(self.hidden_size),
9             nn.ReLU(inplace=True),
10            nn.Dropout(0.2),
11
12            nn.Linear(self.hidden_size, self.hidden_size//2),
13            nn.BatchNorm1d(self.hidden_size//2),
14            nn.ReLU(inplace=True),
15            nn.Dropout(0.2),
16        )
17
18        # Variational layers
19        self.mu_layer = nn.Linear(self.hidden_size//2, self.
20            latent_dim)
21        self.logvar_layer = nn.Linear(self.hidden_size//2, self.
22            latent_dim)
23
24        # CLASSIFIER piu' semplice
25        self.classifier = nn.Sequential(
26            nn.Linear(self.latent_dim, self.latent_dim),
27            nn.ReLU(inplace=True),
28            nn.Dropout(0.1),
29            nn.Linear(self.latent_dim, self.num_classes)
30        )
```

```

29     # DECODER piu' leggero
30     self.decoder_layers = nn.Sequential(
31         nn.Linear(self.latent_dim, self.hidden_size//2),
32         nn.BatchNorm1d(self.hidden_size//2),
33         nn.ReLU(inplace=True),
34         nn.Dropout(0.2),
35
36         nn.Linear(self.hidden_size//2, self.hidden_size),
37         nn.BatchNorm1d(self.hidden_size),
38         nn.ReLU(inplace=True),
39         nn.Dropout(0.2),
40
41         nn.Linear(self.hidden_size, self.input_size),
42     )

```

Listing 3.26: Versione semplificata del CI-VAE per lo studente

3.4.3 Meccanismo di distillazione

Il processo di distillazione della conoscenza rappresenta il nucleo del paradigma implementato. Un modello *teacher*, addestrato su dati centralizzati, funge da supervisore per uno o più modelli *student*, che apprendono su dati locali sfruttando anche le predizioni del teacher come *soft targets*.

Distillazione per modelli fully connected. Per i modelli neurali standard, la funzione di loss combina la *cross-entropy* tra le etichette reali e le predizioni dello studente (*hard loss*) con una *KL-divergence* tra le predizioni del teacher e quelle dello studente (*soft loss*), scalate da una temperatura T :

```

1 def knowledge_distillation_loss(student_out, teacher_out, labels, T
  =4.0, alpha=0.7):
2     hard = F.cross_entropy(student_out, labels)
3     soft = F.kl_div(
4         F.log_softmax(student_out / T, dim=1),
5         F.softmax(teacher_out / T, dim=1),
6         reduction='batchmean'
7     ) * (T ** 2)
8     return alpha * soft + (1 - alpha) * hard

```

Distillazione per modelli CI-VAE. Nel caso del modello CI_VAE_NUMERICAL, la loss di distillazione include anche la componente generativa. La funzione finale combina:

- la loss completa del VAE sullo studente (*reconstruction + classification + KLD*);
- la distillazione sulle predizioni (KL-divergence);

```

1 def kd_loss(student_out, teacher_out, labels, features, T=4.0, alpha
  =0.7, beta=0.3):
2     # unpack
3     x_hat_s, mu_s, _, class_s = student_out

```

```

4  _, mu_t, _, class_t = teacher_out
5
6  _, _, _, vae_loss = fixed_loss_fn(...) # CI-VAE loss
7  distill = F.kl_div(
8      F.log_softmax(class_s / T, dim=1),
9      F.softmax(class_t / T, dim=1),
10     reduction='batchmean'
11 ) * (T ** 2)
12
13 return alpha * distill + beta * vae_loss

```

Durante l’addestramento, il modello *teacher* è mantenuto in modalità `eval()` e non viene aggiornato. Questo approccio ha permesso di trasferire efficacemente la conoscenza da un modello globale più capace a modelli locali compatti, migliorandone la generalizzazione anche in scenari con distribuzioni eterogenee.

3.4.4 Adattamento della funzione di caricamento dati

Per abilitare il paradigma di distillazione centralizzata è stato necessario modificare la funzione `load_data` 3.4.4, originariamente pensata per scenari federati. In particolare, si è introdotta una distinzione tra il caricamento dei dati destinati al modello *teacher* (centralizzato) e quello dei dati utilizzati dai modelli *student* locali.

Nel caso del dataset IRDS, ogni partizione client è suddivisa in tre insiemi secondo le proporzioni 70% training, 15% validation e 15% test. Per il modello *teacher*, si è implementata una funzione dedicata che carica e combina le porzioni di *validation set* provenienti da tutti i client. I dati così ottenuti vengono poi suddivisi mediante *train/-test split stratificato* in un sottoinsieme di training e uno di validazione, utilizzati per l’addestramento e il monitoraggio del *teacher*. I modelli *student*, invece, continuano a utilizzare i dati del proprio client, accedendo esclusivamente alle rispettive porzioni di training e test, senza condivisione tra domini.

Nel caso del dataset FEMNIST, la funzione di caricamento originaria eseguiva uno *split* statico 70/30 su ciascun file di training di ogni client, ottenendo così un sottoinsieme per l’addestramento e uno per la validazione locale. Per il paradigma centralizzato, si è modificato il caricamento in modo da aggregare le porzioni di test di tutti i client in un unico dataset globale. In questo caso, i dati rimanenti di ciascun client vengono suddivisi internamente tra training (80%) e validation (20%) per gli *student*.

È importante sottolineare che, nella scelta dei dati di addestramento del modello *teacher*, non è stato adottato un approccio di tipo *k-fold cross-validation*. Questa scelta è intenzionale e motivata dalla volontà di simulare uno scenario realistico, in cui si dispone di un singolo dataset centralizzato per la supervisione e in cui i dati etichettati sono tipicamente limitati. In particolare, si assume che il dataset validato a livello centrale — ottenuto combinando le porzioni di validation dei client — costituisca l’unica risorsa supervisionata disponibile per l’addestramento del *teacher*.

In scenari reali di apprendimento collaborativo, è comune che l’aggregazione centralizzata dei dati non avvenga in forma completa o ripetibile, e che si disponga di una singola istanza del dataset da sfruttare nel modo più efficiente possibile. Pertanto, si è preferito evitare la *cross-validation*, concentrandosi invece su uno *split stratificato* statico del da-

taset centralizzato, con il duplice obiettivo di riflettere condizioni realistiche e garantire riproducibilità sperimentale.

3.5 Paradigma 3: Federated Model Distillation

L'implementazione del paradigma FedMD segue la formulazione descritta descritto nella Sez. 2.4.1. Viene implementato attraverso un ciclo iterativo di round federati. Ciascun round prevede due fasi fondamentali:

1. **Addestramento supervisionato locale**, in cui ciascun client ottimizza il proprio modello sui dati privati, utilizzando la classica loss di cross-entropy rispetto alle etichette reali;
2. **Distillazione collaborativa su dataset pubblico**, in cui i client producono predizioni (logit) su un dataset pubblico condiviso. Tali logit vengono aggregati dal server centrale (tramite media pesata rispetto alla dimensione dei dataset locali), producendo una distribuzione consensuale per ciascun campione. I client aggiornano poi localmente i modelli, minimizzando una loss combinata che include sia la cross-entropy supervisionata sia la divergenza KL rispetto alle soft-label aggregate.

Tale struttura riflette il principio concettuale fondamentale del paradigma FedMD (cfr. Sez. 2.4.1): spostare la cooperazione federata dal livello dei parametri (come in FedAvg) a quello delle predizioni, riducendo drasticamente i rischi per la privacy e migliorando la compatibilità con modelli locali eterogenei.

3.5.1 Ciclo operativo del protocollo FedMD

Il ciclo operativo del paradigma *Federated Model Distillation* (FedMD) è implementato nella classe `FedMDSys`tem, in particolare nel metodo `run_federated_training()`. Il ciclo prevede un numero configurabile di round federati, durante i quali si alternano addestramento supervisionato locale e distillazione collaborativa basata su un dataset pubblico sintetico. Ogni fase è modulare e gestita da metodi dedicati, come mostrato di seguito.

```
1 def run_federated_training(self) -> Dict:
2     all_results = []
3     best_accuracy = -float('inf')
4     patience = self.config.get('early_stopping_patience', 5)
5     no_improve_counter = 0
6
7     for round_num in range(self.config['rounds']):
8         # 1. Addestramento locale
9         self.local_training_phase()
10        # 2. Consensus
11        consensus_preds = self.consensus_phase()
12        # 3. Distillazione
13        distillation_losses = self.distillation_phase(
14            consensus_preds)
15        # 4. Valutazione
16        metrics = self.evaluation_phase(round_num + 1,
17            distillation_losses)
18        all_results.append(metrics)
19        self.log_round_results(round_num + 1, metrics)
```

```

18     ...
19     return {"config": self.config, "results": all_results, ...}

```

Listing 3.27: Metodo principale del ciclo FedMD

Fase 1: Addestramento supervisionato locale

L'addestramento locale viene orchestrato da `local_training_phase()`, che per ciascun client carica i dati locali e richiama il metodo `_train_local_model`. Quest'ultimo ottimizza i parametri del modello tramite Adam e loss `CrossEntropyLoss`.

```

1 def _train_local_model(self, model, train_loader, epochs, lr):
2     optimizer = torch.optim.Adam(model.parameters(), lr=lr)
3     criterion = nn.CrossEntropyLoss()
4     for epoch in range(epochs):
5         for x, y in train_loader:
6             x, y = x.to(device), y.to(device)
7             optimizer.zero_grad()
8             outputs = model(x)
9             loss = criterion(outputs, y)
10            loss.backward()
11            optimizer.step()

```

Listing 3.28: Training supervisionato locale

Fase 2: Estrazione dei logit e consenso

Dopo il training locale, ogni client produce predizioni normalizzate sul dataset pubblico $\mathcal{D}_{\text{pub}}$. La funzione `_get_soft_predictions()` calcola le distribuzioni di probabilità applicando un softmax con temperatura T :

```

1 def _get_soft_predictions(self, model, dataloader, temperature):
2     model.eval()
3     all_preds = []
4     with torch.no_grad():
5         for x, _ in dataloader:
6             x = x.to(device)
7             logits = model(x)
8             soft_preds = F.softmax(logits / temperature, dim=1)
9             all_preds.append(soft_preds.cpu())
10    return torch.cat(all_preds, dim=0)

```

Listing 3.29: Predizioni normalizzate (soft labels)

Le predizioni di tutti i client vengono aggregate da `consensus_phase()` tramite media pesata rispetto alla dimensione dei dataset locali:

```

1 total_weight = sum(weights)
2 normalized_weights = [w / total_weight for w in weights]
3
4 consensus_preds = torch.zeros_like(all_predictions[0])
5 for preds, weight in zip(all_predictions, normalized_weights):

```

```
6 consensus_preds += weight * preds
```

Listing 3.30: Aggregazione per il consenso

Il risultato è un insieme di soft-label condivise, che rappresentano il consenso federato.

Fase 3: Distillazione verso il consenso

Nella distillazione, ciascun modello viene riallineato al consenso minimizzando una loss composita, implementata in `_distill_towards_consensus()`:

```
1 student_soft = F.log_softmax(outputs / temperature, dim=1)
2 soft_loss = F.kl_div(student_soft, target_preds, reduction='
    batchmean')
3
4 hard_loss = F.cross_entropy(outputs, y)
5 loss = (1 - alpha) * hard_loss + alpha * (temperature ** 2) *
    soft_loss
```

Listing 3.31: Loss combinata per la distillazione

Il parametro `alpha` controlla il bilanciamento tra loss supervisionata e loss di distillazione. Il fattore T^2 corregge la scala dei gradienti derivante dalla softmax a temperatura elevata.

Fase 4: Valutazione e logging

Alla fine di ogni round, ogni modello è valutato non solo sul proprio test set, ma anche su quelli degli altri client, ottenendo una matrice di accuratezza inter-client. Questa fase è implementata in `evaluation_phase()` e usa la funzione `_evaluate_model()`:

```
1 def _evaluate_model(self, model, dataloader) -> float:
2     correct, total = 0, 0
3     with torch.no_grad():
4         for x, y in dataloader:
5             x, y = x.to(device), y.to(device)
6             outputs = model(x)
7             _, predicted = torch.max(outputs, 1)
8             total += y.size(0)
9             correct += (predicted == y).sum().item()
10    return correct / total if total > 0 else 0.0
```

Listing 3.32: Valutazione delle predizioni

Infine, le metriche aggregate (accuratezza media, accuratezza cross-client, loss di distillazione) vengono salvate su file CSV da `log_round_results()`, insieme agli iperparametri usati nel round corrente.

3.5.2 Costruzione dei modelli locali

Architettura del classificatore (MLP)

In accordo con i principi del paradigma *Federated Model Distillation* (FedMD), ogni client federato dispone di un proprio modello locale di classificazione, addestrato in maniera

indipendente sui dati privati e ottimizzato attraverso una combinazione di apprendimento supervisionato e distillazione collaborativa.

Nel contesto sperimentale considerato, tutti i client adottano un classificatore MLP (Multilayer Perceptron), progettato per operare direttamente sulle rappresentazioni vettoriali generate dal feature extractor MobileNetV2.

L'architettura è composta da tre layer completamente connessi, con funzioni di attivazione ReLU e due livelli di Dropout per migliorare la capacità di generalizzazione:

```
1 class MLPClassifier(nn.Module):
2     def __init__(self, input_dim=1280, num_classes=10):
3         super().__init__()
4         self.net = nn.Sequential(
5             nn.Linear(input_dim, 512),
6             nn.ReLU(),
7             nn.Dropout(0.2),
8             nn.Linear(512, 256),
9             nn.ReLU(),
10            nn.Dropout(0.2),
11            nn.Linear(256, num_classes)
12        )
13
14    def forward(self, x):
15        return self.net(x)
```

Listing 3.33: Architettura del classificatore MLP

L'input della rete è costituito da vettori di dimensione 1280, corrispondenti all'output del feature extractor MobileNetV2, mentre l'output è un vettore di logit di dimensione pari al numero di classi del task di classificazione (ad esempio, 10 per FEMNIST e 4 per IRDS). La scelta di un MLP riflette un compromesso tra semplicità architetturale ed efficacia rappresentativa, in linea con l'obiettivo di FedMD: analizzare la qualità della distillazione in scenari federati piuttosto che massimizzare la performance assoluta dei modelli.

Eterogeneità modellistica e flessibilità architetturale

Sebbene negli esperimenti condotti sia stata adottata una configurazione omogenea — in cui tutti i client utilizzano la stessa architettura MLP — il framework è stato progettato per supportare scenari con eterogeneità architetturale.

La flessibilità è garantita dal metodo `_initialize_clients()` della classe `FedMDSys-tem`, che permette di istanziare modelli diversi per ciascun client. Un estratto del codice mostra come i modelli vengano creati ciclicamente in base al numero di client:

```
1 def _initialize_clients(self) -> List[Dict]:
2     model_types = [MLPClassifier] # estendibile con altre
3     # architetture
4     clients = []
5     for i in range(self.num_clients):
6         model_type = model_types[i % len(model_types)]
7         model = model_type(self.input_dim, self.num_classes).to(
8             device)
```

```

7     clients.append({
8         'id': i,
9         'model': model,
10        'model_type': model_type.__name__,
11        'dataset_size': 0
12    })
13    return clients

```

Listing 3.34: Creazione dei modelli locali

Ogni modello è quindi istanziato in modo indipendente, consentendo eventuali personalizzazioni architetturali (profondità, dimensioni degli hidden layer, tipologia di rete). La cooperazione federata non viene compromessa, poiché il paradigma FedMD si fonda esclusivamente sullo scambio di predizioni (logit) e non di parametri interni.

Questa proprietà rende FedMD particolarmente adatto a scenari realistici caratterizzati da dispositivi eterogenei — come edge device, sensori embedded o terminali mobili — che possono disporre di capacità computazionali e vincoli energetici differenti.

3.5.3 Generazione del dataset sintetico

La logica di generazione del dataset sintetico, già discussa dal punto di vista concettuale nella Sez. 2.4.1, è implementata nella classe `SyntheticDatasetGenerator`. Il costruttore della classe inizializza l'oggetto raccogliendo, tramite il metodo privato `_collect_real_data_statistics()`, le statistiche aggregate delle embedding provenienti dai client. Vengono calcolate la media e la deviazione standard globali, nonché le statistiche per ciascuna classe (media, deviazione standard, frequenza) e i valori minimi/massimi osservati.

Sulla base di queste statistiche, il metodo `generate_gaussian_mixture_data()` genera il dataset sintetico. Per ciascuna classe viene campionata una quantità controllata di vettori da distribuzioni gaussiane multivariate parametrizzate secondo i valori appresi. I campioni vengono poi normalizzati, mescolati e convertiti in un oggetto `TensorDataset`. L'output finale è un dataset normalizzato e strutturato in modo da rispettare le proporzioni delle classi, pur essendo completamente generato artificialmente.

Ogni campione è etichettato con un'identità sintetica coerente con la distribuzione da cui è stato generato, il che permette l'uso di una componente supervisionata nella fase di distillazione, bilanciata dalla conoscenza collettiva appresa tramite consenso (cfr. Sez. 3.5.4).

```

1 class SyntheticDatasetGenerator:
2     def __init__(self, input_dim: int, num_classes: int,
3                 num_clients: int):
4         self.input_dim = input_dim
5         self.num_classes = num_classes
6         self.num_clients = num_clients
7         self.real_data_stats = self.
8             _collect_real_data_statistics()
9
10    def _collect_real_data_statistics(self) -> Dict:
11        ...

```

```

10     # Calcolo di media, deviazione, min, max
11     return {
12         'global_mean': ...,
13         'global_std': ...,
14         'class_stats': {...},
15         'feature_ranges': {'min': ..., 'max': ...}
16     }
17
18     def generate_gaussian_mixture_data(self, num_samples: int)
19     -> TensorDataset:
20         """Genera dataset sintetico campionando da gaussiane
21         per classe"""
22         for class_id in range(self.num_classes):
23             ...
24             class_features = torch.normal(
25                 mean.unsqueeze(0).repeat(samples_per_class, 1)
26                 ,
27                 std.unsqueeze(0).repeat(samples_per_class, 1)
28             )
29             ...
30             labels = torch.full((samples_per_class,), class_id
31                                 )
32             ...
33             all_features = torch.cat(features_list, dim=0)
34             all_labels = torch.cat(labels_list, dim=0)
35
36         # Shuffle e normalizzazione
37         all_features = F.normalize(all_features, p=2, dim=1)
38         return TensorDataset(all_features, all_labels)

```

Listing 3.35: Struttura della classe SyntheticDatasetGenerator

3.5.4 Funzione di perdita nella distillazione federata

Nel paradigma *Federated Model Distillation* (FedMD), la fase di distillazione rappresenta il momento chiave della cooperazione tra i client. In questa fase ciascun modello locale viene aggiornato esclusivamente utilizzando un dataset pubblico sintetico, costruito centralmente sulla base di statistiche aggregate dai client..

L'implementazione di questa logica è contenuta nella funzione `_distill_towards_consensus()` della classe `FedMDSysytem`. Essa combina i due segnali tramite una funzione di perdita che somma, in proporzione pesata, la cross-entropy supervisionata e la divergenza di Kullback–Leibler tra la distribuzione predetta dal modello locale e quella consensuale.

La loss totale è quindi definita dalla formula 2.2:

$$\mathcal{L}(x) = (1 - \alpha) \cdot \mathcal{L}_{\text{CE}}(x) + \alpha \cdot T^2 \cdot \mathcal{L}_{\text{KL}}(x),$$

dove il primo termine agisce sulle etichette hard del dataset sintetico, mentre il secondo realizza la vera e propria distillazione, imponendo coerenza rispetto al consenso federato.

```
1 def _distill_towards_consensus(self, model, public_dataset,
2   consensus_preds, epochs, lr, temperature, alpha=None):
3
4     if alpha is None:
5         alpha = self.config.get('alpha', 0.5)
6
7     optimizer = torch.optim.Adam(model.parameters(), lr=lr)
8     dataloader = DataLoader(public_dataset, batch_size=self.
9         config['batch_size'], shuffle=False)
10
11     for _ in range(epochs):
12         for batch_idx, (x, y) in enumerate(dataloader):
13             ...
14             outputs = model(x)
15
16             # Soft targets (consenso)
17             student_soft = F.log_softmax(outputs / temperature
18                 , dim=1)
19             soft_loss = F.kl_div(student_soft, target_preds,
20                 reduction='batchmean')
21
22             # Hard targets (etichette sintetiche)
23             hard_loss = F.cross_entropy(outputs, y)
24
25             # Loss combinata
26             loss = (1 - alpha) * hard_loss + alpha * (
27                 temperature ** 2) * soft_loss
28             loss.backward()
29             optimizer.step()
30             ...
```

Listing 3.36: Routine di distillazione federata

3.5.5 Salvataggio dei risultati e monitoraggio

Il tracciamento delle metriche durante l'esecuzione del protocollo FedMD è gestito tramite un meccanismo di logging su file CSV, inizializzato all'avvio del sistema. Il file di destinazione viene definito dinamicamente in base al dataset utilizzato (femnist o irds). Il metodo `setup_csv_logging()` crea il file (se non esistente) e ne scrive l'intestazione con le informazioni necessarie per ogni esperimento: identificativo, iperparametri, metriche di accuratezza e perdita di distillazione.

Durante l'esecuzione, al termine di ciascun round federato, il metodo `log_round_results()` calcola le medie delle metriche locali e le salva nel file CSV. In particolare, vengono riportati:

- `avg_self_accuracy`: accuratezza media di ciascun modello sul proprio test set;
- `avg_cross_accuracy`: accuratezza media incrociata sui test set degli altri client;

Oltre al salvataggio su file, il sistema fornisce un riepilogo su console delle metriche più importanti nella fase di valutazione, facilitando il monitoraggio in tempo reale. In aggiunta, la stampa della matrice di accuratezza inter-client consente di osservare visivamente eventuali squilibri di generalizzazione tra i modelli.

Grazie a questa struttura, i risultati possono essere facilmente analizzati in post-processing con strumenti come `pandas` per l'analisi quantitativa o `matplotlib` per la visualizzazione grafica delle metriche.

3.6 Paradigma 3: FedKD

3.6.1 Struttura generale del codice

L'implementazione del paradigma FedKD segue fedelmente la formulazione presentata nell'apposita sezione. La pipeline è gestita da componenti modulari scritte in PyTorch. La logica è centralizzata attorno alla funzione `run_federated_knowledge_distillation()`, che coordina l'interazione sincrona tra client e server lungo i round federati.

La struttura è composta da tre entità principali:

- `FedKDClient`, che rappresenta il nodo periferico e gestisce l'addestramento locale e la generazione delle soft prediction;
- `FedKDServer`, che mantiene il modello globale, aggrega la conoscenza ricevuta e conduce la fase di distillazione centrale;
- `SyntheticDataGenerator`, incaricato della generazione del dataset sintetico $\mathcal{D}_{\text{syn}}$, a partire dalle statistiche locali condivise dai client.

Tutti i modelli client e il modello globale condividono la stessa architettura, implementata nella classe `StudentModel`, progettata per operare nello spazio delle feature latenti di dimensione $d = 1280$, derivate da un backbone MobileNet pre-addestrato.

Ciclo di apprendimento federato

Il ciclo operativo del protocollo FedKD è implementato nella funzione `run_federated_knowledge_distillation()` e si ripete per un numero predefinito di round (`num_rounds`). Ogni round è composto da una sequenza di fasi sincronizzate che coinvolgono tutti i client e il server. In apertura di ciascun round, ogni client esegue il training locale sul proprio dataset privato. Questo è realizzato tramite la funzione `client.local_training(global_model)`, dove il modello globale corrente viene utilizzato come inizializzazione.

Terminate le fasi locali, il server richiede la generazione di un nuovo dataset sintetico basato sulle statistiche aggregate inviate dai client. La generazione è gestita dal metodo `generate_synthetic_dataset()` della classe `SyntheticDataGenerator` (equivalente a quella descritta in 3.5.3).

Su tale dataset condiviso, ciascun client applica il proprio modello per generare predizioni probabilistiche (softmax) che riflettono la propria visione del task. Le predizioni vengono raccolte in un dizionario tramite `generate_soft_predictions()`. Le predizioni vengono successivamente aggregate sul server mediante media pesata:

Infine, la distillazione centrale viene condotta sul modello globale `global_model`, il quale viene aggiornato usando le soft-label aggregate come target. Il processo è gestito da `distill_knowledge()`.

Alla fine del round, il server valuta le prestazioni del modello aggiornato su ciascun client tramite la funzione `evaluate_global_model()` sui test dei singoli client. Inoltre, è attivo un meccanismo di *early stopping*, basato su una soglia di miglioramento minimo (`min_delta`) e un parametro di pazienza, che consente di terminare anticipatamente l'addestramento se non si osservano progressi.

```

1 for round_num in range(num_rounds):
2
3     # 1. Training locale
4     for client in clients:
5         client.local_training(global_model)
6
7     # 2. Dataset sintetico
8     synthetic_data = server.synthetic_data_generator.
9         generate_synthetic_dataset()
10
11     # 3. Soft predictions
12     client_predictions = [
13         client.generate_soft_predictions(synthetic_data)
14         for client in clients
15     ]
16
17     # 4. Aggregazione
18     aggregated_soft_labels = server.aggregate_knowledge(
19         client_outputs=client_predictions,
20         client_weights=[len(loader.dataset) for loader in
21             client_train_loaders],
22         aggregation_method="weighted_avg"
23     )
24
25     # 5. Distillazione globale
26     server.distill_knowledge(
27         teacher_outputs=aggregated_soft_labels,
28         synthetic_dataset=synthetic_data,
29         temperature=T,
30         alpha=alpha
31     )
32
33     # 6. Valutazione
34     eval_results = server.evaluate_global_model(
35         client_test_loaders)
36
37     # Early Stopping (opzionale)
38     if early_stopping and no_improvement_for >= patience:
39         break

```

3.6.2 Struttura delle classi `FedKDClient` e `FedKDServer`

Nel paradigma FedKD, le due entità principali coinvolte nel ciclo di apprendimento sono rappresentate dalle classi `FedKDClient` e `FedKDServer`. Entrambe sono progettate per operare in modo cooperativo e asincrono all'interno del ciclo federato, mantenendo separazione tra la conoscenza locale e quella condivisa. Le due classi condividono un'istanza

comune del modello `StudentModel` e operano su dati differenti: i client agiscono su dati privati locali, mentre il server si occupa della distillazione centralizzata basata su dati sintetici.

Client: `FedKDClient`

La classe `FedKDClient` incapsula tutta la logica associata al comportamento di un nodo periferico. Ogni oggetto di questa classe rappresenta un client e gestisce due responsabilità principali:

1. L'addestramento locale del modello sul dataset privato;
2. La generazione di predizioni morbide (*soft predictions*) su un dataset sintetico condiviso;

La sua definizione è riportata di seguito:

```
1 class FedKDClient:
2     """
3     Federated Knowledge Distillation Client
4     """
5     def __init__(self, client_id: int, input_dim: int, num_classes:
6         int):
7         self.client_id = client_id
8         self.input_dim = input_dim
9         self.num_classes = num_classes
10        self.local_model = StudentModel(input_dim, num_classes).to(
11            device)
12
13    def local_training(self, train_loader: DataLoader,
14        global_model: nn.Module,
15        epochs: int = 5,
16        lr: float = 0.01) -> Dict:
17        self.local_model.load_state_dict(global_model.state_dict())
18        ...
19        for epoch in range(epochs):
20            for x, y in train_loader:
21                ...
22                outputs = self.local_model(x)
23                loss = F.cross_entropy(outputs, y)
24                loss.backward()
25                optimizer.step()
26            ...
27        return {
28            'final_loss': epoch_losses[-1] if epoch_losses else 0,
29            'epoch_losses': epoch_losses
30        }
31
32    def generate_soft_predictions(self, synthetic_dataset:
33        TensorDataset) -> torch.Tensor:
```

```

31     dataloader = DataLoader(synthetic_dataset, batch_size=64,
32                             shuffle=False)
33     ...
34     with torch.no_grad():
35         for x, _ in dataloader:
36             ...
37             predictions = F.softmax(logits, dim=1)
38             ...
39     return torch.cat(all_predictions, dim=0)
40
41 def get_dataset_size(self, train_loader: DataLoader) -> int:
42     return len(train_loader.dataset)

```

Listing 3.37: Struttura della classe FedKDClient

Ogni client mantiene un proprio modello locale. L'addestramento avviene mediante il metodo `local_training()`, che copia i pesi del modello globale corrente prima di eseguire l'ottimizzazione locale. L'inferenza distribuita è gestita tramite il metodo `generate_soft_predictions()`, che applica softmax ai logit per ottenere probabilità normalizzate. La funzione `get_dataset_size()` fornisce la dimensione del dataset privato, utile durante l'aggregazione pesata.

Server: FedKDServer

La classe `FedKDServer` incarna il ruolo del nodo centrale responsabile della costruzione del modello globale. Essa si occupa di aggregare le predizioni morbose inviate dai client e di distillare la conoscenza su un dataset sintetico costruito in modo indipendente. La sua struttura è riportata di seguito:

```

1 class FedKDServer:
2     def __init__(self, input_dim, num_classes, num_clients):
3         self.global_model = StudentModel(input_dim, num_classes).to(
4             device)
5         self.synthetic_data_generator = SyntheticDataGenerator(
6             input_dim, num_classes, num_clients
7         )
8         ...
9
10    def _initialize_global_model(self):
11        """Initialize global model with Xavier initialization"""
12        ...
13
14    def aggregate_knowledge(self, client_outputs, client_weights,
15                           aggregation_method='weighted_avg'):
16        """Aggregate client predictions"""
17        ...
18        return aggregated
19
20    def distill_knowledge(self, teacher_outputs, synthetic_dataset,
21                          temperature=3.0, alpha=0.5,
22                          distill_epochs=10, lr=0.001):

```

```

22     """Distill aggregated knowledge into global model"""
23     ...
24     return epoch_losses
25
26     def get_global_model_copy(self):
27         """Return a copy of the global model"""
28         return copy.deepcopy(self.global_model)
29
30     def evaluate_global_model(self, test_loaders):
31         """Evaluate global model on all test sets"""
32         ...
33         return {
34             'global_accuracy': global_accuracy,
35             'client_accuracies': client_accuracies,
36             'avg_client_accuracy': np.mean(client_accuracies),
37             'std_client_accuracy': np.std(client_accuracies)
38         }

```

Listing 3.38: Struttura della classe FedKDSERVER

Oltre al modello globale, il server mantiene un'istanza del generatore di dati sintetici, che consente di campionare dati a partire dalle statistiche locali ricevute dai client. I metodi principali implementati nella classe sono:

- `aggregate_knowledge()`: calcola una distribuzione consensuale a partire dalle predizioni morbose ricevute dai client, mediandole con strategie pesate;
- `distill_knowledge()`: aggiorna i parametri del modello globale ottimizzando una loss basata su KL-divergence e cross-entropy rispetto alle predizioni aggregate;
- `evaluate_global_model()`: valuta il modello globale su uno o più dataset di test forniti esternamente.

La logica implementativa assicura che il server non acceda mai direttamente ai dati locali dei client, né ai loro modelli o pesi. Questo garantisce la piena adesione al principio di privacy-by-design tipico dell'apprendimento federato.

3.6.3 Modello condiviso: `StudentModel`

Il modello utilizzato da tutti i client e dal server nel protocollo FedKD è implementato nella classe `StudentModel`, un multilayer perceptron progettato per lavorare direttamente sulle feature di dimensione $d = 1280$ estratte da MobileNetV2.

L'architettura è composta da una sequenza di due livelli `Linear` intercalati da normali operazioni di regolarizzazione e attivazione, seguiti da un livello finale di classificazione. Più precisamente:

- il primo layer lineare proietta l'input $[1280]$ in uno spazio intermedio più ampio $[2 \cdot \text{hidden_dim}]$, seguito da batch normalization, funzione di attivazione ReLU e dropout;
- il secondo layer lineare riduce la dimensionalità a $[\text{hidden_dim}]$, anch'esso seguito da batch normalization, ReLU e dropout;

- infine, un layer lineare di output mappa la rappresentazione latente nello spazio delle classi, restituendo i logit non normalizzati.

La funzione `forward` prende in input un vettore di feature e produce direttamente i logit su cui vengono calcolate le funzioni di perdita (CrossEntropy o KL divergence per la distillazione).

La struttura del modello è riassunta di seguito:

```

1 class StudentModel(nn.Module):
2     def __init__(self, input_dim=1280, num_classes=10, hidden_dim
      =256):
3         super(StudentModel, self).__init__()
4         self.feature_extractor = nn.Sequential(
5             nn.Linear(input_dim, hidden_dim * 2),
6             nn.BatchNorm1d(hidden_dim * 2),
7             nn.ReLU(inplace=True),
8             nn.Dropout(0.3),
9             nn.Linear(hidden_dim * 2, hidden_dim),
10            nn.BatchNorm1d(hidden_dim),
11            nn.ReLU(inplace=True),
12            nn.Dropout(0.2),
13        )
14        self.classifier = nn.Linear(hidden_dim, num_classes)
15        self._initialize_weights()
16
17    def _initialize_weights(self):
18        for m in self.modules():
19            if isinstance(m, nn.Linear):
20                nn.init.kaiming_normal_(m.weight, mode='fan_out',
21                    nonlinearity='relu')
22                ...
23            elif isinstance(m, nn.BatchNorm1d):
24                ...
25
26    def forward(self, x):
27        features = self.feature_extractor(x)
28        return self.classifier(features)

```

Listing 3.39: Struttura del modello StudentModel

3.6.4 Distillazione della conoscenza

La distillazione rappresenta il nucleo del protocollo FedKD: essa consente al server di aggiornare il proprio modello globale f_g a partire dalle predizioni morbide ricevute dai client sul dataset sintetico condiviso $\mathcal{D}_{\text{syn}}$. Questa fase è implementata nella funzione `distill_knowledge()` della classe `FedKDServer`.

Nel dettaglio, il server esegue un ciclo di addestramento su $\mathcal{D}_{\text{syn}}$, in cui per ogni batch vengono combinati due segnali: (i) le etichette sintetiche vere y , e (ii) le distribuzioni probabilistiche aggregate $\bar{P}(x)$, ottenute dalla media pesata delle predizioni dei client. Il modello globale è quindi ottimizzato con una loss composita che combina l'entropia

incrociata standard e la *Kullback-Leibler divergence* tra le distribuzioni predette e quelle fornite dagli ensemble dei client.

Di seguito si riporta la struttura della funzione:

```
1 def distill_knowledge(self, teacher_outputs: torch.Tensor,
2                       synthetic_dataset: TensorDataset,
3                       temperature: float = 3.0,
4                       alpha: float = 0.5,
5                       distill_epochs: int = 10,
6                       lr: float = 0.001) -> Dict:
7     ...
8     for epoch in range(distill_epochs):
9         for x, y in dataloader:
10             ...
11             student_logits = self.global_model(x)
12             kd_loss = ...
13             ce_loss = ...
14             loss = alpha * kd_loss * T**2 + (1 - alpha) * ce_loss
15             ...
16     return epoch_losses
```

Listing 3.40: Metodo `distill_knowledge()` per la distillazione centrale

3.6.5 Logging, metriche e salvataggio dei risultati

La fase di monitoraggio delle metriche e il salvataggio dei risultati sono aspetti fondamentali per l'analisi delle prestazioni del protocollo FedKD. L'intero processo di logging è gestito all'interno della funzione principale `run_federated_knowledge_distillation()`, dove, al termine di ogni round federato, vengono compute e registrate le metriche di accuratezza a livello globale e per ciascun client.

Alcune delle metriche utilizzate sono:

- **Accuracy globale:** misura la percentuale complessiva di predizioni corrette sul test set aggregato di tutti i client;
- **Accuratezza per client:** calcolata individualmente per ogni nodo federato sul rispettivo dataset di test;

Tali metriche vengono ottenute tramite il metodo `evaluate_global_model()` della classe `FedKDServer`, che valuta il modello globale su una lista di `DataLoader` relativi ai test set locali.

In aggiunta, se specificato, i risultati possono essere salvati in formato CSV per l'analisi post-hoc. Ciò consente la generazione di grafici, confronti tra esperimenti e riproducibilità tramite strumenti esterni come `Pandas`.

Capitolo 4

Risultati

In questo capitolo vengono presentati e discussi i risultati degli esperimenti condotti sui diversi paradigmi di apprendimento analizzati: *Federated Learning classico*, *distillazione centralizzata* e *distillazione federata* (FedMD e FedKD). L'obiettivo di questa sezione è valutare in modo sistematico le prestazioni delle varie strategie, confrontandole sia in termini di accuratezza locale sia in termini di capacità di generalizzazione cross-client, su due dataset eterogenei: FEMNIST, caratterizzato da dati manoscritti con distribuzione non-IID per scrivente, IRDS, composto da immagini riabilitative organizzate per paziente.

La presentazione dei risultati segue una struttura coerente: per ciascun paradigma viene descritto lo *spazio degli iperparametri* esplorato, evidenziando le combinazioni più performanti. Successivamente, vengono riportate le *metriche globali* (accuracy, F1-score), seguite da un'analisi disaggregata delle prestazioni per singolo client, allo scopo di evidenziare l'impatto dell'eterogeneità dei dati e della personalizzazione locale. Nel caso dei paradigmi privi di un modello globale, come la distillazione, si introduce inoltre la metrica di accuratezza *cross-client*, per valutare la trasferibilità delle conoscenze acquisite su domini non visti durante l'addestramento.

4.1 Metriche di valutazione

Per valutare le prestazioni dei modelli federati vengono utilizzate le seguenti metriche:

- **Accuracy (Accuratezza):** proporzione di esempi correttamente classificati su tutti gli esempi. È un'indicazione generale della correttezza predittiva, ma può essere fuorviante in presenza di classi sbilanciate.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

dove TP = veri positivi, TN = veri negativi, FP = falsi positivi, FN = falsi negativi.

- **Precision (Precisione):** misura quanto siano affidabili le previsioni positive: ovvero, tra tutti gli esempi che il modello ha classificato come positivi, quale frazione

è corretta.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Sensibilità o Richiamo):** misura la capacità del modello di catturare tutti gli esempi della classe positiva: ovvero, tra tutti gli esempi che in verità sono positivi, quale frazione è stata identificata come tale.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-score:** media armonica tra precision e recall. È particolarmente utile quando si desidera un compromesso tra le due, soprattutto in scenari in cui una delle due metriche da sola non fornisce un quadro completo (ad esempio quando le classi sono sbilanciate).

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Metriche aggregate nei diversi paradigmi. Il significato delle metriche globali (es. accuracy o F1 media) varia a seconda della struttura del paradigma federato considerato.

- Nei paradigmi **con modello globale** (es. FedAvg, FedProx), il modello federato f_G è condiviso da tutti i client e viene testato separatamente sui loro domini. Le metriche aggregate vengono calcolate come media delle prestazioni su ciascun test set locale:

$$\text{AvgAcc}_{\text{global}} = \frac{1}{K} \sum_{i=1}^K \text{Acc}(f_G, \mathcal{D}_i)$$

- Nei paradigmi **con modelli locali** (es. distillazione centralizzata, FedMD), ogni client addestra il proprio modello $f_{S,i}$ in modo indipendente. In questi casi, non esiste un modello federato comune, e le metriche aggregate sono calcolate come media delle performance locali:

$$\text{AvgAcc}_{\text{local}} = \frac{1}{K} \sum_{i=1}^K \text{Acc}(f_{S,i}, \mathcal{D}_i)$$

Tuttavia, queste metriche locali non danno informazioni sulla capacità di generalizzazione oltre il dominio di addestramento. Per questo motivo, nei paradigmi con modelli locali viene introdotta una metrica aggiuntiva: l'**accuratezza cross-client**.

Accuratezza cross-client. La *cross-client accuracy* misura la capacità dei modelli locali di generalizzare su dati provenienti da client diversi dal proprio. Per ciascun modello $f_{S,i}$, si calcola la media delle accuratezze ottenute sui test set degli altri client:

$$\text{CrossAcc}(f_{S,i}) = \frac{1}{K-1} \sum_{\substack{j=1 \\ j \neq i}}^K \text{Acc}(f_{S,i}, \mathcal{D}_j)$$

La *cross accuracy globale* è ottenuta mediando il valore su tutti i client:

$$\text{AvgCrossAcc} = \frac{1}{K} \sum_{i=1}^K \text{CrossAcc}(f_{S,i})$$

Questa metrica è fondamentale per quantificare il *grado di trasferibilità* della conoscenza appresa localmente. Un modello che generalizza bene presenterà valori elevati sia su $\mathcal{D}_i$ che su $\mathcal{D}_j$ con $j \neq i$.

Considerazioni metodologiche. Tutte le metriche sono calcolate sui test set, separati e non utilizzati in fase di addestramento o validazione. I risultati riportati fanno riferimento ai modelli selezionati tramite *early stopping* sulla validation locale, salvo diversa indicazione. Per garantire confrontabilità, le metriche sono sempre espresse come media sui client, eventualmente accompagnate da deviazioni standard o da disaggregazioni per singolo dominio.

4.2 Paradigma 1: Federated Learning Classico

In questa sezione vengono presentati e analizzati i risultati ottenuti applicando diverse strategie federate (FEDAVG, FEDPROX, FEDADAM, FEDADAGRAD) sui dataset FEMNIST e IRDS. L'obiettivo è valutare l'efficacia assoluta e la robustezza di ciascuna strategia in scenari caratterizzati da forte eterogeneità statistica (non-IID) e variazioni architetturali (dimensione dello spazio latente, epoche locali, dimensione del batch, ecc.). Per ogni configurazione si riporta lo spazio di ricerca iperparametrico, le metriche aggregate di accuratezza e F1-score, nonché un'analisi dettagliata delle performance sui singoli client, al fine analizzare la coerenza del modello globale su domini distribuiti.

4.2.1 Risultati su FEMNIST

FedAvg

La tabella 4.1 sintetizza lo spazio degli iperparametri esplorato per FedAvg su FEMNIST.

Tab. 4.1: Spazio degli iperparametri esplorato per **FedAvg** su FEMNIST.

Parametro	Valori esplorati
Learning Rate	0.0001, 0.001, 0.01
Batch Size	16, 32
Gamma	0.1
Latent Dim	7, 10, 15
Hidden Dim	256, 512
Epoche	5, 10, 15
Round	100

I risultati riportati in Tab. 4.2 mostrano che la strategia FEDAVG raggiunge performance eccellenti nel contesto del dataset FEMNIST. Le metriche globali sono stabilmente elevate, con accuratezze comprese tra 0.970 e 0.990 al variare del numero di client. In particolare, lo scenario con tre client rappresenta il caso ottimale, con $\text{Acc} = 0.990$ e $\text{F1} = 0.989$, mentre i valori restano comunque molto alti anche con cinque (0.973) e sette (0.970) client.

Client	Lat_Dim	Hid_Dim	Batch	LR	Gamma	Step	Rounds/Epochs	Acc.	F1
3	15	512	16	0.01	0.1	10	100 / 15	0.990	0.989
5	15	512	16	0.01	0.1	10	100 / 15	0.973	0.973
7	15	256	32	0.01	0.1	10	100 / 15	0.970	0.969

Tab. 4.2: Iperparametri e metriche globali (Accuracy e F1) con FedAvg su FEMNIST.

Dal punto di vista modellistico, l'ottimo comportamento di FEDAVG può essere attribuito a due fattori principali. Primo, la natura del dataset FEMNIST, che pur essendo non-IID presenta una struttura intrinsecamente clusterizzata per utente (scrittura a mano), offre una forma di regolarità che il CI-VAE riesce a catturare grazie alla propria capacità di rappresentazione. Secondo, il meccanismo di media semplice adottato da FEDAVG evita distorsioni introdotte da dinamiche di ottimizzazione complesse (come nei metodi adattivi), permettendo una propagazione stabile e coerente dei pesi globali.

L’accuratezza per singolo client (Tab. 4.3) conferma la qualità della generalizzazione: anche con l’aumento del numero di partecipanti, il modello globale riesce a mantenere alte prestazioni per quasi tutti i client, con diversi casi di accuratezza pari a 1.000. La leggera flessione osservata con sette client può essere attribuita a una maggiore varianza statistica tra utenti, ma non incide significativamente sull’efficacia complessiva del sistema.

Nel complesso, FEDAVG rappresenta una baseline altamente competitiva su FEMNI-ST. In presenza di modelli locali sufficientemente espressivi e dati con strutture latenti coerenti, la semplicità del metodo si traduce in una convergenza rapida e robusta, senza la necessità di introdurre penalizzazioni o correzioni aggiuntive lato server.

# Client	Client 1	Client 2	Client 3	Client 4	Client 5	Client 6	Client 7
3	0.970	1.000	1.000	–	–	–	–
5	0.955	1.000	0.967	0.973	0.969	–	–
7	0.939	1.000	1.000	0.882	1.000	0.967	1.000

Tab. 4.3: Accuratezza per ciascun client nei diversi scenari di FedAvg su FEMNIST. Per i client non presenti nello scenario è riportato –.

FedProx

La tabella 4.4 sintetizza lo spazio degli iperparametri esplorato per FedProx su FEMNI-ST.

Tab. 4.4: Iperparametri esplorati per FedProx su FEMNIST

Parametro	Valori esplorati
Proximal μ	0.01, 0.1, 1.0
Learning Rate	0.0001, 0.001, 0.01
Batch Size	16, 32
Gamma	0.1
Latent Dim	7, 10, 15
Hidden Dim	256, 512
Epoche	5, 10, 15
Round	100

Tab. 4.5: Spazio degli iperparametri esplorato per **FedProx** su FEMNIST.

I risultati sintetizzati in Tab. 4.6 mostrano come la strategia FEDPROX raggiunga prestazioni eccellenti su FEMNIST, con valori di accuratezza e F1 molto elevati in tutti gli scenari. In particolare, si osserva un’accuratezza perfetta (1.000) con tre client, mentre per cinque e sette client i valori si mantengono rispettivamente su 0.982 e 0.964, accompagnati da F1-score di 0.980 e 0.963. Questi risultati sono comparabili con quelli ottenuti da FEDAVG, ma suggeriscono una maggiore robustezza in presenza di un numero più elevato di client, ovvero in condizioni di eterogeneità statistica crescente.

Client	Lat.Dim	Hid.Dim	Batch	LR	Gamma	Step	Rounds/Epochs	μ	Acc.	F1
3	15	512	16	0.01	0.1	10	100 / 15	0.01	1.000	1.000
5	15	256	32	0.01	0.1	5	100 / 15	0.10	0.982	0.980
7	15	512	32	0.01	0.1	5	100 / 5	1.00	0.964	0.963

Tab. 4.6: Iperparametri e metriche globali (Accuracy, F1) con FedProx su FEMNIST al variare del numero di client.

Il principale contributo di FEDPROX risiede nell'introduzione del termine di penalizzazione prossimale controllato dal parametro μ , che agisce come regolarizzatore sulla distanza tra i pesi locali e quelli globali. In presenza di dati fortemente non-IID, come nel caso di FEMNIST, questo vincolo riduce il *client drift*, impedendo che i modelli locali si adattino in modo eccessivo alle distribuzioni specifiche. Il risultato è un modello globale più coerente e stabile.

L'analisi per-client (Tab. 4.7) rafforza questa interpretazione. In tutti gli scenari, i livelli di accuratezza locale sono estremamente alti e distribuiti in modo omogeneo, anche nei casi più complessi con sette client. In particolare, si osservano più client con accuratezza pari a 1.000, a dimostrazione del fatto che FEDPROX riesce a valorizzare le peculiarità locali senza compromettere la generalizzazione.

Dal punto di vista iperparametrico, l'efficacia della strategia è legata a una selezione oculata del coefficiente prossimale: con tre client è sufficiente un $\mu = 0.01$ per ottenere performance ottimali, mentre all'aumentare dei client valori più elevati di μ (fino a 1.0) diventano più efficaci nel contenere la divergenza dei modelli locali. Questo comportamento è coerente con quanto atteso in scenari federati ad alta eterogeneità.

Dunque, FEDPROX dimostra un'elevata efficacia su FEMNIST, riuscendo a contenere il drift e a promuovere una convergenza stabile anche in scenari con elevata diversità tra client. Il termine prossimale si rivela uno strumento essenziale per bilanciare la tensione tra adattamento locale e coerenza globale, in particolare in presenza di modelli generativi come il CI-VAE, che possono facilmente iperadattarsi alle distribuzioni specifiche dei singoli client.

# Client	Client 1	Client 2	Client 3	Client 4	Client 5	Client 6	Client 7
3	1.000	1.000	1.000	—	—	—	—
5	0.973	0.970	1.000	0.969	1.000	—	—
7	1.000	0.967	1.000	0.853	0.955	1.000	0.973

Tab. 4.7: Accuratezza per ciascun client nei diversi scenari di FedProx su FEMNIST. Per i client non presenti nello scenario è riportato —.

FedAdam

La tabella 4.8 sintetizza lo spazio degli iperparametri esplorato per FedAdam su FEMNIST.

Tab. 4.8: Spazio degli iperparametri esplorato per **FedAdam** su FEMNIST.

Parametro	Valori esplorati
Server LR (η)	0.001, 0.01
β_1	0.9
β_2	0.99
Learning Rate (client)	0.0001, 0.001, 0.01
Batch Size	32
Gamma	0.1
Latent Dim	7, 10, 15
Hidden Dim	256, 512
Epoche	5, 10, 15
Step	5, 10
Round	100

I risultati riportati in Tab. 4.9 mostrano che FEDADAM su FEMNIST raggiunge un’accuratezza massima di circa 0.659 con cinque client e valori leggermente inferiori con sette (0.655) e tre (0.510). Le metriche F1 e precisione si mantengono in linea con l’accuratezza, con F1 che raggiunge al massimo 0.607. Questo posiziona FEDADAM nettamente al di sotto delle performance ottenute con FEDAVG e FEDPROX, che nello stesso contesto superano il 96% di accuratezza. La discrepanza sottolinea alcune criticità strutturali dell’ottimizzatore adattivo in combinazione con l’architettura locale adottata e la natura del dataset.

FEMNIST presenta una marcata eterogeneità statistica, essendo partizionato per *writer_id*: ogni client osserva stili calligrafici unici, che portano a manifold latenti locali fortemente divergenti. Il modello utilizzato, un CI-VAE operante su feature congelate da MobileNetV2, cerca di bilanciare tre obiettivi—ricostruzione, regolarizzazione KL e classificazione latente—ma nelle prime fasi del training prevalgono le componenti generative, che favoriscono l’adattamento ai dati locali piuttosto che la generalizzazione cross-client.

In tale contesto, FEDADAM, che aggiorna i pesi globali accumulando momenti di primo (m_t) e secondo ordine (v_t), mostra limiti importanti. Il vantaggio teorico dell’adattività viene vanificato dal fatto che gli pseudo-gradienti aggregati riflettono direzioni molto diverse da client a client: le statistiche che FEDADAM costruisce round dopo round finiscono per smussare aggiornamenti potenzialmente correttivi, rallentando l’allineamento tra le rappresentazioni locali. In particolare, con soli tre client, il drift introdotto da ciascun partecipante pesa di più sull’aggregazione, e i momenti calcolati riflettono traiettorie incompatibili, portando ad accuratèzze basse (0.510).

Al crescere del numero di client (cinque e sette), la maggiore varietà statistica introduce un effetto di media utile, ma non sufficiente: l’accuratezza si stabilizza attorno a 0.66, ben lontana dai valori ottimali. I migliori risultati si ottengono con un learning rate locale relativamente alto (0.01), che accentua il contributo della loss di classificazione e aiuta a costruire uno spazio latente più separabile.

Nel confronto con FEDAVG, che aggrega semplicemente i pesi in media pesata, e FEDPROX, che introduce un termine di penalizzazione per il drift, FEDADAM si dimostra meno efficace. Entrambe le strategie alternative favoriscono una convergenza più coerente tra client, consentendo al classificatore latente del CI-VAE di emergere più rapidamente. In particolare, FEDPROX riesce a regolare l’oscillazione dei client intorno alla media globale, contenendo il disallineamento e migliorando la qualità degli aggiornamenti.

In definitiva, le scarse prestazioni osservate non derivano unicamente dalla strategia di ottimizzazione o dalla natura del modello, ma piuttosto dall’interazione poco sinergica tra i due. Il CI-VAE, nella sua fase iniziale, privilegia l’adattamento locale tramite ricostruzione, generando spazi latenti eterogenei tra client. FedAdam, dal canto suo, fatica a combinare aggiornamenti così divergenti, poiché i momenti accumulati tendono a smorzare proprio quelle correzioni forti che sarebbero necessarie per allineare le traiettorie. L’effetto combinato è una convergenza instabile e poco generalizzabile.

Client	Lat_Dim	Hid_Dim	Round	Epoch	LR	η_{Adam}	β_1	β_2	γ	Step	Acc.	P.	F1	R.
3	7	256	100	10	0.001	0.01	0.9	0.99	0.1	5	0.510	0.421	0.435	0.510
5	7	256	100	10	0.010	0.01	0.9	0.99	0.1	5	0.659	0.633	0.602	0.659
7	7	256	100	5	0.010	0.01	0.9	0.99	0.1	5	0.655	0.612	0.607	0.655

Tab. 4.9: Migliori configurazioni e risultati di FedAdam al variare del numero di client. I valori sono arrotondati a tre cifre decimali.

FedAdagrad

La tabella 4.10 sintetizza lo spazio degli iperparametri esplorato per FedAdagrad su FEMNIST.

Tab. 4.10: Spazio degli iperparametri esplorato per **FedAdagrad** su FEMNIST.

Parametro	Valori esplorati
Server LR (η)	0.001, 0.01, 0.1, 0.5
Learning Rate (client)	0.00001, 0.0001, 0.001, 0.01
Batch Size	16, 32
Gamma	0.1
Latent Dim	7, 10
Hidden Dim	512
Epoche	5, 10, 15
Step	5, 10
Round	100

I risultati riportati in Tab. 4.11 mostrano che FEDADAGRAD, pur essendo concepito per migliorare la stabilità dell’ottimizzazione in scenari non-IID, ha ottenuto prestazioni significativamente inferiori rispetto ad altre strategie federate nel contesto di FEMNIST. Le accuratze globali si collocano tra 0.610 e 0.721, con valori di F1 che variano da 0.527 a 0.673, ben al di sotto di quanto osservato con FEDAVG o FEDPROX (entrambe oltre il 96% di accuratezza).

Questi risultati non possono essere attribuiti unicamente alla strategia di ottimizzazione o al modello utilizzato, ma derivano da una combinazione poco sinergica tra l’algoritmo FEDADAGRAD e il modello locale CI-VAE. Il meccanismo di aggiornamento di FEDADAGRAD si basa sull’adattamento individuale del learning rate dei parametri, riducendoli proporzionalmente all’accumulo del quadrato dei gradienti storici. In contesti federati, ciò dovrebbe garantire maggiore stabilità contro l’oscillazione dei gradienti locali, ma in combinazione con un CI-VAE questa proprietà si rivela problematica.

Il CI-VAE, infatti, è un modello complesso che ottimizza simultaneamente tre obiettivi: la ricostruzione dell’input, la regolarizzazione tramite KL divergence, e la classificazione sullo spazio latente. Nelle fasi iniziali dell’addestramento, la componente generativa (ricostruzione + KL) tende a prevalere, portando i modelli a specializzarsi sul dominio locale e costruire rappresentazioni latenti poco generalizzabili. FEDADAGRAD, riducendo progressivamente il learning rate effettivo dei parametri, congela prematuramente queste traiettorie di specializzazione, bloccando il successivo adattamento verso una rappresentazione condivisa tra client.

Di conseguenza, l’ottimizzazione guidata da FEDADAGRAD non riesce a controbilanciare il bias locale introdotto dalla componente generativa del CI-VAE. L’effetto complessivo è duplice: da un lato si ottiene un modello localmente coerente ma incapace di apprendere

strutture comuni, dall'altro la ridotta plasticità impedisce alle successive fasi di classificazione di emergere pienamente.

In definitiva, l'accoppiamento tra FEDADAGRAD e CI-VAE si dimostra inadatto per scenari federati altamente non-IID come FEMNIST. La convergenza è lenta, fortemente influenzata dal rumore locale, e le rappresentazioni risultanti sono poco trasferibili. Questo sottolinea la necessità di strategie più plastiche o di modelli meno soggetti a dominanza generativa nelle prime fasi, per ottenere risultati competitivi in scenari reali.

Client	Lat_Dim	Hid_Dim	Rounds	Epochs	Batch	LR	η_{Adagrad}	γ	Step	Acc.	P	F1	R
3	10	512	100	10	16	0.0001	0.1	0.1	10	0.721	0.723	0.673	0.721
5	15	512	100	5	32	0.00001	0.1	0.1	5	0.610	0.497	0.527	0.610
7	15	512	100	15	16	0.00001	0.1	0.1	5	0.717	0.643	0.654	0.717

Tab. 4.11: Migliori configurazioni e risultati di FedAdagrad su FEMNIST al variare del numero di client. I valori sono arrotondati a tre cifre decimali.

Confronto tra strategie

La Tabella 4.12 fornisce una panoramica comparativa delle principali strategie federate testate sul dataset FEMNIST, al variare del numero di client.

Tecnica	3 client		5 client		7 client	
	Acc	F1	Acc	F1	Acc	F1
FedAvg	0.990	0.989	0.973	0.973	0.970	0.969
FedProx	1.000	1.000	0.982	0.980	0.964	0.963
FedAdam	0.510	0.435	0.659	0.602	0.655	0.607
FedAdagrad	0.721	0.673	0.610	0.527	0.717	0.654

Tab. 4.12: Confronto delle performance su FEMNIST: accuratezza e F1-score per ciascuna tecnica federata e numero di client.

I risultati confermano che FedProx è la strategia più efficace nel contesto di dati altamente non-IID come FEMNIST. In particolare, raggiunge valori di accuratezza e F1-score pari al 100% con 3 client e mantiene prestazioni molto elevate anche con 5 e 7 client. L'introduzione del termine prossimale consente al modello di contenere la divergenza tra le ottimizzazioni locali, migliorando la stabilità dell'addestramento.

FedAvg, pur non integrando meccanismi di regolarizzazione, mostra prestazioni competitive e costanti, con una lieve flessione all'aumentare della frammentazione (da 0.990 a 0.970 di accuratezza da 3 a 7 client). Ciò dimostra che, in presenza di dati con distribuzione moderatamente non bilanciata, anche strategie semplici possono risultare molto efficaci.

Al contrario, le strategie FedAdam e FedAdagrad evidenziano una maggiore instabilità e una forte dipendenza dalla specifica configurazione. Questo comportamento suggerisce che i meccanismi di adattamento dinamico del learning rate possono risultare meno efficaci con l'architettura presa in esame.

È importante notare che le differenze tra strategie si osservano nonostante l'uniformità dell'architettura CI-VAE. Questo suggerisce che, anche a parità di modello, la strategia di ottimizzazione federata influisce in modo determinante sulle performance, in particolare sul bilanciamento tra specializzazione locale e coerenza globale. La robustezza di

FedProx e la semplicità di FedAvg ne fanno approcci particolarmente adatti a scenari in cui è richiesto un buon compromesso tra accuratezza, stabilità e scalabilità.

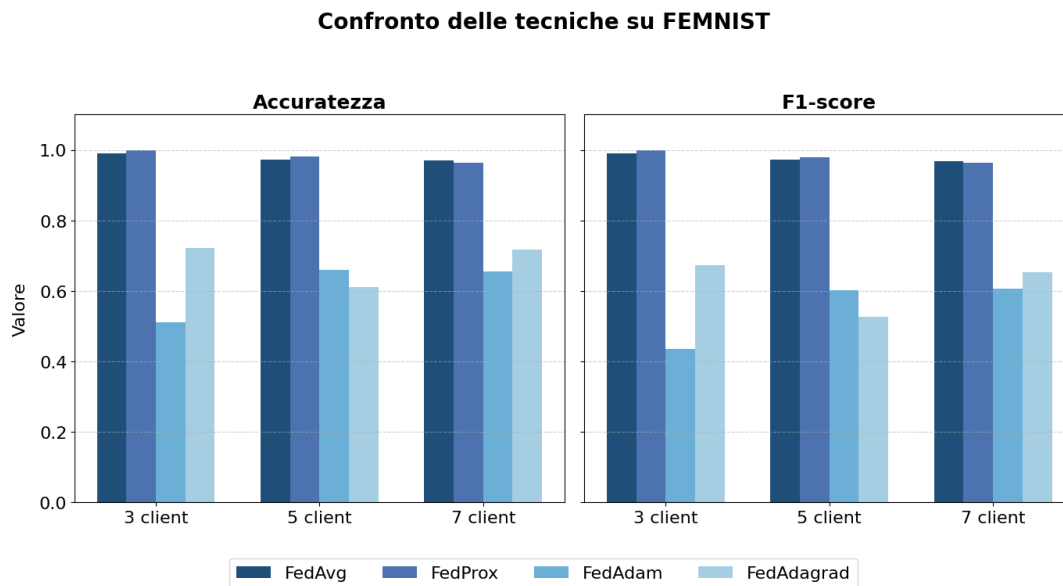


Fig. 4.1: Confronto tra tecniche su FEMNIST: accuratezza e F1-score in funzione del numero di client.

4.2.2 Risultati su IRDS

FedAvg

Tab. 4.13: Spazio degli iperparametri per la strategia **FedAvg**.

Parametro	Valori esplorati
Learning Rate	0.0005, 0.001, 0.01
Batch Size	32, 64, 128, 256, 512
Gamma	0.1
Latent Dim	7, 10, 12
Hidden Dim	256, 512
Epoche	1, 2, 3
Round	200

I risultati riportati in Tab. 4.17 mostrano che la strategia FEDAVG ottiene prestazioni solide su IRDS, con accuratezze globali comprese tra 0.863 e 0.911, e valori di F1-score perfettamente allineati. Nello scenario con cinque client si osserva la performance migliore (Acc = 0.911, F1 = 0.911), mentre con tre e sette client le metriche si attestano rispettivamente su 0.903 e 0.863.

Client	Lat_Dim	Hid_Dim	Batch	LR	Gamma	Step	Rounds	Epochs	Acc.	F1
3	10	512	128	0.001	0.1	20	200	3	0.903	0.902
5	10	512	128	0.001	0.1	20	200	3	0.911	0.911
7	12	512	256	0.001	0.1	20	200	3	0.863	0.860

Tab. 4.14: Iperparametri e metriche globali ottenute con FedAvg al variare del numero di client.

L'efficacia di FEDAVG in questo contesto è attribuibile alla semplicità del meccanismo di aggregazione, che consente di combinare gli aggiornamenti locali senza introdurre dinamiche adattive potenzialmente instabili nei primi round. Questa media pesata si rivela particolarmente efficace in presenza di client che, pur partendo da distribuzioni eterogenee, riescono a produrre rappresentazioni sufficientemente compatibili grazie al CI-VAE.

L'analisi delle accuratezze per singolo client (Tab. 4.15) conferma la buona generalizzazione del modello globale. In ogni scenario, l'accuratezza locale si mantiene al di sopra di 0.817, con valori molto elevati soprattutto nei casi con meno client, dove il disallineamento statistico ha un impatto inferiore. Con cinque client, ad esempio, quattro client superano 0.91, mentre con sette client si osserva una maggiore variabilità, con valori che spaziano tra 0.817 e 0.906. Tale dispersione è fisiologica in scenari altamente non-IID.

# Client	Client 1	Client 2	Client 3	Client 4	Client 5	Client 6	Client 7
3	0.935	0.919	0.855	—	—	—	—
5	0.919	0.920	0.930	0.863	0.924	—	—
7	0.903	0.833	0.818	0.904	0.861	0.906	0.817

Tab. 4.15: Accuratezza per ciascun client nei diversi scenari di FedAvg. Per i client non presenti nello scenario è riportato —.

FEDAVG su IRDS si dimostra una scelta efficace anche in presenza di eterogeneità moderata. L'assenza di vincoli rigidi o meccanismi adattivi consente al modello globale di

mantenere un buon equilibrio tra specificità locale e generalizzazione, purché la qualità delle rappresentazioni latenti sia sufficientemente elevata. La sua stabilità e semplicità operativa ne fanno una baseline solida e competitiva anche in scenari clinici con distribuzioni complesse.

FedProx

Tab. 4.16: Spazio degli iperparametri per la strategia **FedProx**.

Parametro	Valori esplorati
Proximal μ	0.01, 0.1, 1.0
Learning Rate	0.001, 0.01
Batch Size	32, 64, 128, 256, 512
Gamma	0.1
Latent Dim	7, 10, 12, 15
Hidden Dim	256, 512
Epoche	1, 2, 3
Round	200

I risultati riportati in Tab. 4.17 mostrano che la strategia FEDPROX si comporta in modo solido ed efficace anche sul dataset IRDS, raggiungendo un'accuratezza globale di 0.912 con tre client, 0.909 con cinque client e 0.862 con sette client. Le prestazioni F1 sono perfettamente allineate ai valori di accuratezza, indicando una buona coerenza nella capacità discriminativa del modello.

Client	Lat.Dim	Hid.Dim	Batch	LR	Gamma	Step	Rounds/Epochs	μ	Acc.	F1
3	10	512	256	0.001	0.1	20	200 / 2	0.01	0.912	0.912
5	10	512	256	0.001	0.1	20	200 / 3	0.01	0.909	0.908
7	10	512	256	0.001	0.1	20	200 / 3	0.01	0.862	0.858

Tab. 4.17: Iperparametri e metriche globali (Accuracy, F1) con FedProx al variare del numero di client.

Il buon rendimento di FEDPROX è legato all'introduzione di un termine prossimale nella loss locale, che penalizza l'allontanamento del modello locale rispetto a quello globale. Questo vincolo si rivela particolarmente efficace in presenza di forte eterogeneità statistica, come nel caso di IRDS, dove ogni client rappresenta un paziente distinto, con traiettorie motorie e condizioni esecutive uniche. Il termine $\mu = 0.01$ ha garantito un equilibrio ottimale tra adattamento locale e stabilità globale.

L'interazione tra FedProx e l'architettura CI-VAE è un ulteriore fattore di successo. Il modello latente, vincolato a non discostarsi eccessivamente dal centro gravitazionale imposto dal server, riesce a produrre rappresentazioni compatibili anche in presenza di forti specializzazioni locali. La penalizzazione del drift contribuisce così a stabilizzare le traiettorie di ottimizzazione, favorendo la convergenza globale senza compromettere l'adattabilità.

I risultati per singolo client (Tab. 4.18) confermano l'equilibrio raggiunto tra performance locali e generalizzazione. In tutti gli scenari, l'accuratezza locale si mantiene elevata, con valori superiori a 0.84 in ogni configurazione e picchi oltre 0.95. Anche nello scenario

più eterogeneo (sette client), il sistema mostra buona robustezza, pur evidenziando una lieve diminuzione di uniformità, fisiologica all'aumentare della complessità statistica.

# Client	Client 1	Client 2	Client 3	Client 4	Client 5	Client 6	Client 7
3	0.935	0.870	0.931	—	—	—	—
5	0.911	0.913	0.955	0.921	0.844	—	—
7	0.794	0.913	0.846	0.885	0.765	0.916	0.917

Tab. 4.18: Accuratezza per ciascun client nei diversi scenari di FedProx. Per i client non presenti nello scenario è riportato —.

FEDPROX su IRDS si dimostra una strategia efficace e robusta: la regolarizzazione prossimale mitiga il drift tra client, migliora la compatibilità tra rappresentazioni latenti e guida il CI-VAE verso soluzioni stabili e generalizzabili.

FedAdam

Tab. 4.19: Spazio degli iperparametri per la strategia **FedAdam**.

Parametro	Valori esplorati
Server LR (η)	0.001, 0.01
β_1	0.9
β_2	0.99
Learning Rate (client)	0.0001, 0.001
Batch Size	128
Gamma	0.1
Latent Dim	7, 10, 15
Hidden Dim	512
Epoche	1, 3, 5
Round	200

I risultati riportati in Tab. 4.20 mostrano che FEDADAM ha raggiunto prestazioni contenute su IRDS, con accuratezza tra 0.644 e 0.684 e F1-score compresi tra 0.607 e 0.677. La precisione risulta più alta della recall, con un massimo di 0.762, segnalando un comportamento selettivo: il modello riconosce bene le classi più frequenti ma fatica con quelle meno rappresentate.

Questo andamento è coerente con le caratteristiche di IRDS, un dataset non-IID partizionato per paziente, che presenta sbilanciamenti tra classi, pose simili tra categorie diverse e condizioni di acquisizione eterogenee. In questo contesto, i client apprendono rappresentazioni molto diverse tra loro e gli aggiornamenti che il server riceve risultano incoerenti.

L'ottimizzazione di FEDADAM si basa su momenti del primo e secondo ordine, ma la non-stazionarietà dei gradienti aggregati tra i client limita l'efficacia del meccanismo adattivo: i momenti possono accumulare direzioni contrastanti, generando un passo di aggiornamento che è troppo cauto o, al contrario, sbilanciato. Questo porta a una convergenza stabile ma poco efficace, che si blocca su plateaux prestazionali.

Anche il modello utilizzato (CI-VAE su embedding congelati di MobileNetV2) influisce sul risultato. L'encoder del CI-VAE deve comprimere le immagini, regolarizzarne la

rappresentazione latente e classificare correttamente, ma le prime fasi di training sono dominate dalla parte generativa della loss, ritardando l'apprendimento di rappresentazioni discriminative. Il numero ridotto di epoche locali e l'uso di batch grandi limitano ulteriormente l'adattabilità, e i learning rate bassi impediscono di correggere efficacemente il drift tra i client.

Con sette client si osserva un leggero miglioramento (F1 e precision più alte), ma l'accuratezza resta attorno a 0.68, indicando un collo di bottiglia strutturale. Con cinque client, le prestazioni scendono a 0.644, evidenziando che l'adattività di FEDADAM da sola non è sufficiente.

In sintesi, FEDADAM non è riuscito a superare le difficoltà legate alla struttura non-IID di IRDS, alla rigidità del modello e a una configurazione conservativa degli iperparametri.

Client	Lat_Dim	Hid_Dim	Rounds	Epochs	LR	η_{Adam}	β_1	β_2	Acc.	P	F1	R.
3	7	512	200	5	0.001	0.01	0.9	0.99	0.667	0.677	0.656	0.667
5	15	512	200	3	0.001	0.01	0.9	0.99	0.644	0.679	0.607	0.644
7	10	512	200	3	0.001	0.01	0.9	0.99	0.684	0.762	0.677	0.684

Tab. 4.20: Migliori configurazioni e risultati di FedAdam al variare del numero di client. I valori sono arrotondati a tre cifre decimali.

FedAdagrad

Tab. 4.21: Spazio degli iperparametri per la strategia **FedAdagrad**.

Parametro	Valori esplorati
Server LR (η)	0.001, 0.01, 0.1
Learning Rate (client)	0.001, 0.01
Batch Size	128
Gamma	0.1
Latent Dim	7, 10, 15
Hidden Dim	512
Epoche	1, 3, 5
Round	200

I risultati riportati in Tab. 4.22 mostrano che FEDADAGRAD ottiene performance piuttosto basse su IRDS, con accuratezze comprese tra 0.487 e 0.628 e valori di F1 sempre inferiori a 0.620. Queste prestazioni risultano significativamente inferiori rispetto a strategie più semplici come FEDAVG o più regolarizzate come FEDPROX, che superano il 90% di accuratezza nello stesso contesto. La causa non è da attribuire esclusivamente al meccanismo di FEDADAGRAD, né al modello CI-VAE, ma piuttosto alla loro combinazione: le dinamiche di ottimizzazione dell'algoritmo entrano in conflitto con le proprietà del modello, compromettendo l'apprendimento condiviso.

Il CI-VAE utilizzato in questo lavoro non è un semplice classificatore, ma un modello generativo-discriminativo che ottimizza una funzione di perdita composta da tre termini: ricostruzione, regolarizzazione (KL divergence) e classificazione. In particolare, nelle fasi iniziali dell'addestramento, è la parte generativa a guidare la costruzione dello spazio latente. Questo comporta che ogni client tende a modellare i propri dati in maniera fedele, ma poco discriminativa rispetto alle classi. In un ambiente federato non-IID come

IRDS, dove ciascun client rappresenta un paziente con movimenti eterogenei, i modelli locali evolvono verso rappresentazioni profondamente divergenti tra loro.

In tale scenario, l'intervento di FEDADAGRAD peggiora la situazione. Il suo principio di adattamento — ridurre il passo di aggiornamento per i parametri frequentemente modificati — entra in conflitto con il bisogno del CI-VAE di correggere progressivamente lo spazio latente e favorire la classificazione. Dopo poche iterazioni, i gradienti accumulati spingono FEDADAGRAD a congelare i parametri più attivi, impedendo l'evoluzione del modello verso rappresentazioni condivise. Ciò blocca la componente discriminativa del CI-VAE prima che possa emergere e rafforzarsi, lasciando prevalere la parte generativa e locale. Gli aggiornamenti aggregati si riflettono solo marginalmente sui parametri cruciali, e la progressiva riduzione del learning rate vanifica gli effetti correttivi che sarebbero necessari per armonizzare le rappresentazioni apprese.

Al contrario, strategie come FEDAVG e FEDPROX mantengono la plasticità dei parametri durante tutto il training. In particolare, FEDPROX introduce una penalità esplicita al drift dei modelli locali rispetto alla media globale, che si rivela compatibile con la necessità del CI-VAE di mantenere uno spazio latente coerente tra client. In questi casi, la componente discriminativa ha più tempo e capacità di emergere, e le prestazioni migliorano sensibilmente.

In sintesi, il problema non risiede soltanto nell'algoritmo FEDADAGRAD, né nella struttura del CI-VAE, ma nella loro interazione. L'adattività eccessivamente conservativa del primo soffoca la dinamica di apprendimento stratificata del secondo. Questa sinergia negativa impedisce l'ottimizzazione efficace del classificatore latente, ostacolando la generalizzazione cross-client e conducendo a prestazioni globali inferiori rispetto a strategie più compatibili con la struttura del modello.

Client	Latent Dim	Hidden Dim	Rounds	Epochs	LR	η_{Adagrad}	Acc.	P.	cR.	F1
3	15	512	200	5	0.001	0.1	0.628	0.687	0.628	0.620
5	10	512	200	5	0.001	0.01	0.487	0.424	0.487	0.413
7	10	512	200	5	0.001	0.01	0.533	0.429	0.533	0.460

Tab. 4.22: Migliori configurazioni e risultati di FedAdagrad al variare del numero di client.

Confronto tra strategie

La Tabella 4.23 e il grafico in Fig. 4.2 mostrano chiaramente come, su IRDS, non tutte le strategie federate siano ugualmente adatte a interagire con il modello CI-VAE.

Tecnica	3 client		5 client		7 client	
	Acc	F1	Acc	F1	Acc	F1
FedAvg	0.903	0.902	0.911	0.911	0.863	0.860
FedProx	0.912	0.912	0.909	0.908	0.862	0.858
FedAdam	0.667	0.656	0.644	0.607	0.684	0.677
FedAdagrad	0.628	0.620	0.487	0.413	0.533	0.460

Tab. 4.23: Confronto delle performance su IRDS: accuratezza e F1-score per ciascuna tecnica federata e numero di client.

FEDAVG si conferma una baseline sorprendentemente efficace: la media pesata degli aggiornamenti, pur priva di sofisticazioni, permette al CI-VAE di sviluppare rappresentazioni compatibili e generalizzabili, mantenendo valori di accuratezza tra 0.86 e 0.91.

FEDPROX rafforza ulteriormente questo equilibrio introducendo un vincolo esplicito al drift locale. Il termine prossimale agisce come regolarizzatore delle traiettorie di ottimizzazione e consente al modello globale di restare stabile anche quando aumenta il numero di pazienti, senza sacrificare adattabilità.

In teoria, l'adattività di FedAdam dovrebbe favorire la convergenza in scenari eterogenei; in pratica, la natura del CI-VAE ne accentua i limiti. Il modello, infatti, è dominato nelle prime fasi dalla componente generativa, che spinge ogni client a creare spazi latenti molto divergenti. I momenti accumulati dal server diventano quindi una media di gradienti incoerenti e non stazionari: invece di correggere il drift, finiscono per smussare le correzioni necessarie. Il risultato è una convergenza piatta intorno al 65%, che riflette più un compromesso inefficace tra traiettorie divergenti che una reale armonizzazione globale.

Il limite di FEDADAGRAD è ancora più evidente. La logica di ridurre progressivamente il passo sui parametri più aggiornati collide con le necessità del CI-VAE, che ha bisogno di un lungo periodo di plasticità per permettere alla componente discriminativa di emergere dopo la fase iniziale dominata dalla ricostruzione. Invece, i learning rate effettivi vengono congelati troppo presto: i modelli locali rimangono bloccati in rappresentazioni generative fortemente soggetto-specifiche, senza possibilità di riallineamento. Il risultato è un modello globale poco trasferibile, con accuratze che precipitano fino a 0.48 su 5 client e restano inferiori al 0.53 su 7 client.

Questo confronto mette in evidenza che non è tanto la sofisticatezza dell'ottimizzatore a determinare il successo, quanto la sua compatibilità con la dinamica del modello. Con architetture generative-discriminative come il CI-VAE, approcci adattivi basati su momenti o su accumulo dei gradienti finiscono per amplificare il client drift o congelare precocemente i parametri. Al contrario, strategie semplici (FedAvg) o dotate di regolarizzazione esplicita (FedProx) risultano più efficaci perché preservano plasticità e stabilità, due elementi essenziali in scenari clinici non-IID.

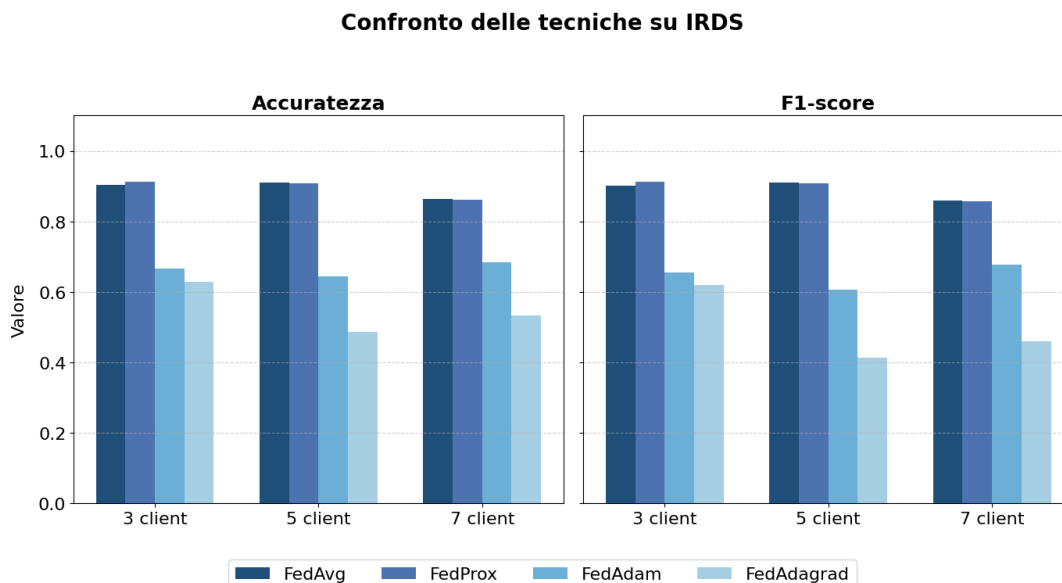


Fig. 4.2: Confronto tra tecniche su IRDS: accuratezza e F1-score in funzione del numero di client.

4.3 Paradigma 2: Distillazione Centralizzata

In questa sezione vengono riportati i risultati della distillazione centralizzata sui dataset FEMNIST e IRDS. Per ciascun caso è stata eseguita una grid search sugli iperparametri principali (tasso di apprendimento, batch size, temperatura T , coefficiente α), come riepilogato nelle Tabelle 4.24, 4.26 e 4.28. I valori sono stati scelti in modo da coprire un ampio intervallo di configurazioni, garantendo al contempo stabilità dell’addestramento. Nelle sezioni seguenti vengono discussi i risultati migliori sia in termini di performance locale, sia di generalizzazione cross-client.

4.3.1 Risultati su FEMNIST

Iperparametro	Valori
Teacher Learning Rate ($\eta_{teacher}$)	{0.01, 0.005, 0.001}
Student Learning Rate ($\eta_{student}$)	{0.01, 0.005, 0.001}
Teacher Epochs	150
Student Epochs	80
Teacher Gamma	0.95
Student Gamma	0.95
Teacher Step Size	15
Student Step Size	12
Temperature (T)	{3, 4, 5}
Alpha (α)	{0.5, 0.7, 0.9}
Batch Size	{32, 64}
Teacher Dropout	0.2
Student Dropout	0.2

Tab. 4.24: Iperparametri della Knowledge Distillation centralizzata su FEMNIST con reti neurali.

La Tabella 4.24 riassume lo spazio di ricerca iperparametrico considerato per gli esperimenti su FEMNIST con architetture MLP. Sono stati esplorati tassi di apprendimento, dimensioni del batch, temperatura T e coefficiente α della loss di distillazione, mantenendo invariati gli altri iperparametri. L’obiettivo era valutare la capacità della distillazione centralizzata di trasferire conoscenza tra client caratterizzati da distribuzioni non-IID di dati di scrittura manuale.

Client	Scenario	η_T	η_S	T	α	B	acc.	cross	P	R	F1
3	migliore locale	0.01	0.01	4	0.7	32	1.000	0.773	1.000	1.000	1.000
3	migliore cross	0.01	0.01	5	0.5	64	0.978	0.799	0.985	0.978	0.976
5	migliore locale	0.001	0.01	3	0.7	32	1.000	0.682	1.000	1.000	1.000
5	migliore cross	0.005	0.005	5	0.7	64	1.000	0.749	1.000	1.000	1.000
7	migliore locale	0.001	0.001	3	0.5	32	0.988	0.600	0.994	0.988	0.988
7	migliore cross	0.005	0.005	5	0.9	64	0.946	0.709	0.941	0.946	0.936

Tab. 4.25: Risultati migliori su FEMNIST (MLP) al variare del numero di client. Sono riportate le configurazioni che massimizzano rispettivamente l’accuratezza locale e la generalizzazione cross-client. Per ciascun modello sono indicati anche precision, recall e F1. B indica la dimensione del batch.

L’analisi dei risultati mostra come la distillazione centralizzata garantisca prestazioni eccellenti in termini di accuratezza locale: i modelli studente raggiungono valori prossimi o uguali al 100%, con metriche di precision, recall e F1 perfettamente allineate. Ciò conferma che il paradigma riesce a trasmettere informazioni sufficienti anche a modelli con

capacità ridotta per ricostruire accuratamente le distribuzioni locali, senza compromessi tra falsi positivi e falsi negativi.

Il quadro è differente se si considera la generalizzazione cross-client. Qui le prestazioni si attestano tra 0.60 e 0.80, con variazioni sensibili al variare degli iperparametri. Temperature più alte ($T = 5$) e valori elevati del coefficiente α (0.7–0.9) favoriscono un miglior trasferimento della conoscenza, mentre configurazioni più conservative, pur massimizzando l'accuratezza sul dominio nativo, mostrano una ridotta capacità di adattarsi a distribuzioni eterogenee.

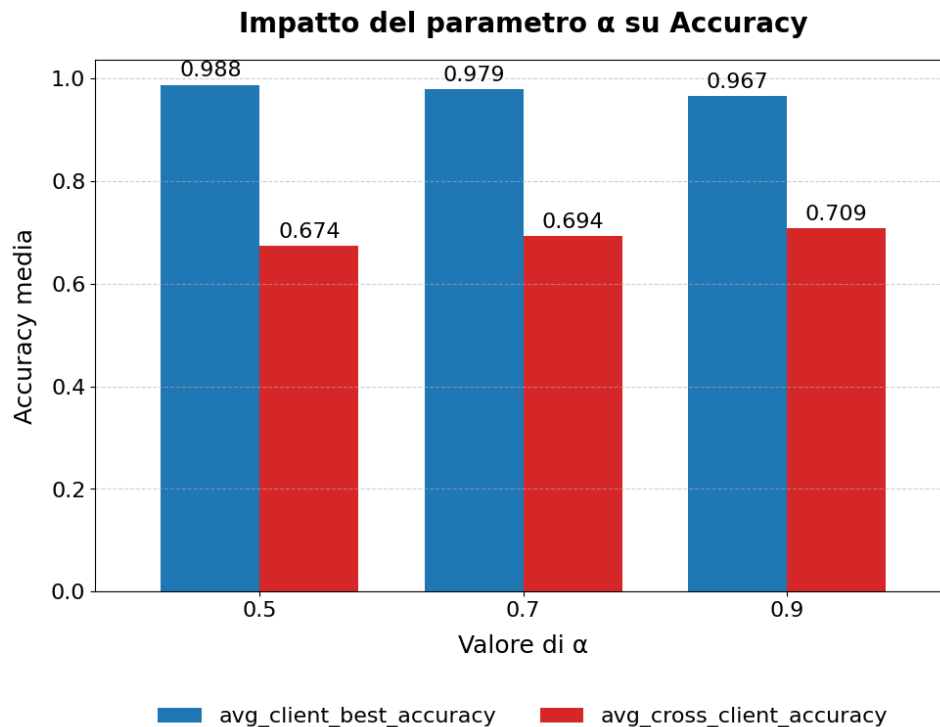


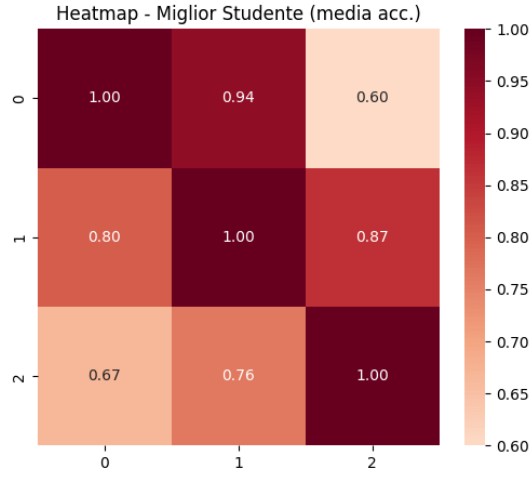
Fig. 4.3: Impatto del coefficiente α sull'accuratezza locale e cross-client su FEMNIST (MLP) su 7 client. In blu il miglior modello locale, in rosso il miglior modello valutato per cross accuracy.

Il ruolo di α emerge chiaramente anche dal grafico in Fig. 4.3, che mostra l'andamento dell'accuratezza al variare di α su 7 client. Il grafico evidenzia come l'incremento di questo iperparametro comporti una leggera riduzione dell'accuratezza locale (da 0.988 a 0.967), ma al tempo stesso migliori in modo consistente la capacità di generalizzazione (da 0.674 a 0.709). Questo risultato dimostra che la loss di distillazione, opportunamente bilanciata, svolge un ruolo cruciale nel regolare il compromesso tra apprendimento specializzato e trasferibilità delle conoscenze.

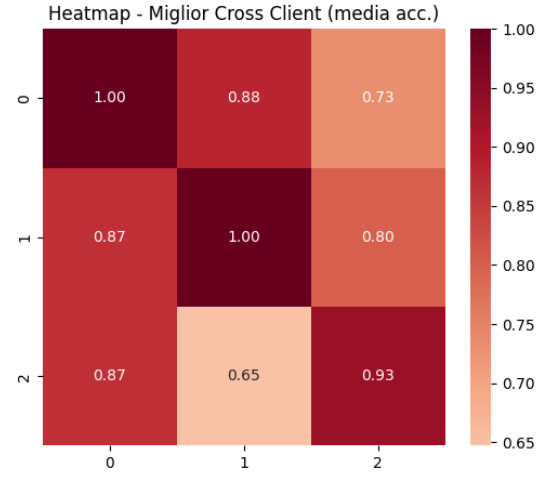
Un ulteriore elemento da considerare riguarda l'effetto del numero di client. Con l'aumentare di K , e quindi della frammentazione dei dati, la generalizzazione tende a ridursi: la migliore cross accuracy scende da 0.799 con tre client a 0.709 con sette. Le matrici di accuratezza in Fig. 4.4 confermano questa dinamica. Nelle configurazioni ottimizzate per la generalizzazione (alte temperature e valori consistenti di α), si osservano incrementi significativi dei valori fuori diagonale, segno che la distillazione riesce effettivamente ad attenuare le differenze tra client.

In definitiva, l'evidenza empirica indica che il compromesso ottimale si ottiene in configu-

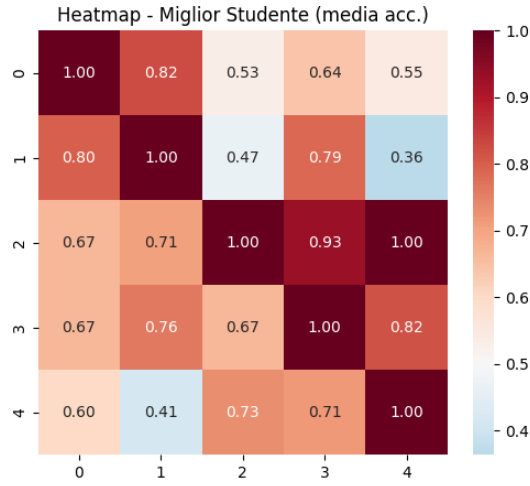
razioni caratterizzate da temperature elevate e valori di α consistenti, che pur sacrificando marginalmente l'accuratezza locale, garantiscono una migliore capacità di trasferire conoscenza tra domini eterogenei.



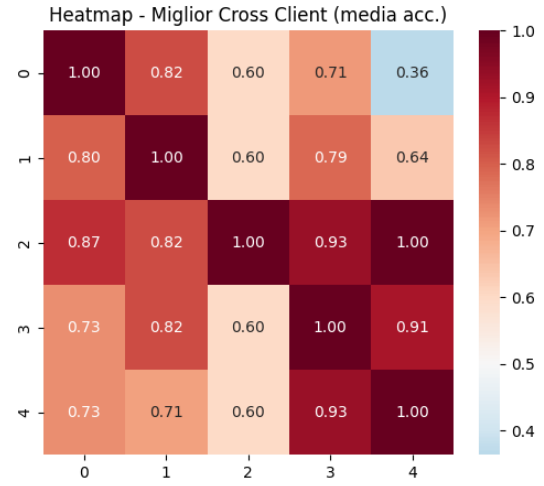
(a) Self-accuracy, 3 client



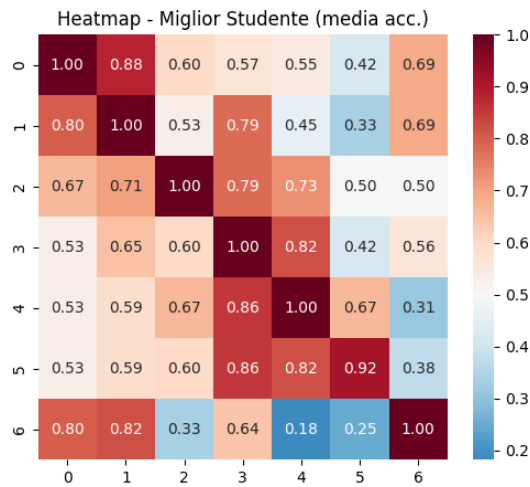
(b) Cross-accuracy, 3 client



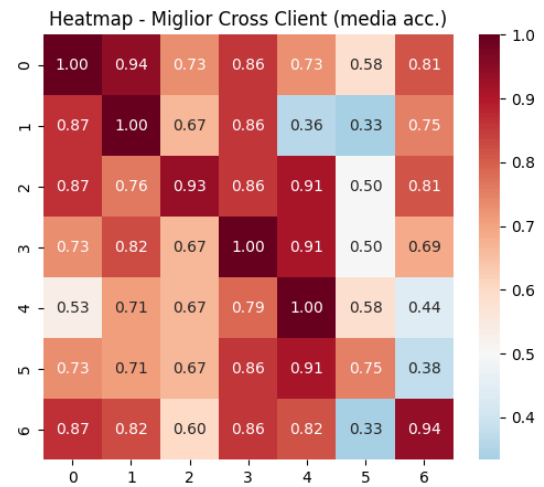
(c) Self-accuracy, 5 client



(d) Cross-accuracy, 5 client



(e) Self-accuracy, 7 client



(f) Cross-accuracy, 7 client

Fig. 4.4: Matrici di accuratezza self e cross-client su FEMNIST (MLP), al variare del numero di client. Le matrici a sinistra mostrano la performance sul dominio proprio, mentre quelle a destra evidenziano la capacità di generalizzazione verso domini eterogenei.

4.3.2 Risultati su IRDS con MLP

La Tabella 4.26 riassume lo spazio di ricerca considerato per gli esperimenti su IRDS con architetture MLP. Sono stati variati in particolare i tassi di apprendimento di teacher e student, la dimensione del batch, la temperatura T e il coefficiente α della loss di distillazione, mantenendo fissi gli altri parametri di addestramento. Questa griglia ha permesso di esplorare configurazioni in grado di bilanciare in modi diversi l'apprendimento locale e il trasferimento di conoscenza.

Iperparametro	Valori
Teacher Learning Rate ($\eta_{teacher}$)	{0.01, 0.001}
Student Learning Rate ($\eta_{student}$)	{0.01, 0.001}
Teacher Epochs	150
Student Epochs	80
Teacher Gamma	0.95
Student Gamma	0.95
Teacher Step Size	15
Student Step Size	12
Temperature (T)	{3, 4, 5}
Alpha (α)	{0.5, 0.7, 0.9}
Batch Size	{32, 64}
Teacher Dropout	0.2
Student Dropout	0.2

Tab. 4.26: Spazio di ricerca iperparametrico considerato per IRDS con MLP.

Dallo spazio di ricerca definito emergono i risultati riportati in Tabella 4.27, che sintetizza per $K = 3, 5, 7$ client le configurazioni migliori secondo due criteri: (i) massimizzazione dell'accuratezza media sul proprio dominio (*client accuracy*) e (ii) massimizzazione della generalizzazione verso domini eterogenei (*cross-client accuracy*).

Client	Scenario	η_T	η_S	T	α	B	client	cross	P	R	F1
3	migliore locale	0.01	0.01	3	0.5	32	0.915	0.583	0.918	0.915	0.915
3	migliore cross	0.01	0.001	4	0.9	64	0.887	0.677	0.889	0.887	0.887
5	migliore locale	0.001	0.01	3	0.5	32	0.938	0.565	0.940	0.938	0.938
5	migliore cross	0.01	0.001	5	0.9	64	0.900	0.670	0.902	0.900	0.900
7	migliore locale	0.001	0.001	3	0.5	32	0.926	0.525	0.928	0.926	0.926
7	migliore cross	0.01	0.001	5	0.9	32	0.885	0.622	0.891	0.885	0.885

Tab. 4.27: Risultati migliori su IRDS (MLP) al variare del numero di client. Sono riportate le configurazioni che massimizzano rispettivamente l'accuratezza locale e la generalizzazione cross-client. Per ciascun modello sono indicate anche le metriche di precision, recall e F1. B indica la dimensione del batch.

L'analisi mette in evidenza un chiaro *trade-off* tra accuratezza locale e capacità di generalizzazione. Configurazioni con $\alpha = 0.5$ e temperature più basse ($T = 3$) tendono a massimizzare le prestazioni sul dominio nativo (fino a 0.938 con $K = 5$), ma mostrano limitata trasferibilità verso altri domini. Viceversa, combinazioni caratterizzate da $\alpha = 0.9$ e temperature più alte ($T = 4-5$) innalzano la cross accuracy fino a 0.677 con tre client e 0.670 con cinque, pur riducendo leggermente l'accuratezza locale.

Se si osservano le metriche oltre all'accuratezza, emerge un quadro coerente ma con valori leggermente più bassi rispetto a FEMNIST. Nei casi locali, precision, recall e F1 si mantengono in un intervallo compreso tra 0.91 e 0.94, segnalando che i modelli classificano in maniera bilanciata. Nelle configurazioni orientate alla generalizzazione, le tre metriche

restano comunque stabili e tra loro allineate, riducendosi solo di pochi punti percentuali insieme all'accuratezza. Questo andamento conferma che, su IRDS, la difficoltà principale non è data da squilibri nella classificazione, bensì dalla maggiore complessità intrinseca del dataset, che rende più arduo trasferire conoscenza tra domini eterogenei.

Un aspetto ulteriore riguarda l'effetto del coefficiente α , il grafico in Fig. 4.5 mostra l'andamento dell'accuratezza al variare di α su 7 client. L'aumento di α comporta una riduzione progressiva dell'accuratezza locale (da 0.926 a 0.903), ma parallelamente un miglioramento della capacità di generalizzazione, che passa da 0.582 a 0.622. Ciò conferma il ruolo cruciale della distillazione nel regolare il compromesso tra apprendimento specializzato e trasferimento di conoscenza: valori più alti di α aumentano il peso della loss di distillazione, incentivando il modello a incorporare meglio le informazioni condivise tra client.

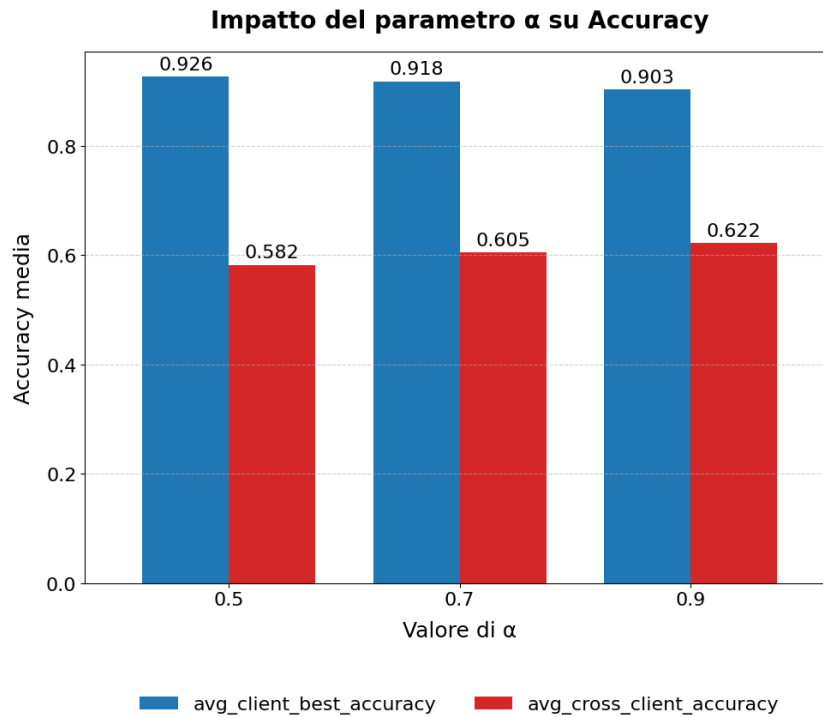
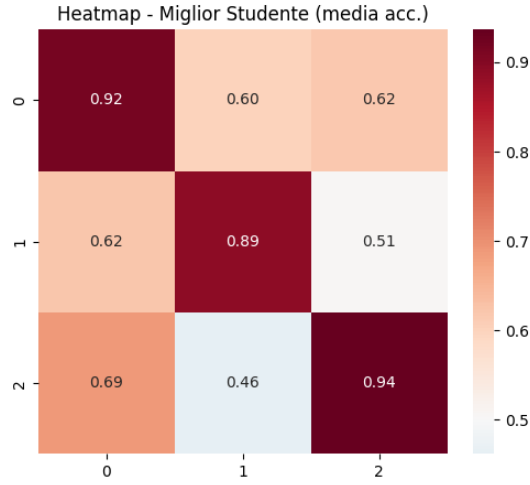
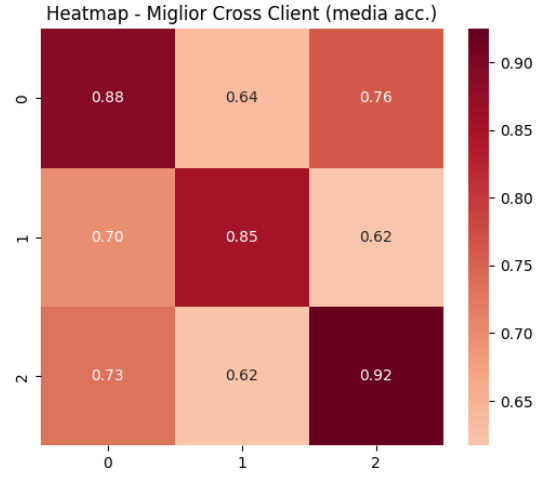


Fig. 4.5: Impatto del coefficiente α sull'accuratezza locale e cross-client su IRDS (MLP) su 7 client.

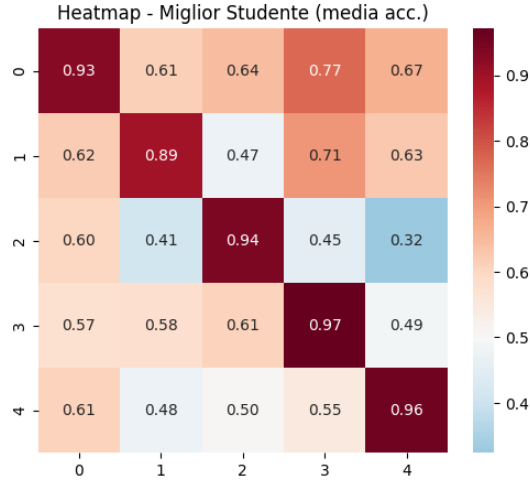
Infine, l'impatto della numerosità dei client non è trascurabile: con l'aumentare di K la generalizzazione si riduce sistematicamente, passando da 0.677 con tre client a 0.622 con sette. Le matrici di accuratezza in Fig. 4.6 rivelano un calo dei valori fuori diagonale con l'aumentare di K , indice della crescente difficoltà a trasferire conoscenza tra domini non-IID. Tuttavia, nelle configurazioni con α elevato e temperature più alte (a sinistra), si osserva un parziale recupero, che dimostra come la distillazione centralizzata, pur con i suoi limiti, sia in grado di attenuare la frammentazione del dataset e costituire una baseline robusta per la generalizzazione cross-client.



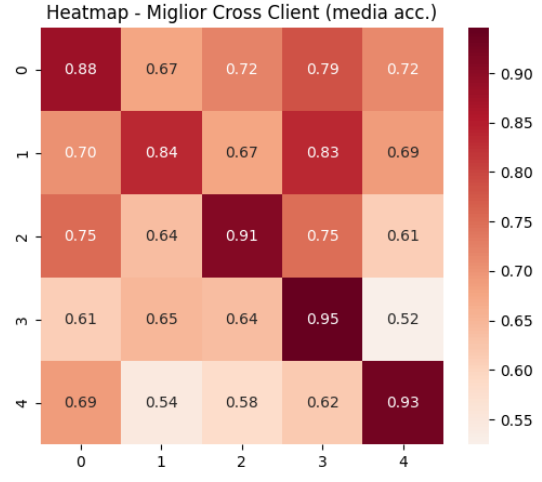
(a) Self-accuracy, 3 client



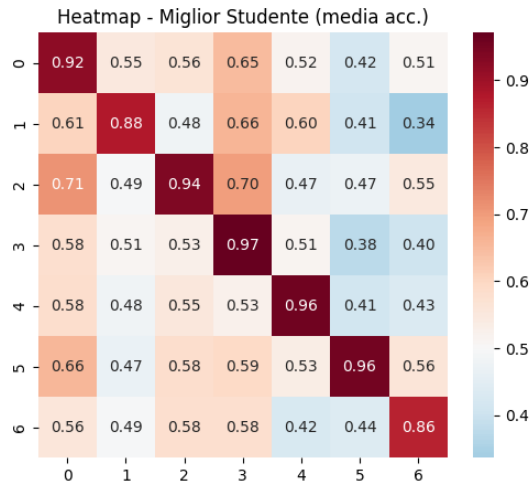
(b) Cross-accuracy, 3 client



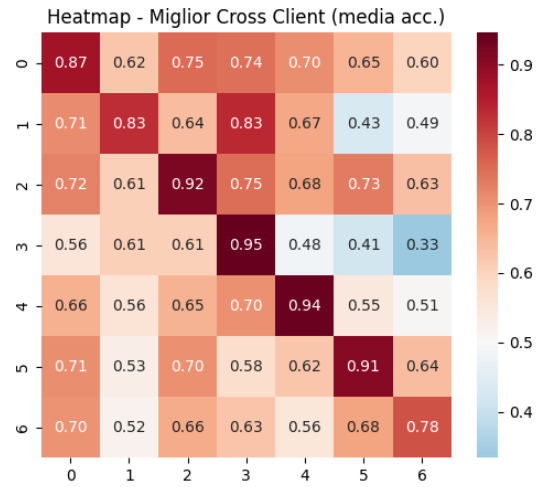
(c) Self-accuracy, 5 client



(d) Cross-accuracy, 5 client



(e) Self-accuracy, 7 client



(f) Cross-accuracy, 7 client

Fig. 4.6: Matrici di accuratezza self e cross-client su IRDS (MLP), al variare del numero di client. Le matrici a sinistra mostrano la performance sul dominio proprio (self), mentre quelle a destra evidenziano la generalizzazione verso domini eterogenei.

4.3.3 Risultati su IRDS con CI-VAE

La Tabella 4.28 riassume lo spazio di ricerca iperparametrico considerato per gli esperimenti con CI-VAE. Rispetto agli MLP, la ricerca ha incluso un numero maggiore di epoche e uno scheduler più aggressivo, in modo da stabilizzare il training di modelli generativi. I parametri principali variati sono i tassi di apprendimento, la temperatura T , la dimensione del batch e il coefficiente α della loss di distillazione.

Iperparametro	Valori
Teacher Learning Rate ($\eta_{teacher}$)	{0.005, 0.001}
Student Learning Rate ($\eta_{student}$)	{0.005, 0.001}
Teacher Epochs	200
Student Epochs	120
Teacher Gamma	{0.9, 0.95}
Student Gamma	{0.9, 0.95}
Teacher Step Size	{8, 10}
Student Step Size	{8, 10}
Temperature (T)	{4, 5}
Alpha (α)	{0.7, 0.9}
Batch Size	{32, 64}

Tab. 4.28: Spazio di ricerca iperparametrico considerato per IRDS con CI-VAE.

I migliori risultati ottenuti dalla grid search sono riportati in Tabella 4.29, che distingue le configurazioni ottimali per accuratezza locale e per generalizzazione cross-client.

Client	Scenario	η_T	η_S	T	α	B	Epoche (T/S)	γ_T/γ_S	Step (T/S)	Acc.	Cross	F1
3	locale	0.001	0.005	5	0.9	64	200/120	0.9/0.95	8/8	0.942	0.545	0.943
3	cross	0.001	0.001	4	0.9	64	200/120	0.9/0.95	10/8	0.928	0.565	0.929
5	locale	0.001	0.001	4	0.7	64	200/120	0.9/0.9	10/10	0.960	0.485	0.960
5	cross	0.001	0.001	4	0.9	32	200/120	0.95/0.95	8/10	0.940	0.526	0.940
7	locale	0.001	0.001	4	0.7	64	200/120	0.9/0.9	10/10	0.951	0.437	0.951
7	cross	0.001	0.001	5	0.9	64	200/120	0.95/0.95	8/10	0.918	0.479	0.918

Tab. 4.29: Risultati migliori su IRDS (CI-VAE) con distillazione centralizzata al variare del numero di client. Sono riportate le configurazioni che massimizzano rispettivamente l'accuratezza locale e la generalizzazione cross-client. La colonna *Epoche (T/S)* indica il numero di epoche per teacher e student; γ_T/γ_S i fattori di decay dello scheduler; Step (T/S) la dimensione degli step. B indica la dimensione del batch.

Le matrici di accuratezza self e cross, riportate in Fig. 4.7, mostrano in dettaglio le prestazioni per ciascun numero di client.

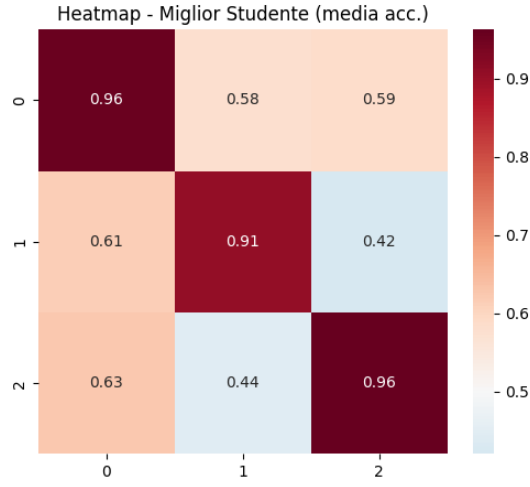
Dall'analisi dei risultati emerge con chiarezza che i modelli CI-VAE, pur garantendo prestazioni eccellenti sul proprio dominio di addestramento, mostrano una capacità di trasferimento notevolmente più limitata rispetto agli MLP. In particolare, l'accuratezza locale degli studenti si mantiene stabilmente tra il 94% e il 96%, valori persino superiori a quelli registrati dalle architetture fully connected. Questo comportamento può essere attribuito alla natura generativa del CI-VAE, che consente di modellare in profondità le distribuzioni dei dati locali e di adattarsi con grande precisione alle peculiarità di ciascun dominio.

Tuttavia, tale capacità di rappresentazione si traduce in una generalizzazione cross-client sensibilmente più debole. Le accuratèzze medie sui domini eterogenei restano infatti comprese tra 0.43 e 0.57, con un divario evidente rispetto agli MLP, che raggiungono valori

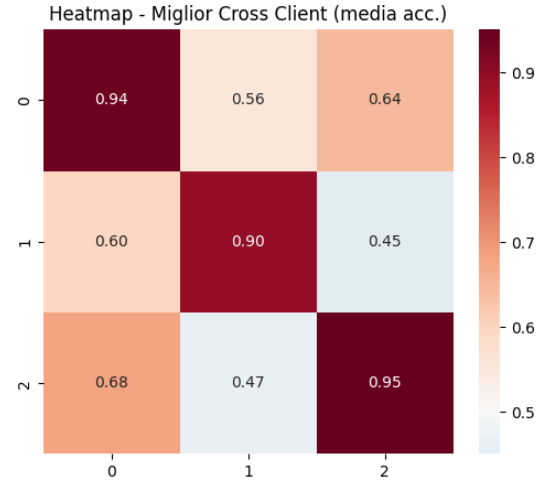
prossimi a 0.67. Le matrici di accuratezza confermano questa tendenza: la diagonale è caratterizzata da valori elevati, a testimonianza della forte specializzazione sul dominio proprio, mentre gli elementi fuori diagonale si mantengono sistematicamente bassi, segnalando una scarsa trasferibilità della conoscenza.

Anche l'analisi degli iperparametri conferma tale comportamento. Le configurazioni con $\alpha = 0.9$ e temperature medio-alte ($T = 4-5$) risultano le più favorevoli per incrementare la generalizzazione, in quanto spostano l'ottimizzazione verso una maggiore dipendenza dalle soft label del teacher. Tuttavia, il miglioramento rimane marginale e non consente di colmare il divario con gli MLP. Valori più bassi di α , al contrario, privilegiano l'accuratezza sul dominio locale, accentuando ulteriormente la specializzazione.

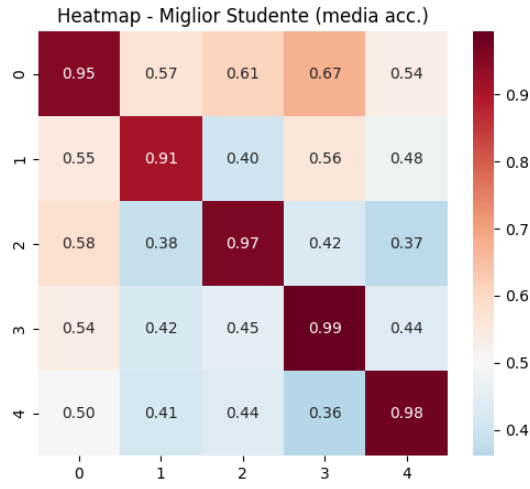
Infine, l'aumento del numero di client aggrava ulteriormente questa dinamica. Con sette client, la cross accuracy scende sotto lo 0.48, segno che l'eterogeneità dei dati rende più difficile per il CI-VAE sfruttare i vantaggi della distillazione centralizzata. In sintesi, mentre gli MLP traggono beneficio dalla distillazione in termini sia di accuratezza locale sia di generalizzazione, i CI-VAE appaiono più inclini a sovradattarsi ai dati propri del client. Questo risultato suggerisce che, per rendere la distillazione efficace anche in modelli generativi, sia necessario introdurre ulteriori meccanismi di regolarizzazione sullo spazio latente o tecniche di allineamento tra distribuzioni, al fine di mitigare l'over-specializzazione e rafforzare la capacità di trasferimento cross-client.



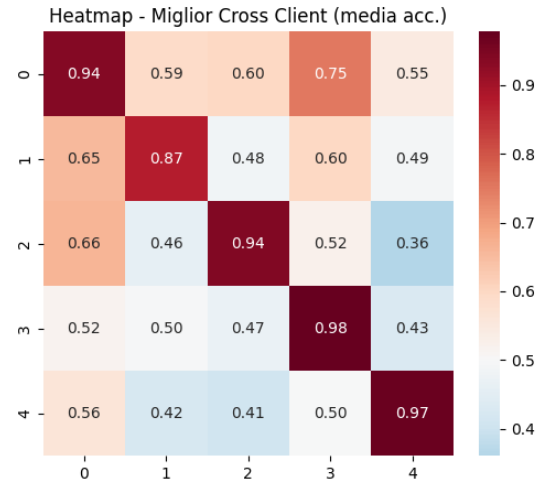
(a) Self-accuracy, 3 client



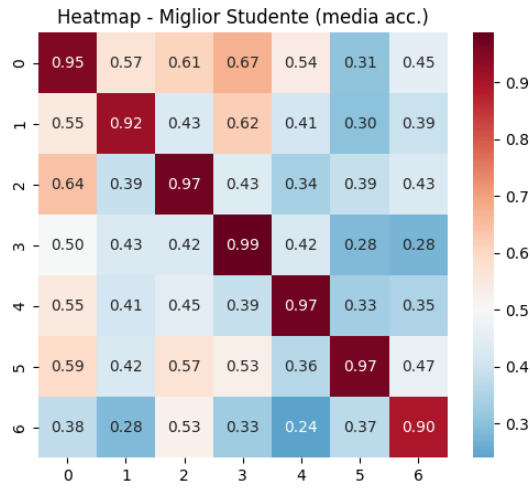
(b) Cross-accuracy, 3 client



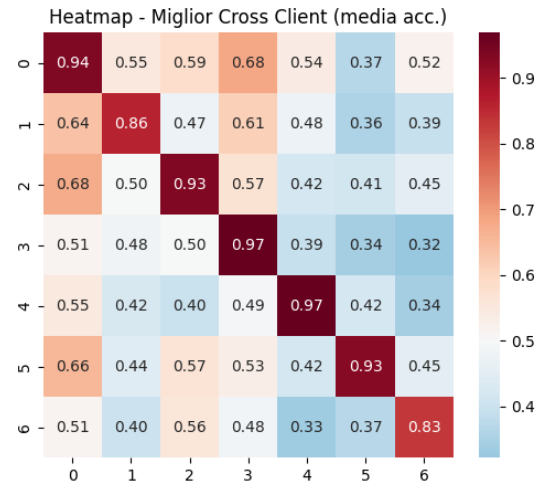
(c) Self-accuracy, 5 client



(d) Cross-accuracy, 5 client



(e) Self-accuracy, 7 client



(f) Cross-accuracy, 7 client

Fig. 4.7: Matrici di accuratezza self e cross-client su IRDS (CI-VAE) al variare del numero di client.

4.4 Paradigma 3: FedMD

4.4.1 Risultati su FEMNIST

Lo spazio iperparametrico esplorato per FedMD su FEMNIST è riportato in Tabella 4.30; include, oltre ai consueti parametri di ottimizzazione, anche il numero di round federati, fissato a 25 per tutti gli esperimenti, così da consentire un’alternanza sufficientemente lunga tra apprendimento locale e riallineamento tramite consenso.

Iperparametro	Valori
Learning Rate (η)	{0.0001, 0.0006, 0.001}
Temperature (T)	{3, 5}
Consensus Epochs	{3, 5, 10}
Local Epochs	{3, 5}
Alpha (α)	{1.0, 0.7, 0.5, 0.3}
Public Samples	200
Batch Size	128
Round federati	25

Tab. 4.30: Spazio iperparametrico esplorato per FedMD su FEMNIST.

I risultati sintetizzati in Tabella 4.31 distinguono, per $K \in \{3, 5, 7\}$, le configurazioni che massimizzano l’accuratezza locale (*self*) e quelle che privilegiano la generalizzazione (*cross*).

Client	Scenario	lr	T	cons. ep.	loc. ep.	α	self	cross	P	R	F1
3	locale	0.0006	3	3	5	1.0	0.981	0.828	0.984	0.988	0.984
3	cross	0.0006	3	10	3	0.7	0.941	0.914	0.940	0.940	0.932
5	locale	0.001	5	3	3	1.0	0.964	0.776	0.958	0.975	0.959
5	cross	0.001	3	10	3	0.7	0.911	0.871	0.904	0.906	0.886
7	locale	0.001	3	3	5	1.0	0.946	0.737	0.945	0.959	0.937
7	cross	0.001	5	10	5	0.7	0.912	0.849	0.899	0.883	0.871

Tab. 4.31: Risultati migliori di FedMD su FEMNIST al variare del numero di client. Sono riportate le configurazioni che massimizzano rispettivamente l’accuratezza locale e la generalizzazione cross-client.

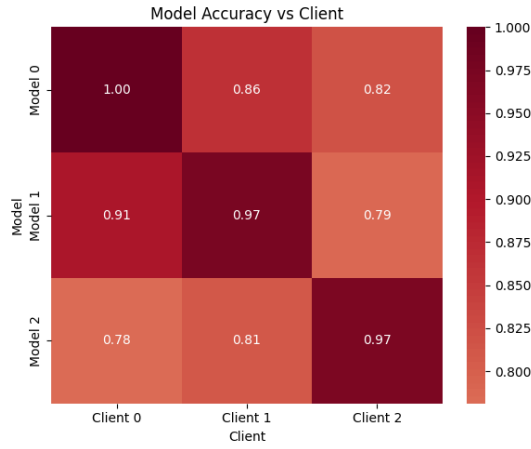
Dall’analisi emergono alcune considerazioni chiave. Le configurazioni con $\alpha = 1.0$, che corrispondono a dare peso massimo alla distillazione sul dataset sintetico, massimizzano la *self accuracy*, raggiungendo valori fino a 0.981 con tre client, ma mostrano una capacità di trasferimento limitata: la *cross accuracy* si colloca infatti tra 0.737 e 0.828. Questo comportamento può essere spiegato considerando la natura del dataset sintetico: pur fornendo un terreno comune di confronto, esso non riesce a rappresentare fedelmente la complessità e la variabilità dei dati reali di ciascun client. La distillazione, in questo scenario, agisce quindi come una sorta di regolarizzazione che rafforza le caratteristiche dominanti del dominio locale, senza riuscire a costruire rappresentazioni veramente condivise.

Al contrario, riducendo α a 0.7, e dunque bilanciando maggiormente la componente di consenso con la supervisione sintetica, si osserva una chiara inversione di tendenza. L’accuratezza locale diminuisce leggermente (circa 3–5 punti percentuali), ma la *cross accuracy* cresce sensibilmente, superando 0.91 con tre client e 0.87 con cinque. Ciò dimostra come il processo di consenso, se opportunamente bilanciato, permetta di attenuare

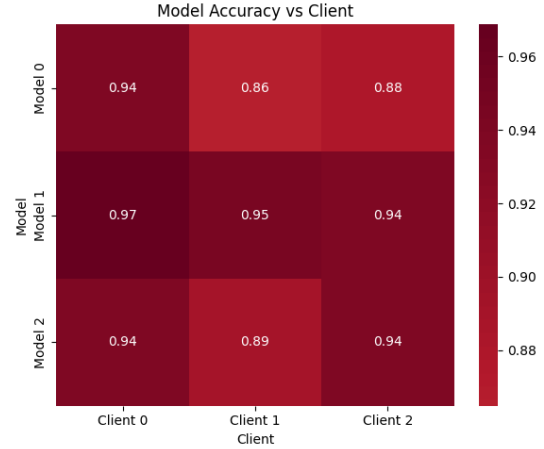
gli effetti dell'eterogeneità e favorisca l'emergere di una conoscenza comune tra i partecipanti. Non sorprende, inoltre, che le configurazioni ottimali per la generalizzazione richiedano più epoche di consenso: il processo iterativo di scambio e distillazione diventa infatti il principale motore della generalizzazione cross-client.

Le metriche di precision, recall e F1 si mantengono elevate in tutte le configurazioni, ma appaiono più bilanciate negli scenari orientati alla generalizzazione. Questo suggerisce che la riduzione della self accuracy non corrisponde a un degrado qualitativo del modello, bensì a una maggiore capacità di adattarsi a domini eterogenei, evitando l'overfitting sui dati locali.

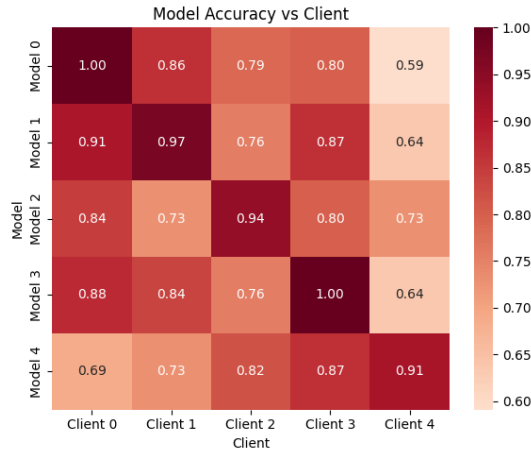
In sintesi, i risultati su FEMNIST mettono in luce il compromesso strutturale di FedMD: un peso eccessivo sulla distillazione sintetica ($\alpha = 1.0$) porta a massimizzare l'accuratezza locale ma penalizza la trasferibilità, mentre un bilanciamento più equilibrato ($\alpha = 0.7$) permette di migliorare sensibilmente la generalizzazione. Nel complesso, i risultati su FEMNIST suggeriscono che FedMD agisce da interpolatore tra apprendimento locale puro e consenso globale: la posizione lungo questo continuum è modulata da α e dal numero di round, con effetti chiaramente visibili tanto nelle medie aggregate (Tabella 4.31) quanto nella struttura delle heatmap self/cross (Fig. 4.8).



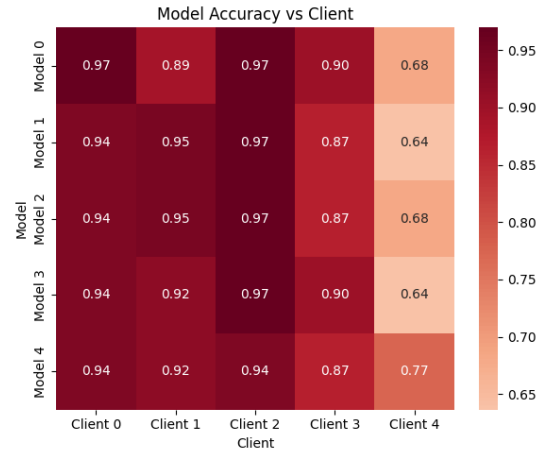
(a) Self-accuracy, 3 client



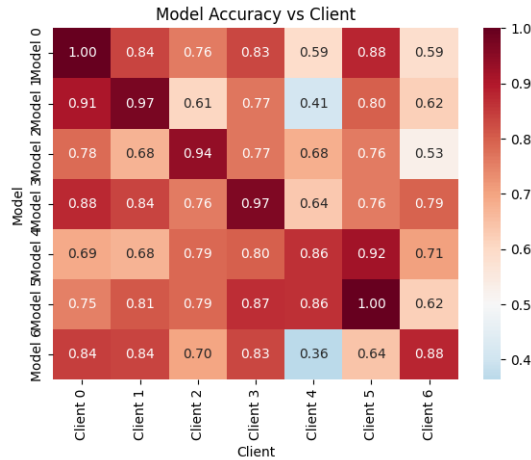
(b) Cross-accuracy, 3 client



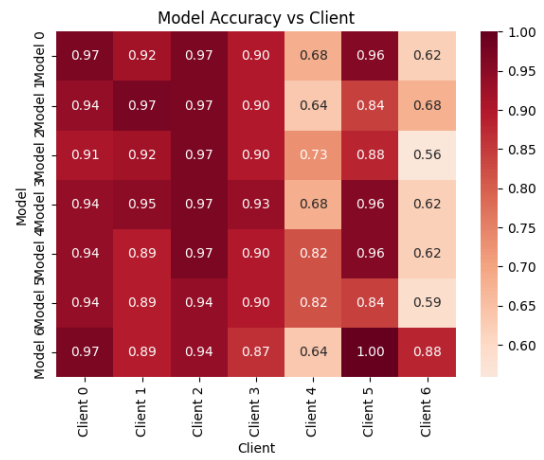
(c) Self-accuracy, 5 client



(d) Cross-accuracy, 5 client



(e) Self-accuracy, 7 client



(f) Cross-accuracy, 7 client

Fig. 4.8: Matrici di accuratezza self e cross-client di FedMD su FEMNIST, al variare del numero di client. Le matrici a sinistra mostrano le prestazioni sui dati propri, quelle a destra la capacità di generalizzazione.

4.4.2 Risultati su IRDS

La Tabella 4.32 riassume lo spazio di ricerca iperparametrico esplorato per FedMD su IRDS. Per coerenza con le sezioni precedenti, valori alti di α corrispondono quindi a una maggiore enfasi sul consenso, mentre valori bassi privilegiano la componente di cross-entropy sul dataset pubblico sintetico.

Iperparametro	Valori
Learning Rate (η)	{0.0001, 0.0006, 0.001}
Temperature (T)	{3, 5}
Consensus Epochs	{3, 5, 10}
Local Epochs	{3, 5}
Alpha (α)	{1.0, 0.7, 0.5, 0.3}
Public Samples	500
Batch Size	128
Round federati	40

Tab. 4.32: Iperparametri di FedMD su IRDS.

I risultati riportati in Tabella 4.33 mostrano un comportamento peculiare. Quando la distillazione è enfatizzata ($\alpha = 0.7$), i modelli raggiungono le massime prestazioni sul proprio dominio: la self accuracy tocca valori pari a 0.943 con cinque client e oltre 0.92 con sette client, accompagnata da valori di precision e recall superiori a 0.94. Tuttavia, questa scelta penalizza la generalizzazione: la cross accuracy rimane infatti contenuta tra 0.45 e 0.51. Al contrario, quando la cross-entropy sul dataset condiviso diventa più rilevante ($\alpha = 0.3$), la self accuracy cala sensibilmente (fino a 0.801 con cinque client), ma la cross accuracy cresce fino a 0.600 con tre client e 0.582 con cinque client. In altre parole, l'attenzione alla cross-entropy sul dataset pubblico, pur riducendo la specializzazione locale, consente ai modelli di sviluppare rappresentazioni più trasferibili tra domini eterogenei.

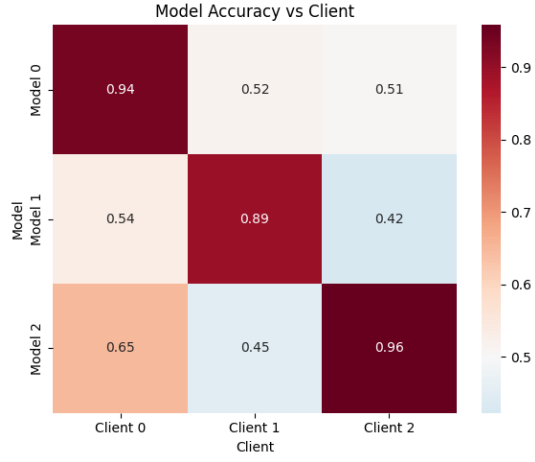
Client	Scenario	lr	T	cons. ep.	loc. ep.	α	self	cross	P	R	F1
3	locale	0.001	5	10	3	0.7	0.930	0.513	0.933	0.929	0.930
3	cross	0.0001	5	10	3	0.3	0.871	0.600	0.887	0.869	0.870
5	locale	0.0006	3	3	5	0.7	0.943	0.486	0.947	0.942	0.942
5	cross	0.0006	3	10	3	0.3	0.801	0.582	0.840	0.790	0.791
7	locale	0.001	5	3	3	0.7	0.921	0.450	0.925	0.919	0.920
7	cross	0.0001	5	10	3	0.3	0.805	0.515	0.844	0.802	0.797

Tab. 4.33: Risultati migliori di FedMD su IRDS al variare del numero di client. Sono riportate le configurazioni che massimizzano rispettivamente l'accuratezza locale e la generalizzazione cross-client.

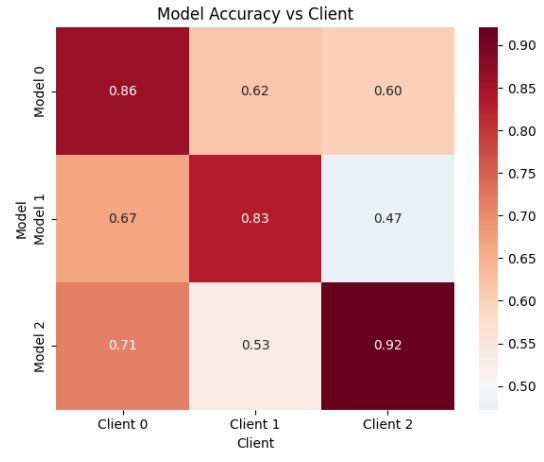
Le matrici di accuratezza in Fig. 4.9 confermano questo trade-off. Nei casi con α alto, le diagonali (*self*) sono molto elevate, a testimonianza di un forte adattamento al proprio dominio, mentre i valori off-diagonal restano bassi. Con α basso, invece, le diagonali si abbassano, ma i valori fuori diagonale aumentano, indicando un miglior trasferimento di conoscenza tra i client. Questo fenomeno diventa ancora più marcato con l'aumentare del numero di partecipanti: con $K = 7$, la cross accuracy resta comunque inferiore rispetto agli scenari meno frammentati, ma il miglioramento relativo rispetto alla configurazione puramente distillativa è evidente.

I risultati su IRDS mettono in luce un aspetto cruciale di FedMD: il ruolo del dataset

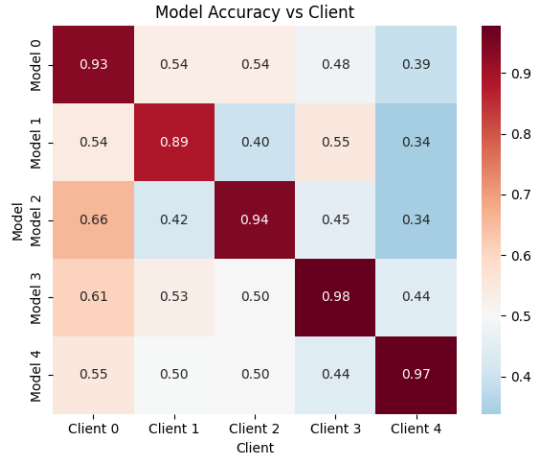
condiviso. Quando le distribuzioni locali sono altamente divergenti, la distillazione pura tende a rafforzare la specializzazione locale, mentre la componente supervisionata sul dataset comune diventa essenziale per favorire la generalizzazione cross-client. Questo evidenzia un limite strutturale di FedMD in scenari realistici: l'efficacia del consenso dipende in modo critico dalla qualità e dalla coerenza del dataset condiviso, e in sua assenza le strategie di bilanciamento tra distillazione e cross-entropy assumono un ruolo determinante.



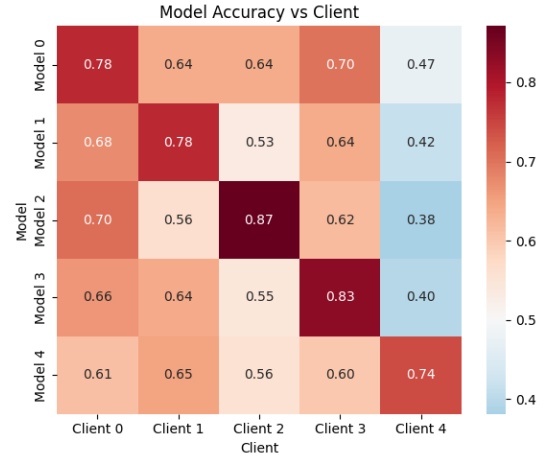
(a) Self-accuracy, 3 client



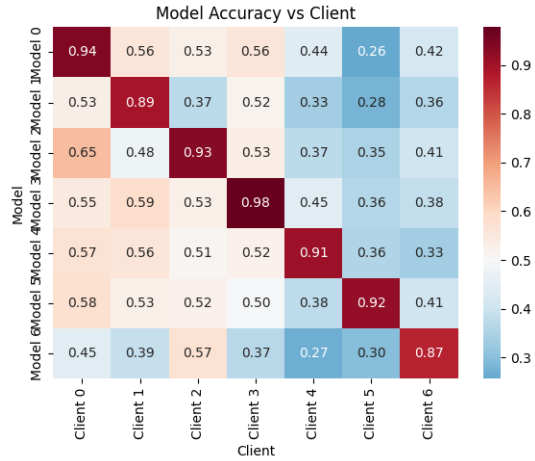
(b) Cross-accuracy, 3 client



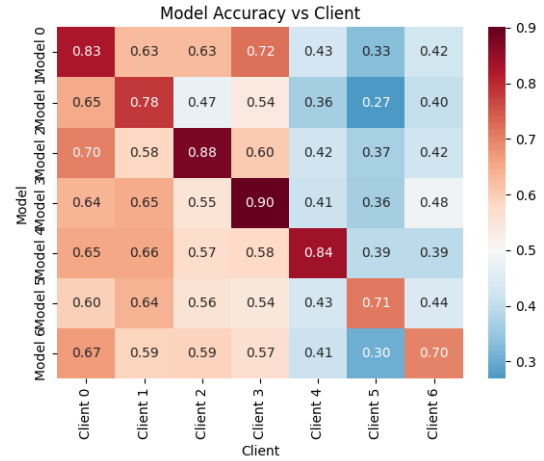
(c) Self-accuracy, 5 client



(d) Cross-accuracy, 5 client



(e) Self-accuracy, 7 client



(f) Cross-accuracy, 7 client

Fig. 4.9: Matrici di accuratezza self e cross-client per FedMD su IRDS, al variare del numero di client.

4.5 Paradigma 3: FedKD

4.5.1 Risultati su FEMNIST

Lo spazio iperparametrico esplorato per FedKD su FEMNIST è riportato in Tabella 4.34.

Iperparametro	Valori
Communication Rounds	70
Local Epochs	{3, 5, 10}
Distillation Epochs	{5, 10, 15}
Temperature (T)	{ 3.0, 5.0, 7.0}
Alpha (α)	{0.3, 0.5, 0.7, 0.9}
Base Learning Rate (η)	{0.001, 0.01, 0.1}
Synthetic Data Size	1000

Tab. 4.34: Iperparametri di FedKD su FEMNIST.

Gli esperimenti hanno mantenuto costante il numero di round federati (70), variando gli altri iperparametri. I risultati aggregati, riportati in Tabella 4.35, mostrano che FedKD è in grado di mantenere prestazioni elevate anche al crescere della frammentazione. Con tre client l'accuratezza globale raggiunge circa 0.95, con valori di precisione e F1 altrettanto alti. Con cinque client le prestazioni scendono leggermente sopra 0.90, mentre con sette si assestano intorno a 0.897. La riduzione è graduale e contenuta, segno che FedKD è in grado di mitigare gli effetti dell'eterogeneità non-IID senza però eliminarli del tutto.

Client	Round	lr	Loc. ep.	Dist. ep.	T	α	Acc.	Prec.	F1
3	70	0.01	10	5	5.0	0.5	0.951	0.960	0.951
5	70	0.01	10	10	3.0	0.7	0.909	0.915	0.910
7	70	0.01	5	10	3.0	0.9	0.897	0.898	0.897

Tab. 4.35: Risultati globali di FedKD su FEMNIST al variare del numero di client.

Un'analisi più granulare è riportata in Tabella 4.36. Con tre client le accuratze sono tutte molto elevate e bilanciate, con un client che raggiunge addirittura il 100%. Con cinque client il modello globale rimane stabile su tutti i partecipanti, con accuratze comprese tra 0.87 e 0.94. Con sette client, invece, emergono differenze più marcate: alcuni client superano il 95%, mentre altri scendono sotto 0.85. Questo evidenzia come il modello globale riesca a mantenere prestazioni competitive nella media, ma non garantisca uniformità assoluta tra domini eterogenei.

Num. client	C0	C1	C2	C3	C4	C5	C6
3	0.938	0.919	1.000	-	-	-	-
5	0.875	0.892	0.939	0.933	0.909	-	-
7	0.938	0.892	0.909	0.900	0.818	0.960	0.853

Tab. 4.36: Accuratezza per singolo client nei diversi scenari di FedKD su FEMNIST.

Dal punto di vista degli iperparametri, i risultati mostrano che valori intermedi di α (es. 0.5 con tre client) garantiscono un buon compromesso tra accuratezza locale e consolidamento della conoscenza globale, mentre scenari con maggiore eterogeneità richiedono un peso più alto alla distillazione (fino a $\alpha = 0.9$ con sette client) per stabilizzare il modello. Ciò indica che la distillazione diventa progressivamente più importante man mano che

cresce la frammentazione dei dati. La temperatura T influisce sull'equilibrio tra specializzazione e consenso. Con pochi client, un valore relativamente alto ($T = 5.0$) favorisce predizioni più morbide e una maggiore capacità di allineamento; con più partecipanti, valori più moderati ($T = 3.0$) risultano più efficaci per non appiattire eccessivamente le distribuzioni, preservando la diversità utile alla generalizzazione. Il numero di epoche di distillazione si rivela anch'esso determinante. Cinque epoche sono sufficienti per consolidare la conoscenza quando i client sono pochi e i dati relativamente omogenei, mentre scenari più complessi richiedono dieci epoche per ridurre la variabilità e garantire convergenza. La combinazione tra epoche locali e di distillazione appare quindi un parametro critico per bilanciare il contributo del segnale supervisionato e quello della distillazione.

I risultati su FEMNIST mostrano che FedKD è in grado di adattarsi alle diverse condizioni federate: con pochi client si può privilegiare il segnale supervisionato, mentre con molti è necessario aumentare il peso della distillazione e prolungare le fasi di consenso. La possibilità di modulare il comportamento tramite α , T ed epoche di distillazione rappresenta uno dei principali punti di forza del paradigma.

4.5.2 Risultati su IRDS con FedKD

Iperparametro	Valori
Synthetic Data Size	1000
Temperature (T)	{3.0, 5.0}
Distillation Epochs	{5, 10}
Local Epochs	{3, 5, 10}
Base Learning Rate (η)	{0.001, 0.01, 0.1}
Alpha (α)	{0.3, 0.7, 1.0}
Communication Rounds	70

Tab. 4.37: Iperparametri di FedKD su IRDS.

La Tabella 4.37 riassume lo spazio iperparametrico esplorato per valutare FedKD sul dataset IRDS. Il numero di round federati è stato mantenuto costante a 70, mentre sono stati variati gli iperparametri chiave: epoche locali e di distillazione, temperatura T , coefficiente α e tasso di apprendimento. Il dataset sintetico è stato fissato a 1000 campioni per garantire un terreno comune per la distillazione.

Client	Round	lr	Loc. ep.	Dist. ep.	T	α'	Acc.	Prec.	F1
3	70	0.001	10	10	5.0	0.7	0.650	0.652	0.650
5	70	0.001	5	10	3.0	1.0	0.640	0.640	0.639
7	70	0.001	10	5	3.0	1.0	0.590	0.591	0.589

Tab. 4.38: Risultati globali di FedKD su IRDS al variare del numero di client.

La Tabella 4.38 mostra le metriche globali di accuratezza, precisione e F1-score nei tre scenari considerati. Con tre client, FedKD raggiunge un'accuratezza di circa 0.65, valore che si riduce progressivamente a 0.64 con cinque client e 0.59 con sette. Le metriche di precisione e F1 risultano sostanzialmente allineate, segnalando che la riduzione delle performance non è dovuta a squilibri di classe o falsi positivi/negativi, ma a una difficoltà complessiva di generalizzazione.

Num. client	C0	C1	C2	C3	C4	C5	C6
3	0.653	0.688	0.611	–	–	–	–
5	0.652	0.649	0.556	0.805	0.596	–	–
7	0.644	0.603	0.560	0.744	0.547	0.496	0.555

Tab. 4.39: Accuratezza per singolo client nei diversi scenari di FedKD su IRDS. Nei casi con meno di 7 partecipanti, le colonne rimanenti sono riempite con trattini.

Un’analisi più dettagliata è fornita dalla Tabella 4.39, che riporta l’accuratezza di ciascun client. Con tre partecipanti, le prestazioni rimangono relativamente bilanciate, oscillando tra 0.61 e 0.69. Con cinque client emergono squilibri più marcati: alcuni modelli raggiungono prestazioni elevate (fino a 0.80), mentre altri non superano lo 0.55. Con sette partecipanti, la forbice si amplia ulteriormente: il miglior client arriva a 0.74, mentre il peggiore scende a 0.50. Questa variabilità segnala che la distillazione non riesce a compensare pienamente l’eterogeneità delle distribuzioni locali, lasciando che i client con domini più atipici rimangano penalizzati.

Dal punto di vista degli iperparametri, emerge un quadro chiaro. Con tre client, il valore ottimale di α è intermedio (0.7), segnalando che un contributo significativo della distillazione aiuta a consolidare la conoscenza globale quando le distribuzioni locali sono ancora relativamente compatibili. Con cinque e sette partecipanti, invece, le configurazioni migliori corrispondono a valori estremi ($\alpha = 1.0$), cioè a scenari in cui la distillazione domina completamente sulla cross-entropy supervisionata.

Anche la temperatura T gioca un ruolo importante. Con tre client, valori più alti ($T = 5.0$) risultano efficaci perché producono distribuzioni morbide, facilitando il processo di distillazione. Con cinque e sette partecipanti, invece, valori moderati ($T = 3.0$) sono preferibili: un’eccessiva morbidezza avrebbe infatti l’effetto di livellare troppo le predizioni, riducendo la capacità del modello globale di distinguere tra classi in domini eterogenei.

Infine, il numero di epoche di distillazione si adatta al grado di complessità del sistema. Con tre client, dieci epoche risultano sufficienti per consolidare la conoscenza condivisa. Con cinque client si conferma la necessità di dieci epoche, mentre con sette si riduce a cinque, probabilmente per evitare che un processo troppo lungo amplifichi il rumore proveniente da client molto divergenti.

In sintesi, i risultati mostrano che su IRDS FedKD riesce a mantenere prestazioni moderate anche con molti client, ma il contributo della distillazione deve essere spinto al massimo ($\alpha \rightarrow 1.0$) per controbilanciare la forte eterogeneità delle distribuzioni locali. Questo comportamento evidenzia come la qualità del consenso giochi un ruolo centrale: quando i client condividono predizioni altamente divergenti, solo un’enfasi marcata sulla distillazione permette al modello globale di mantenere stabilità, sebbene a un livello di accuratezza inferiore rispetto a dataset più omogenei.

4.5.3 Confronto tra FedMD e FedKD

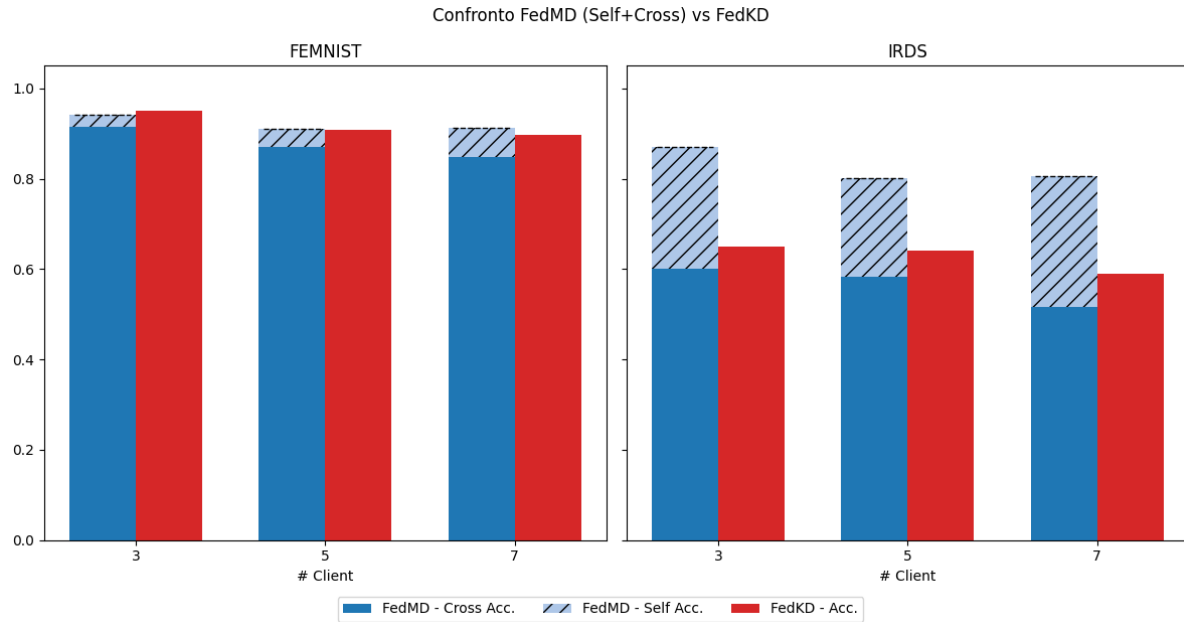


Fig. 4.10: Confronto tra FedMD (Accuracy e Cross Accuracy) e FedKD (Accuracy) su FEMNIST e IRDS.

Il confronto tra i due approcci di distillazione federata, FedMD e FedKD, mette in luce differenze sostanziali sia nella struttura algoritmica che nelle prestazioni ottenute. FedMD, che mantiene un modello locale per ciascun client, mostra risultati competitivi in termini di accuratezza e F1-score sui dati propri, con performance elevate e stabili all'aumentare del numero di client. FedKD, al contrario, si basa su un unico modello globale appreso centralmente e mostra risultati più modesti: su FEMNIST l'accuratezza resta accettabile, ma inferiore rispetto a FedMD, mentre su IRDS le prestazioni calano sensibilmente, indicando una maggiore difficoltà nel trattare la variabilità del dominio.

Un aspetto interessante riguarda tuttavia la generalizzazione: sebbene FedKD non consenta una valutazione esplicita cross-client, le sue prestazioni globali si avvicinano alla cross accuracy ottenuta da FedMD. In altri termini, anche se i modelli FedMD eccellono sul proprio dominio, la loro capacità di trasferimento verso domini differenti è limitata e comparabile con quella del modello globale appreso da FedKD.

Entrambe le tecniche, comunque, risultano inferiori rispetto agli approcci di federated learning classico, come FedAvg e FedProx. Questi ultimi, infatti, producono modelli globali che, pur non ottimizzati per ogni singolo dominio, raggiungono performance più elevate sia localmente che in termini di generalizzazione. Di conseguenza, la distillazione federata — pur offrendo vantaggi in termini di privacy e modularità — mostra limiti concreti sul piano della trasferibilità, che richiedono ulteriori meccanismi di regolarizzazione o allineamento tra domini per poter competere con le strategie tradizionali.

Tecnica	# Client	Accuracy	Cross Acc.	Precision	F1-score
FedMD	3	0.941	0.914	0.940	0.932
FedKD	3	0.951	—	0.960	0.951
FedMD	5	0.911	0.871	0.904	0.886
FedKD	5	0.909	—	0.915	0.910
FedMD	7	0.912	0.849	0.899	0.871
FedKD	7	0.897	—	0.898	0.897

Tab. 4.40: Confronto tra FedMD (configurazioni ottimizzate per la generalizzazione cross-client) e FedKD su FEMNIST. Per FedKD la metrica cross non è definita.

Tecnica	# Client	Accuracy	Cross Acc.	Precision	F1-score
FedMD	3	0.871	0.600	0.887	0.870
FedKD	3	0.650	—	0.652	0.650
FedMD	5	0.801	0.582	0.840	0.791
FedKD	5	0.640	—	0.640	0.639
FedMD	7	0.805	0.515	0.844	0.797
FedKD	7	0.590	—	0.591	0.589

Tab. 4.41: Confronto tra FedMD (modelli ottimizzati per la generalizzazione cross-client) e FedKD su IRDS. La colonna **Cross Acc.** è assente per FedKD, che non produce modelli individuali.

4.6 Confronto tra paradigmi

KD Centralizzato	Client 0	Client 1	Client 2	Client 3	Client 4	Client 5	Client 6
Model 0	1.000	0.941	0.733	0.857	0.727	0.583	0.813
Model 1	0.867	1.000	0.667	0.857	0.364	0.333	0.750
Model 2	0.867	0.765	0.933	0.857	0.909	0.500	0.813
Model 3	0.733	0.824	0.667	1.000	0.909	0.500	0.688
Model 4	0.533	0.706	0.667	0.786	1.000	0.583	0.438
Model 5	0.733	0.706	0.667	0.857	0.909	0.750	0.375
Model 6	0.867	0.824	0.600	0.857	0.818	0.333	0.938
FedMD							
Model 0	0.969	0.919	0.970	0.900	0.682	0.960	0.618
Model 1	0.938	0.973	0.970	0.900	0.636	0.840	0.676
Model 2	0.906	0.919	0.970	0.900	0.727	0.880	0.559
Model 3	0.938	0.946	0.970	0.933	0.682	0.960	0.618
Model 4	0.938	0.892	0.970	0.900	0.818	0.960	0.618
Model 5	0.938	0.892	0.939	0.900	0.818	0.840	0.588
Model 6	0.969	0.892	0.939	0.867	0.636	1.000	0.882
FedProx (globale)	1.000	0.967	1.000	0.853	0.955	1.000	0.973

Tab. 4.42: Confronto delle performance di KD centralizzato (MLP), FedMD e FedProx su ciascun client nel dataset FEMNIST. I valori in grassetto evidenziano la performance del modello sul proprio dominio (diagonale).

La Tabella 4.42, nello scenario con $K = 7$ client, consente un confronto diretto tra le tre strategie analizzate: FedProx, Knowledge Distillation centralizzata e FedMD. Dal quadro emerge con chiarezza la superiorità di FedProx, che garantisce prestazioni uniformi e prossime alla perfezione su tutti i domini, dimostrando l’efficacia del termine prossimale nel contenere il *client drift* e armonizzare rappresentazioni latenti anche in presenza di scritture eterogenee.

I paradigmi basati su distillazione mostrano invece dinamiche più complesse. La Knowledge Distillation centralizzata raggiunge una *self accuracy* media molto elevata (0.946), ma la *cross accuracy* cala sensibilmente (0.709), rivelando che i modelli studente si specializzano fortemente sul dominio di origine e generalizzano poco verso gli altri. Questo comportamento è evidente osservando le accuratezze off-diagonal della Tabella 4.42: alcuni client, come *Client 5* e *Client 6*, risultano particolarmente problematici, con valori spesso inferiori a 0.60. Si tratta di client caratterizzati da distribuzioni di classi più sbilanciati (cfr. Fig. 3.1), che fungono da veri e propri “outlier statistici”. I modelli distillati su questi domini raggiungono alte prestazioni locali (accuratezze diagonali perfette o vicine al 100%), ma la conoscenza appresa si trasferisce con difficoltà, riducendo la coerenza globale.

FedMD si colloca in una posizione intermedia. Nonostante utilizzi un dataset sintetico come base di consenso — intrinsecamente meno rappresentativo della complessità reale — riesce a ottenere una *cross accuracy* media superiore (0.849), pur a fronte di una *self accuracy* leggermente inferiore (0.912). Il dataset sintetico, costruito a partire da statistiche aggregate, agisce infatti come un livellatore statistico: pur riducendo l’accuratezza locale, attenua le differenze più marcate tra domini, portando a una distribuzione degli errori meno polarizzata. In particolare, i client più atipici (es. *Client 5*) rimangono

difficili da generalizzare, ma l'asimmetria tra domini si riduce rispetto alla KD centralizzata, con off-diagonal più uniformi e valori cross-client più alti. Questo evidenzia un compromesso strutturale: la qualità artificiale del dataset limita la capacità di catturare finzze stilistiche e riduce la specializzazione locale, ma facilita la costruzione di un terreno comune che rende il consenso più trasferibile.

Su FEMNIST confronto mostra quindi tre approcci distinti: FedProx massimizza stabilità e uniformità, riducendo quasi del tutto le differenze tra client (inclusi i più difficili); la KD centralizzata eccelle nella specializzazione locale ma penalizza la trasferibilità, con errori concentrati su client outlier; FedMD sacrifica parte della performance locale, ma migliora la coerenza cross-client grazie al dataset sintetico.

KD Centralizzato	Client 0	Client 1	Client 2	Client 3	Client 4	Client 5	Client 6
Model 0	0.873	0.625	0.754	0.744	0.705	0.652	0.597
Model 1	0.711	0.831	0.637	0.834	0.672	0.430	0.494
Model 2	0.723	0.605	0.919	0.748	0.678	0.731	0.634
Model 3	0.558	0.605	0.605	0.946	0.484	0.415	0.334
Model 4	0.658	0.560	0.655	0.703	0.939	0.548	0.506
Model 5	0.706	0.531	0.704	0.578	0.623	0.906	0.639
Model 6	0.702	0.518	0.665	0.626	0.555	0.679	0.781
FedMD							
Model 0	0.826	0.629	0.627	0.719	0.426	0.328	0.423
Model 1	0.653	0.783	0.474	0.540	0.365	0.269	0.403
Model 2	0.703	0.584	0.881	0.604	0.420	0.365	0.420
Model 3	0.635	0.649	0.554	0.901	0.414	0.363	0.482
Model 4	0.655	0.655	0.573	0.581	0.838	0.393	0.391
Model 5	0.595	0.638	0.560	0.543	0.430	0.709	0.440
Model 6	0.669	0.588	0.591	0.565	0.414	0.304	0.700
FedProx (globale)	0.794	0.913	0.846	0.885	0.765	0.916	0.917

Tab. 4.43: Confronto delle performance di KD centralizzato (MLP), FedMD e FedProx su ciascun client nel dataset IRDS. I valori in grassetto evidenziano la performance del modello sul proprio dominio (diagonale).

La Tabella 4.43 sintetizza i risultati su IRDS con $K = 7$ client, mostrando un quadro profondamente diverso rispetto a FEMNIST. Il confronto tra paradigmi rivela infatti che la complessità dei dati clinici, caratterizzati da forti variazioni inter-individuali, amplifica le difficoltà di generalizzazione e mette in crisi in particolare i metodi basati su distillazione.

La Knowledge Distillation centralizzata ottiene una *self accuracy* media di 0.885, con valori diagonali elevati (spesso superiori a 0.90), a testimonianza della capacità dei modelli studente di specializzarsi sul proprio paziente. Tuttavia, la *cross accuracy* crolla a 0.622, segnalando che la conoscenza rimane fortemente vincolata al dominio di origine e non si trasferisce in modo efficace verso altri pazienti. L'analisi delle celle off-diagonal mostra come alcuni client (ad esempio *Client 5* e *Client 6*) fungano da veri e propri *outlier*, ricevendo valori di accuratezza cross-client molto bassi: in questi casi i modelli addestrati altrove non riescono ad adattarsi alle peculiarità motorie del paziente target. La KD centralizzata produce dunque modelli eccellenti in termini locali, ma incapaci di costituire una base di consenso realmente condivisa.

FedMD, che su FEMNIST aveva dimostrato una capacità superiore di mediazione cross-

client, su IRDS risulta sensibilmente più fragile. Le performance calano sia in termini di *self accuracy* (0.805) sia di *cross accuracy* (0.515), mostrando che il dataset sintetico non riesce a svolgere il ruolo di terreno comune efficace: nel caso di movimenti riabilitativi gli embedding di MobileNetV2 presentano strutture più complesse, difficili da rappresentare con un modello statistico semplice. Il risultato è che il dataset sintetico, pur attenuando parzialmente le differenze estreme, introduce rumore e impoverisce la rappresentazione condivisa, con un effetto penalizzante sia sulla specializzazione locale che sulla trasferibilità. Interessante notare che i client già problematici nella KD centralizzata (come il 5 e il 6) rimangono i più difficili da generalizzare anche in FedMD, ma l'effetto si estende a un numero maggiore di domini, con una dispersione delle prestazioni più marcata.

Al contrario, FedProx si conferma l'approccio più robusto. Il modello globale raggiunge valori stabili e vicini alla perfezione su tutti i client, con accuratezze superiori a 0.90 in quasi tutti i casi. L'aggiunta del termine prossimale limita efficacemente il fenomeno del *client drift*, che nei dati clinici emerge con particolare intensità: ogni paziente costituisce infatti un dominio distinto, con distribuzioni locali scarsamente sovrapponibili. FedProx agisce come meccanismo di armonizzazione, costringendo i modelli locali a non deviare eccessivamente dal modello globale e garantendo così che anche i client più problematici non penalizzino eccessivamente la coerenza del sistema.

Nel complesso, il confronto tra paradigmi su IRDS porta a tre osservazioni principali. Primo, i metodi di distillazione soffrono nei domini clinici perché la conoscenza appresa è fortemente soggetto-specifica e difficilmente trasferibile, specialmente quando il dataset di consenso è sintetico e semplificato. Secondo, la distillazione centralizzata massimizza la specializzazione locale ma fallisce nella generalizzazione, mentre FedMD, pur introducendo un meccanismo di mediazione, non riesce a compensare la complessità intrinseca degli embedding clinici. Terzo, FedProx emerge come la strategia più equilibrata, dimostrando che, in scenari reali e non-IID come quello clinico, la regolarizzazione federata è più efficace della distillazione per costruire un modello globale stabile e trasversale.

Capitolo 5

Conclusioni

Questo lavoro si è inserito nel crescente ambito dell'apprendimento federato applicato a scenari clinici, con l'obiettivo di esplorare l'efficacia di diverse strategie collaborative per l'addestramento di modelli predittivi in condizioni di distribuzione non omogenea dei dati. In particolare, il focus è stato posto sulla classificazione automatica di esercizi fisioterapici, un compito rappresentativo delle sfide tipiche dell'ambito riabilitativo: dati eterogenei, sensibili, frammentati tra soggetti e strutture diverse.

Per affrontare questo problema, la tesi ha messo a confronto due approcci principali: le strategie classiche di federated learning e i paradigmi alternativi di distillazione della conoscenza, valutandone l'efficacia su due dataset complementari. FEMNIST, benchmark consolidato, ha offerto un riferimento controllato e riproducibile; IRDS, invece, ha permesso di testare le tecniche in un contesto clinico realistico, caratterizzato da alta variabilità soggetto-specifica e da distribuzioni marcatamente non-IID.

5.1 Risultati principali

I risultati sperimentali hanno mostrato con chiarezza che le strategie di Federated Learning classico, in particolare FedAvg e FedProx, si sono dimostrate robuste ed efficaci anche in presenza di forte eterogeneità statistica. FedAvg, grazie alla sua semplicità di aggregazione basata sulla media pesata, e FedProx, con l'aggiunta di un termine prossimale volto a ridurre il *client drift*, hanno mantenuto performance elevate e stabili, con valori di accuratezza e F1-score superiori al 90% su entrambi i dataset. La Tabella 5.1 riassume i risultati ottenuti su 3 client per FEMNIST e IRDS.

Tab. 5.1: Prestazioni su 3 client delle strategie classiche su FEMNIST e IRDS

Tecnica	Dataset	Accuracy	F1-score
FedAvg	FEMNIST	0.990	0.989
FedProx	FEMNIST	1.000	1.000
FedAdam	FEMNIST	0.659	0.602
FedAdagrad	FEMNIST	0.721	0.620
FedAvg	IRDS	0.903	0.902
FedProx	IRDS	0.912	0.912
FedAdam	IRDS	0.640	0.602
FedAdagrad	IRDS	0.487	0.615

Le strategie basate su ottimizzatori adattivi, come FedAdam e FedAdagrad, hanno invece evidenziato prestazioni inferiori e meno stabili. Tale risultato è attribuibile non solo alle dinamiche interne di questi algoritmi, ma soprattutto alla loro interazione con l'architettura impiegata, il CI-VAE. Questo modello, che combina obiettivi generativi e discriminativi, tende a concentrare l'ottimizzazione iniziale sulla componente generativa (ricostruzione), accentuando la divergenza tra i client. In questo contesto, gli ottimizzatori adattivi non si sono dimostrati in grado di mitigare il disallineamento, finendo anzi per amplificarlo.

La valutazione dei paradigmi distillativi ha evidenziato risultati più eterogenei. La distillazione centralizzata ha mostrato ottime prestazioni sui dati locali ma ha sofferto in termini di generalizzazione cross-client. I modelli studente, infatti, si sono dimostrati fortemente specializzati sul proprio dominio, senza riuscire a trasferire efficacemente la conoscenza ad altri domini, soprattutto nei casi con maggiore eterogeneità.

FedMD, che si basa sulla creazione di un dataset sintetico condiviso a partire da statistiche aggregate locali, ha raggiunto un miglior equilibrio tra accuratezza locale e trasferibilità. Su FEMNIST, il modello ha mantenuto buone prestazioni sia localmente sia in termini di accuratezza su dati di altri client. Tuttavia, su IRDS — caratterizzato da dati reali e complessi — la capacità di generalizzazione si è ridotta, segnalando i limiti del dataset di consenso in scenari clinici complessi.

FedKD ha permesso la costruzione di un modello globale tramite distillazione iterativa, aggregando predizioni multiple. Anche in questo caso, su FEMNIST si sono osservate buone prestazioni, mentre su IRDS l'efficacia è risultata significativamente compromessa dalla qualità del dataset.

La Tabella 5.2 riporta i risultati relativi alle tecniche distillative. Per ciascuna strategia, sono indicati i valori di accuratezza locale, accuratezza cross-client (Cross Acc) e F1-score riferiti al miglior modello selezionato in base alla massima cross-accuracy ottenuta su 3 client.

Tab. 5.2: Prestazioni su 3 client delle tecniche di distillazione su FEMNIST e IRDS (modello selezionato per massima Cross Accuracy)

Tecnica	Dataset	Accuracy	Cross Acc	F1-score
KD Centralizzato	FEMNIST	1.000	0.773	1.000
FedMD	FEMNIST	0.941	0.914	0.932
FedKD	FEMNIST	0.951	–	0.951
KD Centralizzato	IRDS	0.915	0.583	0.915
FedMD	IRDS	0.871	0.600	0.870
FedKD	IRDS	0.650	–	0.650

Questi risultati mettono in evidenza il ruolo centrale della qualità e della rappresentatività del dataset di consenso nella distillazione. In scenari relativamente semplici, come FEMNIST, un dataset sintetico generato a partire da statistiche aggregate è risultato sufficiente a garantire un buon livello di consenso. In contesti clinici come IRDS, invece, tale semplificazione si è rivelata un ostacolo alla generalizzazione. I modelli hanno appreso in modo efficace le peculiarità locali, ma non sono riusciti a trasferire tali conoscenze tra domini differenti, generando un marcato divario tra specializzazione e generalizzazione.

Nel complesso, i risultati suggeriscono che, affinché la distillazione possa rappresentare un’alternativa realmente efficace in contesti complessi, è necessario investire nella costruzione di dataset sintetici più espressivi, oppure adottare strategie ibride che integrino regolarizzazione federata e meccanismi distillativi. In questo modo sarà possibile combinare la stabilità dell’aggiornamento federato con la modularità e leggerezza proprie della distillazione, rendendo l’approccio sostenibile anche in ambiti ad alta sensibilità come quello sanitario.

5.2 Implicazioni e Sviluppi Futuri

I risultati ottenuti mostrano che, nei contesti clinici non-IID, le strategie classiche di federated learning continuano a offrire una combinazione convincente di stabilità, generalizzazione e semplicità implementativa. FedAvg e FedProx, pur nella loro essenzialità, si sono adattate bene alla variabilità intrinseca di IRDS, dimostrando che l’adozione di tecniche consolidate non esclude prestazioni competitive anche in scenari complessi. La distillazione, invece, pur rappresentando un’alternativa concettualmente elegante — capace di ridurre il carico computazionale e migliorare la modularità — si è dimostrata sensibile alla qualità e alla struttura dei dati sintetici utilizzati.

In applicazioni come la riabilitazione motoria, dove i dati sono raccolti in modo distribuito e disomogeneo tra strutture, la costruzione di un modello globale deve necessariamente tenere conto delle dinamiche locali. I risultati suggeriscono che, in assenza di un dataset di consenso sufficientemente rappresentativo, la distillazione tende a specializzarsi troppo, perdendo la capacità di trasferire la conoscenza tra pazienti o centri differenti. Questo implica che le soluzioni distillative proposte, nella loro forma statica e sintetica, sono ancora lontane dall’essere pronte per l’uso clinico diretto, se non in domini con distribuzioni più regolari o controllate.

Un altro punto di rilievo emerso riguarda il ruolo dell’architettura. L’impiego del CI-VAE nelle strategie federate ha consentito una rappresentazione latente ricca, ma ha anche amplificato il rischio di divergenza tra client nelle fasi iniziali dell’addestramento. Questo comportamento ha messo in difficoltà gli ottimizzatori adattivi lato server, che non sono riusciti a compensare lo squilibrio generato dalla componente generativa. Il fatto che strategie semplici abbiano performato meglio di quelle più raffinate non deve essere interpretato come un ritorno alla semplicità acritica, ma come un richiamo alla necessità di allineare le scelte architetturali con il comportamento dinamico dell’addestramento federato.

Questo lavoro presenta alcune limitazioni che è importante considerare. L’analisi è stata svolta su due dataset rappresentativi, ma comunque circoscritti. Anche il numero di client è stato mantenuto volutamente contenuto, in funzione delle risorse computazionali disponibili e della necessità di controllare la complessità sperimentale. Per quanto riguarda le tecniche di distillazione, si è scelto di utilizzare modelli MLP semplici, privilegiando la trasparenza metodologica e la confrontabilità tra strategie, piuttosto che l’adozione di architetture più complesse. Infine, la ricerca sugli iperparametri è stata deliberatamente limitata a combinazioni ritenute significative e realistiche, senza perseguire l’eshaustività esplorativa.

Alla luce dei risultati ottenuti, le prospettive di ricerca riguardano principalmente due

direzioni. Da un lato, sarà importante approfondire l'utilizzo di modelli generativi più espressivi — come GAN o diffusion models — per la creazione di dataset sintetici che siano in grado di catturare la complessità semantica e la variabilità intra- e inter-soggetto tipiche dei dati clinici. Un dataset di consenso costruito con maggiore fedeltà alle distribuzioni originali potrebbe migliorare significativamente l'efficacia dei paradigmi distillativi, soprattutto in scenari non-IID spinti. Dall'altro lato, una direzione di ricerca particolarmente promettente riguarda l'esplorazione di approcci ibridi, che combinino la robustezza delle tecniche federate con la leggerezza e modularità della distillazione.

Un ulteriore sviluppo futuro riguarderà la valutazione sistematica degli aspetti legati alla privacy, che in ambito clinico costituiscono un requisito imprescindibile. Sebbene le tecniche federate riducano già il rischio di esposizione dei dati sensibili, non tutte garantiscono lo stesso livello di protezione rispetto alle informazioni latenti condivise o ai gradienti trasmessi. In prospettiva, sarà quindi necessario analizzare in modo comparativo il grado di tutela offerto dalle diverse strategie — in particolare tra modelli federati e distillativi — e identificare quali meccanismi di aggregazione o distillazione preservino in maniera più efficace la privacy dei pazienti senza compromettere le prestazioni predittive.

I risultati ottenuti in questa tesi mostrano che non esiste una soluzione unica per l'apprendimento federato in ambito clinico, ma che l'efficacia delle diverse strategie dipende strettamente dalla coerenza tra modello, tecnica di aggregazione e caratteristiche dei dati. Le strategie classiche, come FedAvg e FedProx, hanno confermato la loro solidità anche in contesti non-IID, dimostrandosi adatte a scenari in cui la semplicità e la stabilità sono elementi prioritari.

Le tecniche di distillazione hanno evidenziato un potenziale interessante, in particolare per quanto riguarda la modularità e la protezione della privacy, ma hanno anche mostrato una maggiore sensibilità alla qualità del dataset di consenso e alla variabilità tra domini. Nei casi più complessi, come IRDS, la distillazione ha faticato a mantenere una buona generalizzazione, suggerendo che questi approcci richiedono ulteriori affinamenti per essere efficaci in contesti clinici reali.

Nel complesso, l'analisi condotta ha permesso di evidenziare punti di forza e criticità dei diversi paradigmi, offrendo indicazioni utili per orientare scelte progettuali future. In particolare, gli approcci ibridi che combinano elementi di federated learning e distillazione rappresentano una prospettiva promettente, a condizione che siano supportati da meccanismi di adattamento e valutazione coerenti con la complessità del dominio.

Bibliografia

- [1] D. J. BEUTEL, T. TOPAL, A. MATHUR, X. QIU, J. FERNANDEZ-MARQUES, Y. GAO, L. SANI, K. H. LI, T. PARCOLLET, P. P. B. DE GUSMÃO, AND N. D. LANE, *Flower: A friendly federated learning research framework*, 2022.
- [2] A. ESPOSITO, Y. MOGHBELAN, I. ZYRIANOFF, AND M. D. FELICE, *Transfer-informed variational autoencoder for federated learning on imbalanced iot data*, (2025). accepted in IEEE International Conference on Computer Communications: Call for Demos and Posters, INFOCOM 2025).
- [3] H. FACE, *Dataset card for flwrlabs/femnist*. <https://huggingface.co/datasets/flwrlabs/femnist>. Accessed 2025-08-01.
- [4] C. HE, M. ANNAVARAM, AND S. AVESTIMEHR, *Group knowledge transfer: Federated learning of large cnns at the edge*, 2020.
- [5] G. HINTON, O. VINYALS, AND J. DEAN, *Distilling the knowledge in a neural network*, 2015.
- [6] A. G. HOWARD, M. ZHU, B. CHEN, D. KALENICHENKO, W. WANG, T. WEYAND, M. ANDREETTO, AND H. ADAM, *Mobilenets: Efficient convolutional neural networks for mobile vision applications*, 2017.
- [7] S. ITAHARA, T. NISHIO, Y. KODA, M. MORIKURA, AND K. YAMAMOTO, *Distillation-based semi-supervised federated learning for communication-efficient collaborative training with non-iid private data*, IEEE Transactions on Mobile Computing, 22 (2023), p. 191–205.
- [8] X. JIANG, Y. GUO, M. HU, R. JIN, H. PHAN, J. ALBERTS, AND R. LIU, *Federated joint learning of robot networks in stroke rehabilitation*, 2024.
- [9] P. KAIROUZ, H. B. MCMAHAN, B. AVENT, A. BELLET, M. BENNIS, A. N. BHAGOJI, K. BONAWITZ, Z. CHARLES, G. CORMODE, R. CUMMINGS, R. G. L. D’OLIVEIRA, H. EICHNER, S. E. ROUAYHEB, D. EVANS, J. GARDNER, Z. GARRETT, A. GASCÓN, B. GHAZI, P. B. GIBBONS, M. GRUTESER, Z. HARCHAOUI, C. HE, L. HE, Z. HUO, B. HUTCHINSON, J. HSU, M. JAGGI, T. JAVIDI, G. JOSHI, M. KHODAK, J. KONEČNÝ, A. KOROLOVA, F. KOUSHANFAR, S. KOYEJO, T. LEPOINT, Y. LIU, P. MITTAL, M. MOHRI, R. NOCK, A. ÖZGÜR, R. PAGH, M. RAYKOVA, H. QI, D. RAMAGE, R. RASKAR, D. SONG, W. SONG, S. U. STICH, Z. SUN, A. T. SURESH, F. TRAMÈR, P. VEPAKOMMA, J. WANG, L. XIONG, Z. XU, Q. YANG, F. X. YU, H. YU, AND S. ZHAO, *Advances and open problems in federated learning*, 2021.

- [10] D. P. KINGMA AND M. WELLING, *Auto-encoding variational bayes*, 2022.
- [11] D. LI AND J. WANG, *Fedmd: Heterogenous federated learning via model distillation*, 2019.
- [12] T. LI, A. K. SAHU, A. TALWALKAR, AND V. SMITH, *Federated learning: Challenges, methods, and future directions*, IEEE Signal Processing Magazine, 37 (2020), p. 50–60.
- [13] T. LI, A. K. SAHU, M. ZAHEER, M. SANJABI, A. TALWALKAR, AND V. SMITH, *Federated optimization in heterogeneous networks*, 2020.
- [14] T. LIN, L. KONG, S. U. STICH, AND M. JAGGI, *Ensemble distillation for robust model fusion in federated learning*, 2021.
- [15] H. B. MCMAHAN, E. MOORE, D. RAMAGE, S. HAMPSON, AND B. A. Y ARCAS, *Communication-efficient learning of deep networks from decentralized data*, 2023.
- [16] A. MIRON, N. SADAWI, W. ISMAIL, H. HUSSAIN, AND C. GROSAN, *Intellirehabds (irds)—a dataset of physical rehabilitation movements*, Data, 6 (2021).
- [17] A. PASZKE, S. GROSS, F. MASSA, A. LERER, J. BRADBURY, G. CHANAN, T. KILLEEN, Z. LIN, N. GIMELSHEIN, L. ANTIGA, A. DESMAISON, A. KÖPF, E. YANG, Z. DEVITO, M. RAISON, A. TEJANI, S. CHILAMKURTHY, B. STEINER, L. FANG, J. BAI, AND S. CHINTALA, *Pytorch: An imperative style, high-performance deep learning library*, 2019.
- [18] N. RAZFAR, R. KASHEF, AND F. MOHAMMADI, *Psa-fl-cdm: A novel federated learning-based consensus model for post-stroke assessment*, Sensors, 24 (2024).
- [19] S. REDDI, Z. CHARLES, M. ZAHEER, Z. GARRETT, K. RUSH, J. KONEČNÝ, S. KUMAR, AND H. B. MCMAHAN, *Adaptive federated optimization*, 2021.
- [20] A. ROMERO, N. BALLAS, S. E. KAHOU, A. CHASSANG, C. GATTA, AND Y. BENGIO, *Fitnets: Hints for thin deep nets*, 2015.
- [21] M. SANDLER, A. HOWARD, M. ZHU, A. ZHMOGINOV, AND L.-C. CHEN, *Mobilenetv2: Inverted residuals and linear bottlenecks*, 2019.
- [22] K. SOHN, H. LEE, AND X. YAN, *Learning structured output representation using deep conditional generative models*, in Advances in Neural Information Processing Systems, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, eds., vol. 28, Curran Associates, Inc., 2015.
- [23] UNIVERSITÀ DI BOLOGNA, *I-trophyts: Iot and humanoid robotics for autonomic physio-therapeutic monitoring, coaching and supervising in smart spaces*. Progetto PRIN 2022, 2022. <https://site.unibo.it/itrophyts/en/objectives>.
- [24] Z. ZHU, J. HONG, AND J. ZHOU, *Data-free knowledge distillation for heterogeneous federated learning*, 2021.